Documentation de l'outil

Construction de tables d'expérience
Package `tablexp`, méthodes, vérifications et application

Une table d'expérience est l'estimation d'une transition d'un modèle multi-états

Établie le 24/10/2026 | Version 0.1 | Statut : Projet | Package `tablexp` 0.0.0.9000

Objet du document

Ce document est la documentation de l'**outil de construction de tables d'expérience** `outil_tables`. Il couvre le package R `tablexp`, qui porte toute la logique quantitative, les vérifications qui l'accompagnent (batteries de tests, contrôles du dépôt, plateforme de calcul) et, quand elle existera, l'application interactive qui l'exploite. Une *table d'expérience* est construite à partir des observations d'un portefeuille d'assurés : pour un risque donné, l'outil estime le taux d'une *transition* (vivant → décédé, en cours → racheté…) selon un ou plusieurs axes, le lisse ou le positionne sur une table de référence, le ferme aux âges extrêmes, le valide, et restitue la table avec tout ce qui permet de la rejouer et de la relire.

Les destinataires des tables et des rapports sont, par priorité, les hypothèses biométriques de la meilleure estimation (*best estimate*) Solvabilité II, que l'ACPR revoit en contrôle, puis la certification d'une table d'expérience par un actuaire indépendant agréé (`docs/exigences.md`, E1.2). Le document doit permettre à un relecteur externe, sans lire le code, de savoir ce qui a été calculé, avec quelles hypothèses, et de rejouer le calcul (E3.7). Les textes qui font foi sont recensés par le cahier des charges (`docs/exigences.md`, § 2.4) ; ils ne sont cités dans le présent document qu'une fois établis sur la source (issue #13).

Il est organisé en une partie par module du package (E3.1). La **partie I** présente l'outil : forme du package et graphe des modules, services transverses, reproductibilité, batteries de vérification et plateforme de calcul, contrôles du dépôt. La **partie II** décrit le point d'entrée unique et le plan de construction. Les parties suivantes suivent l'ordre des modules du moteur : données, exposition, taux bruts, lissage, positionnement, table prospective, fermeture, validation, rapport, puis le simulateur, qui produit les jeux de données sous une loi vraie connue. La dernière partie rassemble l'index des fonctions exportées, la traçabilité des exigences et les références. Chaque partie de module se termine par le tableau des fonctions du module, qui relie la description de la méthode à son implémentation ; un module qui n'existe pas encore a déjà sa partie, avec un tableau vide.

Le document ne porte aucun chiffre issu d'une exécution (E3.4) : les résultats d'une construction sont dans son rapport figé et sa fiche de construction, et les bilans des vérifications dans la sortie des batteries. Ses conventions de rédaction sont fixées par `docs/latex/CONVENTIONS.md` ; le vocabulaire est celui du glossaire du projet (`CONTEXT.md`).

Élément	Référence
Cahier des charges	`docs/exigences.md` (exigences E1.1 à E5.5)
Décisions d'architecture	`docs/adr/` (ADR 0002 à 0009)
Moteur de calcul	package `tablexp` (`R/`), packages de priorité `base` seulement (ADR 0002)
Interface	`app/app.R`, `app/affichage.R` (à venir ; aucun calcul)
Version de R	4.3.3 ; `DESCRIPTION` déclare `Depends: R (>= 4.3.3)`
Plateforme de référence	conteneur `rocker/r-ver:4.3.3` épinglé par empreinte, BLAS et LAPACK de référence (ADR 0008 ; section 4.3)
Données	aucune donnée réelle : jeux simulés sous une loi vraie et une graine

Sommaire

Partie I — Présentation de l'outil 6

1 Architecture du package et graphe des modules 6
1.1 Principe de séparation . . . 6
1.2 Fichiers du package . . . 6
1.3 Graphe d'appels autorisés . . . 7

2 Services transverses : `R/outils.R` 10
2.1 Refus : une condition R typée . . . 10
2.2 Avertissements métier . . . 10
2.3 Fonctions du module . . . 11

3 Reproductibilité et aléa 12
3.1 Exigence et portée . . . 12
3.2 Générateur fixé . . . 13
3.3 Exécution sous graine locale . . . 13
3.4 Vérifications . . . 14

4 Batteries de vérification et plateforme de CI 14
4.1 Trois batteries . . . 14
4.2 Harnais . . . 15
4.3 Plateforme de référence . . . 17
4.4 Jobs de la CI . . . 18
4.5 Références de non-régression . . . 19

5 Contrôles du dépôt 26
5.1 Scripts de contrôle . . . 26
5.2 Absence de Java . . . 28

Partie II — Point d'entrée et plan de construction 29

6 Point d'entrée unique : `R/construire_table.R` 29
6.1 Plan de construction et validation . . . 29
6.2 Exécution . . . 31
6.3 Objet résultat . . . 32
6.4 Rejeu outillé . . . 32
6.5 Vérifications . . . 38
6.6 Fonctions du module . . . 39

Partie III — Module `donnees` 41

7 Données, rôles et contrôles : `R/donnees.R` 41
7.1 Conventions de temps . . . 41
7.2 Catalogue des rôles . . . 42

7.3 Contrôles, refus et exclusions . . . 43
7.4 Unité d'observation et fusion par tête . . . 46
7.5 Table de référence . . . 48
7.6 Vérifications . . . 48
7.7 Fonctions du module . . . 48

Partie IV — Module `exposition` 52

8 Exposition et événements : `R/exposition.R` 52
8.1 Recette d'exposition et conventions d'âge . . . 52
8.2 Troncature à gauche et censure à droite . . . 54
8.3 Découpage par année . . . 56
8.4 Vérifications . . . 56
8.5 Fonctions du module . . . 58

Partie V — Module `brut` 60

9 Taux bruts : `R/brut.R` 60
9.1 Étape `brut` du plan . . . 60
9.2 Estimateur de Poisson . . . 61
9.3 Estimateur binomial . . . 62
9.4 Estimateur produit-limite (`kaplan_meier`) . . . 64
9.5 Statuts des grandeurs restituées . . . 67
9.6 États des cellules et avertissements . . . 68
9.7 Résultat, paramètres et diagnostics . . . 69
9.8 Vérifications . . . 69
9.9 Constats de simulation : retrouver la loi vraie . . . 71
9.10 Fonctions du module . . . 75

Partie VI — Module `lissage` 77

10 Lissage : `R/lissage.R` 77
10.1 Fonctions du module . . . 77

Partie VII — Module `positionnement` 78

11 Positionnement sur une table de référence : `R/positionnement.R` 78
11.1 Fonctions du module . . . 78

Partie VIII — Module `prospective` 79

12 Tables prospectives : `R/prospective.R` 79
12.1 Fonctions du module . . . 79

Partie IX — Module `fermeture` 80

13 Fermeture aux âges extrêmes : `R/fermeture.R` 80
13.1 Fonctions du module . . . 80

Partie X — Module `validation` 81

14 Validation et suivi d'expérience : `R/validation.R` 81
14.1 Fonctions du module 81

Partie XI — Module `rapport` 82

15 Fiche de construction et empreintes : `R/rapport.R` 82
15.1 Fiche de construction 82
15.2 Empreinte des données 86
15.3 Empreintes du résultat et du code 89
15.4 Texte du plan et de la fiche 92
15.5 Vérifications 96
15.6 Fonctions du module 98

Partie XII — Module `simulation` 103

16 Simulateur : `R/simulation.R` 103
16.1 Loi vraie et tables synthétiques 103
16.2 Portefeuille simulé 104
16.3 Constats de simulation 106
16.4 Vérifications 107
16.5 Fonctions du module 108

Partie XIII — Index et traçabilité 109

17 Index et traçabilité 109
17.1 Index des fonctions exportées 109
17.2 Traçabilité des exigences 110
17.3 Références 111

Partie I

Présentation de l'outil

Architecture du package et graphe des modules

1.1 Principe de séparation

L'outil sépare strictement le *calcul* de l'*affichage*. La règle est la suivante : si un résultat peut être calculé indépendamment de l'interface, son calcul figure dans le package `tablexp` (E4.3). Le moteur reçoit un `data.frame` et la déclaration des *rôles* de ses colonnes, jamais un fichier ; la lecture des fichiers, et les conversions de type qui l'accompagnent, relèvent d'une couche mince hors du moteur, qui n'existe pas encore (E4.4). Le moteur refuse un type incompatible, il ne le convertit pas.

Emplacement	Contenu	Interdit
`R/` (package `tablexp`)	toute la logique quantitative : rôles et validation des données, recettes d'exposition, estimation, lissage, positionnement, prospective, fermeture, validation, simulation, quantités des rapports et empreintes	toute primitive Shiny (`input$`, `reactive`, `render*`) ; tout package hors priorité `base` sans ADR de méthode ; toute dépendance à Java
`app/app.R` (à venir)	déclaration des rôles, édition du plan, appel du point d'entrée sur clic du bouton « Relancer les calculs », affichage	tout calcul ; tout appel d'une fonction non exportée (`tablexp:::`)
`app/affichage.R` (à venir)	formatage, tableaux HTML, tracés à partir des quantités du résultat	tout calcul
`tests/`	batteries, harnais, scripts de contrôle	—

L'application vit hors du package, sous `app/`, et non dans un fichier `R/affichage.R` : tout fichier de `R/` est installé dans l'espace de noms du package (section 1.2), si bien qu'un fichier d'affichage placé là entrerait dans le moteur et rendrait circulaire le contrôle de frontière (ADR 0007, point 1.3). Le dossier `app/` et son script de frontière n'existent pas encore : ils viennent avec la branche de l'application.

1.2 Fichiers du package

Le package `tablexp` est à la racine du dépôt (ADR 0007, point 1.1) :

— `DESCRIPTION` : `Package: tablexp`, `Depends: R (>= 4.3.3)` (décision M44), aucun champ `Imports`, `Suggests` ni `LinkingTo`, `Encoding: UTF-8`, `License: file LICENSE` ; le contrôle de la liste blanche (section 5) en fait une règle vérifiée ;

— `NAMESPACE`, écrit à la main : une directive `export` par fonction, sans `exportPattern` ; les fonctions exportées sont les points d'entrée et les fonctions que l'application appelle (contrôles de saisie à sens métier, constructeurs d'objets partagés), recensées dans l'index (section 17.1) ; aucune méthode S3 n'est exportée ;

— `R/` : un fichier par module et deux fichiers transversaux (tableau 1) ;

— `.Rbuildignore` : exclut de l'archive du package `app/`, `docs/`, `tests/`, `.claude/`, `.github/` et les fichiers de pilotage du dépôt.

Aucun fichier `Rd` ni `roxygen2` : la documentation de référence est le présent document, tenu concordant avec le code par la batterie de concordance (section 4 ; E3.3).

`R/` ne contient que du code du package. Une installation depuis l'arbre de travail (`R CMD INSTALL <dossier>`), qui est celle des batteries, n'applique pas `.Rbuildignore` : tout fichier de `R/` est installé, même s'il y est exclu (constat : un fichier `R/b.R` exclu par `.Rbuildignore` est présent dans le package installé ; ADR 0007, mesures). Un fichier étranger au moteur ne peut donc pas être rangé dans `R/`.

Table 1 : Fichiers de `R/`, dans l'ordre des modules (ADR 0007, points 1.2 et 2.1).

Fichier	Contenu	État
`R/outils.R`	services transverses sans contenu métier : graine locale, refus et avertissements (section 2) ; n'appelle aucun module	présent
`R/donnees.R`	rôles, contrôles, refus et exclusions, unité d'observation, table de référence (section 7)	présent
`R/exposition.R`	recettes d'exposition, conventions d'âge, troncature et censure (section 8)	présent
`R/brut.R`	taux bruts, intervalles et statuts (section 9)	présent
`R/lissage.R`	lissages (section 10)	à créer
`R/positionnement.R`	positionnement sur une table de référence (section 11)	à créer
`R/prospective.R`	tables prospectives (section 12)	à créer
`R/fermeture.R`	fermeture aux âges extrêmes (section 13)	à créer
`R/validation.R`	validation et suivi d'expérience (section 14)	à créer
`R/rapport.R`	quantités des rapports, fiche de construction, empreintes (section 15)	présent
`R/simulation.R`	simulateur (section 16) ; hors de la chaîne du moteur	présent
`R/construire_table.R`	point d'entrée unique, validation du plan, orchestration seule, aucun calcul (section 6) ; rejeu outillé d'une fiche, qui contrôle puis appelle le point d'entrée sans exécuter d'étape (section 6.4)	présent

1.3 Graphe d'appels autorisés

Les modules sont ordonnés : `outils`, `donnees`, `exposition`, `brut`, `lissage`, `positionnement`, `prospective`, `fermeture`, `validation`, `rapport`. Un module ne peut appeler que des fonctions définies dans les modules situés **strictement avant lui** dans cet ordre (ADR 0007, point 2.1). Deux fichiers sont hors de la chaîne : `simulation`, qui ne peut appeler que `donnees` et `outils` et qu'**aucun module du moteur n'appelle**, et `construire_table`, qui peut appeler tous les modules **sauf** `simulation`. La figure 1 et le tableau 2 donnent ce graphe.

Deux raisons fondent cette règle. Un ordre sans cycle rend chaque module lisible et vérifiable avec ses seuls modules amont : une anomalie d'un module ne peut atteindre que les modules situés après lui. Et le

simulateur, qui produit les jeux sur lesquels le moteur est testé, ne partage pas le code qu'il sert à tester : s'il pouvait appeler `exposition`, une erreur de la recette d'exposition se retrouverait à l'identique dans les données simulées et dans leur estimation, et la vérification ne la verrait pas (ADR 0007, « Options écartées »). Le simulateur reproduit le même modèle, pas le même code. Une seule exception à la lecture « en aval seulement » est assumée : `positionnement` peut appeler `lissage`, parce que la méthode 4 de positionnement est un lissage de la table du moment avec dérive de la référence (E2.12) ; le sens inverse est interdit, `lissage` n'appelant jamais `positionnement`.

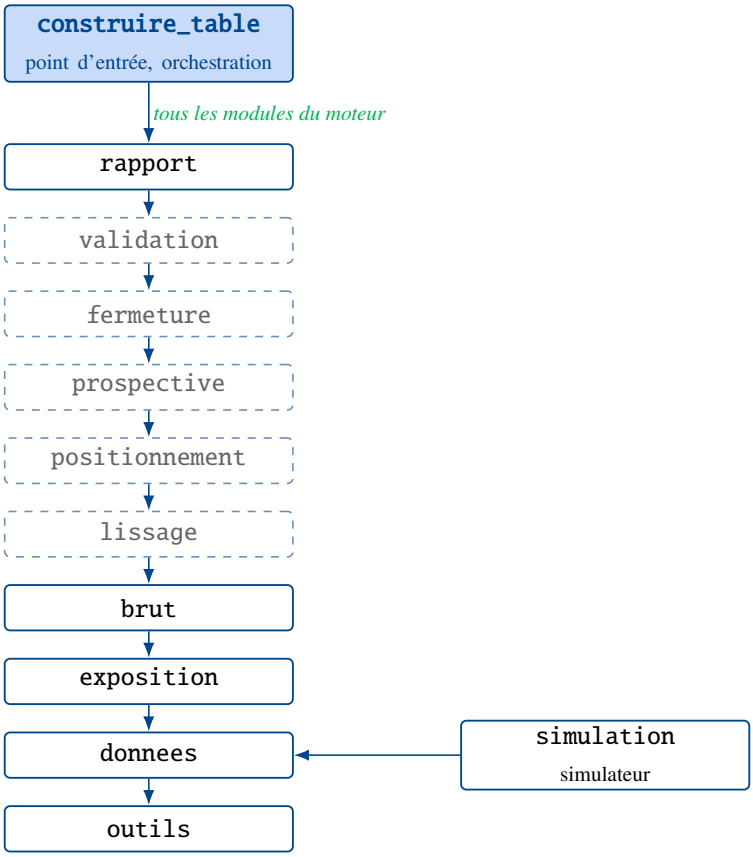


Figure 1 : Graphe d'appels autorisés des modules (ADR 0007, point 2.1). Une flèche de A vers B signifie que A peut appeler B et, la règle étant « strictement avant », tout module situé après B dans le sens des flèches. Blocs pleins : fichiers présents dans `R/` ; blocs en pointillé : modules prévus, fichiers absents. `simulation` n'est appelé par aucun module du moteur, ni par `construire_table` ; `lissage` n'appelle jamais `positionnement`.

Table 2 : Appels autorisés, par module appelant (ADR 0007, point 2.1 ; constante `APPELS_AUTORISES` de `tests/controles/graphe_modules.R`).

Module appelant	Peut appeler
`outils`	aucun module (fonctions des packages de priorité `base` seulement)
`donnees`	`outils`
`exposition`	`donnees`, `outils`
`brut`	`exposition` et les modules amont
`lissage`	`brut` et les modules amont
`positionnement`	`lissage` et les modules amont
`prospective`	`positionnement` et les modules amont
`fermeture`	`prospective` et les modules amont
`validation`	`fermeture` et les modules amont

Module appelant	Peut appeler
`rapport`	tous les modules ci-dessus
`simulation`	`donnees` et `outils` seulement
`construire_table`	tous les modules sauf `simulation`

Contrôle par script

Le graphe est vérifié par `tests/controles/graphe_modules.R`, en R de base (`codetools`, de priorité `recommended`, est exclu par l'ADR 0002), dans le job « Tests (R 4.3.3) » de la CI (ADR 0007, point 2.2) :

1. chaque fichier `R/<module>.R` est lu par `parse()` ; un fichier dont le nom n'est pas un module connu est un écart ;
2. les affectations de premier niveau (`nom <- ...` ou `nom = ...`, fonctions comme constantes) donnent la table « symbole → module » ; un symbole défini dans deux modules est un écart ;
3. pour chaque définition, les noms qu'elle emploie (`all.names()`), croisés avec cette table, donnent les arêtes entre modules ; toute arête absente du tableau 2 est un écart, imprimé avec le module appelant, le module appelé et les deux symboles ;
4. les appels dynamiques interdits (ci-dessous) sont cherchés dans tout l'arbre syntaxique.

Le script imprime toutes les arêtes trouvées et tous les écarts, puis sort avec le code 1 s'il en a trouvé au moins un. Ses limites sont écrites dans son en-tête : un nom local homonyme d'un symbole d'un autre module compte comme une arête (faux positif possible, jamais d'arête manquée par homonymie) ; un code de premier niveau qui n'est pas une affectation est un écart ; les appels vers les packages de base ne sont pas contrôlés ici, mais par la liste blanche de `DESCRIPTION`. Le script est lui-même testé sur des témoins (`tests/unitaires/test_controles.R`) : il échoue, entre autres, quand `exposition` appelle `brut`, quand `donnees` appelle `simulation`, quand `lissage` appelle `positionnement` et quand `construire_table` appelle `simulation`, et il passe sur un graphe conforme où `positionnement` appelle `lissage`.

Appels dynamiques interdits

Un appel dont la cible est calculée à l'exécution (`do.call("f", ...)`, `get("f")`) n'apparaît pas dans `all.names()` : il créerait une arête invisible pour le contrôle. Le script interdit donc dans `R/` (ADR 0007, point 2.3, liste élargie à la mise en œuvre et consignée dans l'annotation du 16/10/2026) :

— `do.call()` dont l'argument `what`, apparié par `match.call()` (position ou nom), n'est ni un symbole ni une fonction anonyme (chaîne, `paste0(...)...`) ; `do.call(f, args)` avec `f` un symbole désignant une fonction reste permis ;

— toute occurrence, appel ou fonction passée comme objet (`lapply(x, get)`), de `match.fun`, `get`, `get0`, `mget`, `getFromNamespace`, `getExportedValue`, `getFunction`, `asNamespace` et `getNamespace` ;

— `eval()` ou `evalq()` d'un `parse()`, `str2lang()`, `str2expression()`, `call()` ou `as.call()` ;

— l'opérateur `:::` et toute référence qualifiée `tablexp::` ;

— les formes qualifiées `base::`, `utils::` et `methods::` de ce qui précède, lues comme la forme nue.

Une fonction passée en argument puis appelée dans le corps de la fonction qui la reçoit crée une arête que le script attribue à l'appelant qui l'a passée : c'est voulu, la dépendance existe bien là. Les tests, seuls, appellent les fonctions non exportées par `tablexp:::` (ADR 0007, point 11.3).

Services transverses : R/outils.R

R/outils.R porte les services sans contenu métier partagés par tous les modules : l'exécution sous graine locale (section 3), la forme des refus et celle des avertissements (ADR 0007, point 1.2), ainsi que le contrôle de la forme exacte d'une valeur reçue, forme_exacte() (type de base et attributs admis, section 6.1 ; issue #23), et les deux contrôles de forme qui s'y appuient, partagés par les modules : liste_aux_noms() (liste nue aux noms exacts) et chaine_unique() (chaîne unique sans attribut, ni NA ni vide ; issue #48), ainsi que la règle des noms complets, noms_complets() (noms présents, sans NA ni nom vide), écrite une seule fois pour le contrôle des rôles et pour celui du plan (issue #22). Il n'appelle aucun module. Sans lui, la fonction de graine locale serait dupliquée dans le simulateur et dans le futur bootstrap par têtes, et chaque module inventerait sa forme de refus.

2.1 Refus : une condition R typée

Le moteur ne devine pas : quand une entrée ne permet pas de construire, il *refuse* (E2.3, E2.7). Un refus est une **condition R typée, jamais un objet retourné** (ADR 0007, point 6.4). refus_tablexp() construit la condition, de classe c("refus_tablexp", "error", "condition"), avec quatre champs :

— module : le module qui refuse ;

— motif : un code court stable, en minuscules ASCII, chiffres et soulignés, conforme au motif MOTIF_CODE_REFUS (^[a-z][a-z0-9_]*$) ;

— message : la phrase qui dit ce qui manque (E5.5) ;

— details : une liste (rôle, colonne, étape, paramètre, décompte, premiers numéros de ligne).

refuser() la lève par stop() et ne rend jamais la main. Un argument mal formé de refus_tablexp() (module, motif ou message qui n'est pas une chaîne unique sans attribut, ni NA ni vide, au sens de chaine_unique(), chaîne nommée comprise ; motif non conforme ; détails qui ne sont pas une liste) n'est pas un refus mais une erreur de programmation du package, signalée par stop(). Les tests vérifient le motif, le décompte et les numéros par tryCatch() ; l'application attrapera la condition et affichera son message. Une condition est préférée à un objet portant ok = FALSE, forme d'outil_USP : un tel objet peut être lu par mégarde comme une construction partielle, une condition ne le peut pas. Les codes de refus de chaque module sont recensés dans la partie du module.

2.2 Avertissements métier

Un avertissement métier signale ce qui n'empêche pas la construction mais doit être lu : unité d'observation par défaut, exclusion appliquée, relevé d'incohérence non bloquant. Il est **inscrit dans l'objet résultat, jamais émis par warning()** (ADR 0007, point 6.5) : l'objet est la seule source du rapport figé et de l'application ; un warning() serait perdu par l'application, absent de la fiche et parasite dans les batteries. avertissements_vides() crée l'accumulateur, un data.frame de colonnes etape, code, message et nombre ; nombre, un dénombrement (lignes, cellules, têtes), est stocké en integer, positif ou nul (issue #24). avertir() lui ajoute une ligne, et refuse, par une erreur de programmation (stop(), sans conversion), un code qui n'est pas une chaîne unique sans attribut, ni NA ni vide, au sens de chaine_unique() (une chaîne nommée ou munie d'une classe est refusée ; issue #60), ou qui n'est pas

conforme à MOTIF_CODE_REFUS, et un nombre qui n'est pas un nombre unique, fini, entier, positif ou nul et au plus égal à .Machine$integer.max ; sinon il le stocke par as.integer(). Chaque fonction d'étape rend ses avertissements ; le point d'entrée les rassemble dans le champ avertissements de l'objet et de la fiche. Les codes de chaque étape sont recensés dans la partie de son module (sections 7.4, 8 et 9.6).

2.3 Fonctions du module

Table 3 : Définitions de premier niveau de R/outils.R.

Nom	Statut	Entrées	Sortie et rôle
GENERATEUR	constante	—	générateur fixé (section 3.2) : Mersenne-Twister, inversion pour la loi normale, rejet pour sample()
MOTIF_CODE_REFUS	constante	—	motif d'un code de refus ou d'avertissement : ^[a-z][a-z0-9_]*$
refus_tablexp()	interne	module, motif, message, details (liste, vide par défaut)	condition de classe refus_tablexp, sans la lever ; stop() si un argument est mal formé (section 2.1)
refuser()	interne	mêmes arguments	lève le refus ; ne rend jamais la main
forme_exacte()	interne	x, types (types de base admis), attributs (noms d'attributs admis, aucun par défaut)	TRUE si typeof(x) est pris dans types et si x ne porte aucun attribut hors de attributs, FALSE sinon, jamais NA ; un attribut admis n'est pas requis et sa valeur n'est pas contrôlée ; ne refuse pas, l'appelant refuse sous son propre motif (section 6.1 ; issue #23)
liste_aux_noms()	interne	x, noms (noms attendus)	TRUE si x est une liste nue (forme_exacte() de type list, seul attribut admis : les noms ; ni data.frame, ni classe) d'autant d'éléments que noms, aux mêmes noms une fois triés par sort(method = "radix"), indépendamment de la locale, FALSE sinon (liste sans noms comprise) ; la longueur est comparée à part, sort() retirant un nom NA ; le contenu des éléments, lus par [[ ]], est contrôlé par l'appelant, qui refuse sous son propre motif (issue #48)
chaine_unique()	interne	v	TRUE si v est de forme exacte character, sans aucun attribut (noms compris), de longueur 1, ni NA ni vide, FALSE sinon, jamais NA ; ne refuse pas (issue #48)

Nom	Statut	Entrées	Sortie et rôle
`noms_complets()`	interne	`noms` (`names(x)`)	`TRUE` si les noms sont présents (non `NULL`), sans `NA` et non vides, `FALSE` sinon, jamais `NA` ; prédicat sur les noms seuls : la forme de l'objet, les doublons et la liste vide sans noms relèvent de l'appelant ; seul endroit où s'écrit la règle, partagée par `controler_roles()` et `liste_noms_complets()` (issues #22, #60 et #62)
`avertissements_vides()`	interne	—	accumulateur vide des avertissements métier, colonne `nombre` en `integer` (section 2.2 ; issues #9 et #24)
`avertir()`	interne	accumulateur, `etape`, `code`, `message`, `nombre`	accumulateur augmenté d'une ligne, message marqué UTF-8 par `texte_moteur()`, sans `warning()` ; `stop()` si le code (chaîne unique au sens de `chaine_unique()`, conforme à `MOTIF_CODE_REFUS`) ou le nombre est mal formé (section 2.2 ; issues #9, #24, #50 et #60)
`texte_moteur()`	interne	`x`, chaîne ou liste (`data.frame` compris)	le même objet, chaînes et noms non ASCII du moteur marqués UTF-8 en toute locale d'installation et d'exécution ; réservée au texte du moteur, jamais aux données, rôles ou plan de l'utilisateur (section 4.2, « Locale de l'installation » ; issue #50)
`sous_graine()`	interne	`graine` (entier explicite, sans défaut ni attribut), `expr`	valeur de `expr` évaluée sous la graine et `GENERATEUR`, état du générateur de l'appelant restauré ; refus `graine_invalide` (section 3.3)

Reproductibilité et aléa

3.1 Exigence et portée

À données, rôles, plan et graine identiques, le programme produit des résultats identiques **au bit près sur une même machine** ; toute simulation passe par une graine locale explicite sous un générateur fixé, consignée dans le résultat, l'état global du générateur étant restauré à la sortie ; aucun horodatage n'entre dans la partie du résultat comparée lors d'un rejeu (E4.6). La présente section décrit les deux mécanismes du socle : le générateur fixé et l'exécution sous graine locale. La consignation de la graine et du générateur dans la fiche de construction et le rejeu de la fiche sont décrits avec la fiche (section 15.1).

Statut. L'identité au bit près est un *constat* des batteries (section 3.4), établi sur la machine qui les exécute ; ce n'est pas une propriété démontrée. L'identité entre deux machines n'est **pas garantie** : deux processeurs différents pourraient différer au dernier bit, même dans le même conteneur, si la bibliothèque mathématique du système choisit ses variantes de calcul selon le processeur (réserve de l'ADR 0008 : non observée sur trois modèles de processeur munis de FMA et d'AVX2, ouverte pour un exécuteur sans FMA ; annotation du 19/10/2026). C'est pourquoi les références de non-régression sont produites sur une plateforme unique et comparées à tolérance (sections 4.3 et 4.5).

3.2 Générateur fixé

Le générateur est la constante `GENERATEUR` de `R/outils.R` : `kind = "Mersenne-Twister"`, `normal.kind = "Inversion"`, `sample.kind = "Rejection"`. Ce sont les valeurs par défaut de R 4.3.3 (`RNGkind()` dans une session neuve), fixées en constante pour qu'un changement de défaut d'une version de R ne change pas les tirages en silence. Le générateur uniforme est le Mersenne-Twister de Matsumoto et Nishimura ; les lois normales s'obtiennent par inversion de la fonction de répartition ; la génération uniforme discrète (`sample()`) emploie la méthode par rejet, défaut de R depuis la version 3.6.0, la méthode antérieure par arrondi rendant `sample()` sensiblement non uniforme sur de grandes populations (documentation de R, `?Random`).

Référence. M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`).

Un changement de `GENERATEUR` change tout résultat simulé : c'est un changement de résultat, soumis au visa (ADR 0007, « Conséquences »).

3.3 Exécution sous graine locale

`sous_graine(graine, expr)` est la seule voie par laquelle le package tire un aléa (ADR 0007, point 8.1). Elle procède ainsi :

1. **contrôle de la graine** : un nombre unique, de type de base `integer` ou `double` et **sans aucun attribut** — ni noms, ni dimension, ni classe (`forme_exacte()`, section 2.3 ; issue #43) : une graine non canonique ne doit pas être consignée —, non manquant, fini, entier et de valeur absolue au plus `.Machine$integer.max` ; sinon, et aussi quand la graine est omise, refus `graine_invalide` du module `outils`. Il n'y a jamais de graine par défaut : une graine par défaut serait une graine non déclarée ;
2. **sauvegarde de l'état de l'appelant** : `.Random.seed` de l'environnement global, ou son absence, et le réglage `RNGkind()` ;
3. **tirage** : `set.seed(graine, kind, normal.kind, sample.kind)` sous `GENERATEUR`, puis évaluation de `expr` ;
4. **restauration**, par `on.exit()`, donc aussi après une erreur dans `expr` : le réglage d'abord, parce que `RNGkind()` réécrit `.Random.seed`, puis `.Random.seed` de l'appelant, ou sa suppression s'il n'existait pas.

Les tirages ne dépendent donc que de la graine : ni de l'état du générateur de l'appelant, ni de son réglage, que `sous_graine()` laisse inchangés. Le simulateur (section 16) est, à cette date, la seule fonction du package qui tire un aléa ; il reçoit sa graine en argument et la consigne, avec `GENERATEUR`, dans le jeu simulé.

Limite. La réserve de la méthode normale de Box-Muller d'un appelant, gardée par R hors de `.Random.seed`, n'est pas restaurable ; c'est sans effet sur le moteur, qui tire toujours sous `Inversion` (limite reprise d'`outil_USP`, ADR 0007, point 8.1).

Aucun aléa hors calcul. Les fichiers temporaires, employés par exemple pour calculer une empreinte par `tools::md5sum()`, sont créés par `tempfile()`, qui ne touche pas `.Random.seed` (constat : dans une session neuve, `.Random.seed` n'existe ni avant ni après `tempfile()` et `tools::md5sum()` ; ADR 0007, point 8.2).

3.4 Vérifications

Deux fichiers de tests portent ces propriétés (section 4) :

— `tests/unitaires/test_outils.R` : valeur de `GENERATEUR` ; mêmes tirages que `set.seed()` nu sous le générateur par défaut ; graine entière écrite en double admise ; appelant sous `L'Ecuyer-CMRG` : tirages de `GENERATEUR`, `.Random.seed` et `RNGkind()` de l'appelant restaurés ; absence de `.Random.seed` restaurée ; état restauré après une erreur dans `expr` ; refus `graine_invalide` des graines mal formées ;

— `tests/test_reproductibilite.R` : à graine égale, tirages identiques (`identical()`) dans un même processus, quel que soit l'état du générateur de l'appelant, et dans un autre processus R de la même machine ; à graines différentes, tirages différents. Les tirages comparés couvrent les trois réglages du générateur (`runif()`, `rnorm()`, `sample.int()`) ; le processus fils les écrit en `%.17g`, dix-sept chiffres significatifs, que le processus parent relit : sur la plateforme de référence (glibc), la relecture au bit près est un constat de mesure, non une propriété démontrée (ADR 0011, points 2.4 et 3.2 ; section 15.2). La même batterie vérifie le simulateur et la construction (sections 16 et 15) et compare les plans de référence à leurs références de non-régression (section 4.5).

Ces vérifications sont des constats sur la machine qui exécute la batterie ; sur la CI, c'est la plateforme de référence (section 4.3).

Batteries de vérification et plateforme de CI

4.1 Trois batteries

Les vérifications du code passent par trois batteries en R de base, sans `testthat` (package du CRAN, hors priorité `base`), lancées depuis la racine du dépôt (ADR 0007, point 11.1) :

```
Rscript tests/test_unitaires.R
Rscript tests/test_reproductibilite.R
Rscript tests/concordance_doc_moteur.R --strict
```

La même liste, dans le même ordre, figure dans `CLAUDE.md` (« Commandes »), dans le workflow `.claude/workflows/circuit-technique.js` et dans le job « Tests (R 4.3.3) » de la CI ; un script le contrôle (section 5). Chaque batterie :

— **installe l'arbre de travail** dans une bibliothèque temporaire (`R CMD INSTALL --no-docs -l <bibliothèque> <racine>`, par `installer_tablexp()`, sous une locale UTF-8 : section 4.2), sans installer aucun autre package, puis charge `tablexp` depuis elle et échoue s'il est chargé d'ailleurs : c'est le code de l'arbre qui est testé, jamais un package installé auparavant ;

— affiche `R.version.string` puis, à la fin, une **ligne de bilan** : version de R, nombre de tests, d'échecs, d'échecs attendus et de succès inattendus ;

— **sort avec un code non nul** en cas d'échec ou de succès inattendu (section 4.2).

Une bibliothèque déjà installée, désignée par la variable d'environnement `TABLEXP_BIBLIOTHEQUE`, n'est réutilisée que si l'*empreinte de l'arbre source* inscrite à son installation est égale à celle de l'arbre courant ; sinon la batterie réinstalle et le dit. Cette empreinte est un MD5 (`tools::md5sum()`) de `DESCRIPTION`, de `NAMESPACE` et, s'ils existent, de tous les fichiers de `R/`, `inst/`, `data/` et `src/`, chemins relatifs compris, triés par octets (`empreinte_source()`). Elle empêche de tester un package périmé ; elle n'a aucun rôle dans les résultats.

`tests/test_unitaires.R` exécute chaque fichier `tests/unitaires/test_*.R`, triés par nom, chacun dans son propre environnement, où il est lu par `parse(encoding = "UTF-8")` puis évalué, si bien que ses littéraux non ASCII sont marqués UTF-8 quelle que soit la locale de la session (décision M54) ; `quit()`, `q()` et `bilan()` y sont neutralisés, une erreur au chargement d'un fichier est un échec, et un fichier sans aucune assertion aussi. **`tests/test_reproductibilite.R`** vérifie les identités de la section 3.4, puis celles du simulateur et de la construction, compare les plans de référence à leurs références de non-régression (section 4.5) et contrôle l'effet d'une permutation des lignes de l'entrée de référence (section 15.2, « Représentation canonique » ; issue #39), puis l'aller-retour du texte du plan et de la fiche et le rejeu à partir de ce texte pour les six plans de référence (section 15.4). **`tests/concordance_doc_moteur.R`** confronte le présent document au code installé : toute fonction citée existe ; toute fonction exportée est citée, ou inscrite dans la liste d'attente `tests/concordance_attente.csv` avec renvoi à l'issue qui la documentera, où elle compte en échec attendu ; chaque catalogue recensé par le document (fonctions exportées, définitions de premier niveau de chaque fichier de `R/`, rôles, recettes d'exposition) est égal, nom pour nom, à sa source dans le code ; toute ligne de la liste d'attente désigne un export, un catalogue requis, une entrée d'un catalogue requis déjà écrit (type `entree`, de la forme `<catalogue>:<nom>`, pour une définition ajoutée à un module dont le tableau existe déjà) ou une valeur citée (type `fiche`, ci-dessous) et renvoie à une issue ; une entrée en attente absente du document compte en échec attendu, présente elle est un succès inattendu. Elle compare en outre au package les valeurs que le document cite (issue #70) : chaque constante de version (`VERSION_FICHE` et les quatre premières lignes `ENTETE_CANONIQUE`, `REPRESENTATION_RESULTAT`, `PREMIERE_LIGNE_PLAN`, `PREMIERE_LIGNE_FICHE`) est citée, et toute valeur citée est la valeur courante ; la liste des champs de la fiche (section 15.1) a les champs de `names(fiche)`, dans leur ordre, et chaque identifiant qu'elle cite est un nom de la fiche, un objet de `tablexp`, un estimateur ou un littéral d'une liste blanche du script ; les blocs de lecture sont ceux de `BLOCS_LECTURE_FICHE` (blocs, champs et ordre). Enfin, aucun nom de fonction n'est coupé par `\allowbreak` entre deux `\code{}` : le fragment final serait lu comme une fonction et le nom entier échapperait au contrôle d'existence (issue #84). Le format des citations et des catalogues qu'elle lit est fixé par `docs/latex/CONVENTIONS.md` (section 3) ; les tableaux de fonctions de chaque module en sont la principale application (E4.9). Sans `--strict`, elle rend compte des écarts sans échouer.

4.2 Harnais

Les trois batteries partagent `tests/harnais.R`, repris de la forme d'`outil_USP` (ADR 0007, point 11.2, et annotation du 16/10/2026, point d) :

— `verifier(nom, expr)` : succès si `expr` vaut `TRUE` ; une chaîne décrit l'écart ; toute autre valeur, ou une erreur, est un échec ;

— `echec_attendu(nom, issue, expr, motif, erreur_quelconque)` : **défaut connu**, codé en échec attendu avec renvoi obligatoire à l'issue, de la forme `#N` ; `expr` décrit le comportement *correct*. Tant qu'elle échoue, le test est « attendu » ; **si elle passe, c'est un succès inattendu et la batterie échoue** : le défaut semble corrigé, la marque est retirée et le test devient ordinaire. Trois natures d'échec attendu sont admises et décomptées : par valeur (`expr` rend `FALSE` ou une chaîne), par refus (un `refus_tablexp`, du motif donné s'il l'est) et, seulement si `erreur_quelconque = TRUE`, par une autre

erreur ; toute autre issue est un échec, pour qu'un défaut connu ne masque pas une panne différente ;

— `proche(a, b, rel, abs)` : égalité numérique à tolérance déclarée, $|a - b| \leq abs + rel\,|b|$ élément par élément, sans tolérance par défaut ; rend un écart pour des valeurs non numériques, des longueurs différentes ou des valeurs manquantes à des positions différentes ; exige l'égalité exacte des valeurs infinies ;

— `leve_refus(expr, motif)` : vrai si `expr` lève un `refus_tablexp` du motif donné ;

— `sans_execution(x)` : retire de l'objet résultat le champ des informations d'exécution, seul champ qui change entre deux exécutions identiques, avant `identical()` ;

— `bilan()` : ligne de bilan et code de sortie. Il compte en outre un échec si un espace de noms hors priorité `base`, autre que `tablexp`, est chargé en fin de batterie (`espaces_hors_base()`) : l'image de la CI contient des packages `recommended`, et cette règle empêche un test de s'appuyer sur l'un d'eux à l'insu de la liste blanche (ADR 0008, « Conséquences »).

Locale de l'installation (décision M54 ; ADR 0007, annotation du 17/10/2026). Un package installé sous une locale qui n'est pas UTF-8 garde ses littéraux non ASCII non marqués, qu'une conversion vers UTF-8 dénature dans une session de même locale (constat consigné par l'ADR 0007) ; les chaînes du moteur entrent dans la fiche et dans les références de non-régression. Si la session n'est pas en UTF-8, `installer_tablexp()` lance donc `R CMD INSTALL` sous la première locale UTF-8 disponible parmi `C.UTF-8`, `C.utf8` et `en_US.UTF-8` (`locale_installation()`), et échoue s'il n'y en a aucune ; la locale d'exécution des batteries reste libre. Après le chargement, une sonde (`sonde_utf8()`) vérifie qu'un littéral non ASCII du moteur, la constante `STATUT_EMPREINTE_RESULTAT`, est de l'UTF-8 valide et inchangé par la conversion ; sinon la batterie échoue. Limite admise : une installation hors UTF-8 sous une session UTF-8 passe la sonde, sans effet sur les résultats (constat de l'audit consigné par l'ADR 0007 : le plan de référence R1 aplati sous les deux locales donne le même fichier). La sonde ne protège que les batteries : une garde au chargement du package, pour tout usage, est l'objet de l'issue #25. Le **texte produit par le moteur** (messages d'avertissement, mentions, hypothèses et statuts de l'étape `brut`, unité d'exposition, hypothèse de censure, en-tête du texte du plan et de la fiche) est en outre marqué UTF-8 par `texte_moteur()`, quelle que soit la locale d'installation ou d'exécution : sans ce marquage, un package installé ou exécuté sous la locale C laissait des chaînes non ASCII du moteur sans codage déclaré, et `ecrire_fiche()` refusait la fiche (constat de la revue finale de l'issue #50). Mesuré pour R1 : même texte de fiche, octet pour octet, dans les quatre combinaisons d'installation et d'exécution sous C et C.UTF-8. Le marquage ne vaut que pour le texte du moteur : une chaîne non ASCII non marquée des données, des rôles ou du plan de l'utilisateur reste refusée (`chaine_codage_indetermine`, section 15.2).

Contrôle sous la locale C (issue #66). Les batteries installant toujours `tablexp` sous une locale UTF-8, une régression propre à un package installé et exécuté sous la locale C leur échapperait ; des défauts de cette nature ont déjà été trouvés par lancement manuel (refus d'`ecrire_fiche()`, ci-dessus). Le contrôle `tests/controles/locale_c.R` (tableau 7) installe donc l'arbre de travail sous `LC_ALL=C` et sous la première locale UTF-8 disponible, exécute dans chaque cas, sous la locale de l'installation, les plans de référence R1 à R6 sur l'entrée de référence, écrit, relit et réécrit leur fiche et leur plan, puis compare octet à octet les douze textes obtenus sous les deux locales ; le moindre écart le fait sortir avec un code non nul. Il est une étape bloquante du job « Tests (R 4.3.3) », distincte des trois batteries. **Statut** : l'identité des textes est un *constat* sur la machine qui exécute le contrôle (la plateforme de référence en CI) et sur ces six plans, non une propriété démontrée ; il ne couvre ni une locale latin-1, ni Windows (issue #49), ni des données de l'utilisateur à chaînes non ASCII.

Une **garde de sortie** (`.Last()`) fait sortir avec le code 1 tout processus qui a chargé le harnais et se termine avant la ligne de bilan, même par `quit(status = 0)`. Le harnais est lui-même testé

(`tests/unitaires/test_harnais.R`) : une assertion fausse donne un code de sortie non nul, et un échec attendu qui réussit fait échouer la batterie.

Chaque test nomme la nature de son attendu (ADR 0007, point 11.4 ; E2.20) : valeur calculée à la main (identité exacte, ou tolérance déclarée) ; *recoupement*, comparaison à la valeur que rend, dans la batterie, un appel direct d'une fonction d'un package de priorité `base`, dont la portée est limitée quand le moteur repose sur la même fonction : le test dit alors ce qu'il vérifie (la formule et son branchement, non la fonction de base) ; *contre-vérification*, valeur de référence produite hors CI par un package avancé et écrite en dur avec sa version et sa date ; *constat de simulation*, qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (`CONTEXT.md` ; E2.19).

4.3 Plateforme de référence

Les références de non-régression (section 4.5) sont produites et comparées sur une *plateforme de référence* unique (E4.7 ; ADR 0008) : le job de la CI s'exécute dans un **conteneur `rocker/r-ver:4.3.3` épinglé par empreinte**, dont R charge la **BLAS et la LAPACK de référence** (Netlib), en monothread. Identifiant fixé par l'annotation du 16/10/2026 de l'ADR 0008 (décision M47) :

> image `rocker/r-ver:4.3.3@sha256:`
> `732d15020af326da9e919c07f70ca32bf5d3e409220af32e0a4b6d0a894373o9`
> suivie de la bascule `update-alternatives` vers les paquets `libblas3` et `liblapack3` 3.10.0-2ubuntu1 d'Ubuntu 22.04 ;
> `R version 4.3.3 (2024-02-29)` ;
> BLAS `/usr/lib/x86_64-linux-gnu/blas/libblas.so.3.10.0` ;
> LAPACK `/usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.10.0`, version 3.10.0 ;
> monothread (`OPENBLAS_NUM_THREADS` et `OMP_NUM_THREADS` valent 1).

Pourquoi un conteneur épinglé. Un libellé d'exécuteur (`ubuntu-24.04`) n'est pas une image figée : elle est mise à jour par GitHub, bibliothèques comprises, sans commit dans le dépôt. `outil_USP`, qui installait R sur l'image de l'exécuteur, a constaté une dérive entre deux exécutions de même R et de même image, due à une BLAS optimisée (OpenBLAS) qui choisit ses noyaux selon le processeur (constat d'`outil_USP` rapporté par l'ADR 0008, « Contexte »). Une étiquette de conteneur peut elle aussi désigner une autre image après une reconstruction (hypothèse de l'ADR 0008, non vérifiée) ; l'empreinte (`@sha256:...`) désigne un contenu immuable. Le conteneur est une fonctionnalité native de GitHub Actions, et non une action tierce, ce que la politique des Actions du dépôt exige.

Pourquoi la bascule. L'image charge par défaut OpenBLAS (relevé de la première exécution, rapporté par l'annotation du 16/10/2026 de l'ADR 0008). Le mainteneur a retenu de basculer dans le job vers la Netlib déjà présente dans l'image, plutôt que de compiler R depuis la source (décision M47 ; le repli par compilation reste prévu par l'ADR 0008, point 10). L'étape « BLAS et LAPACK de référence » de `.github/workflows/ci.yml` pointe par `update-alternatives` les alternatives `libblas.so.3-x86_64-linux-gnu` et `liblapack.so.3-x86_64-linux-gnu` vers `libblas3` et `liblapack3`, puis affiche les cibles résolues et les versions des paquets. La plateforme de référence est l'image épinglée *suivie de cette bascule* : modifier l'étape, l'image ou les bibliothèques est un changement de plateforme, décidé par le mainteneur et, une fois des références existantes, accompagné d'un tableau avant / après de toutes les références (ADR 0008, point 9).

Script de plateforme. `tests/controles/plateforme.R` s'exécute en première étape de chaque job R qui calcule, avant tout calcul (ADR 0008, points 3 à 5) ; le job « Absence de Java », qui n'effectue aucun calcul numérique et ne produit aucun résultat, en est exempté, ainsi que des variables monothread (ADR 0008, annotation du 17/10/2026). En mode strict (`--exiger`, sur la CI), il **échoue** si : `R.version.string` n'est pas exactement `R_ATTENDU` (`R version 4.3.3 (2024-02-29)`) ; l'une des

quatre lectures de la BLAS et de la LAPACK (`extSoftVersion()["BLAS"]`, `sessionInfo()$BLAS`, `sessionInfo()$LAPACK`, `La_library()`) est vide, désigne une BLAS optimisée (motif `MOTIF_OPTIMISE` : `openblas`, `mkl`, `atlas`, `blis`, `flexiblas`) ou sort des chemins de référence (`MOTIF_BLAS`, `MOTIF_LAPACK`) ; `La_version()` n'est pas `LA_VERSION_ATTENDUE` (3.10.0) ; `OPENBLAS_NUM_THREADS` ou `OMP_NUM_THREADS` ne vaut pas 1 ; `TABLEXP_IMAGE`, que le job renseigne avec la même chaîne que l'image du conteneur, n'est pas épinglée par empreinte (`MOTIF_IMAGE`). Sans `--exiger` (poste, session), il affiche les mêmes grandeurs et les écarts sans échouer : les batteries restent exécutables sur le poste Windows du mainteneur, dont la BLAS est celle de R. Dans les deux modes, il écrit l'*identifiant de plateforme* (image, `R.version.string`, BLAS, LAPACK et sa version, `sessionInfo()`) et consigne sans faire échouer le processeur et ses drapeaux (`fma`, `avx2`, `avx512f`), la version de la glibc, dont vient la `libm` (sortie de `getconf GNU_LIBC_VERSION`, à défaut première ligne de `ldd --version`, « inconnue » si aucune de ces sorties ne désigne la glibc ; issue #72), `ImageOS` et `ImageVersion` de l'hôte, `LC_COLLATE`, le fuseau horaire, la présence d'un exécutable `java` et `GITHUB_SHA`. Ces grandeurs consignées n'entrent pas dans l'identifiant de plateforme : elles ne déclenchent aucune bascule. Ce bloc est reporté dans le message de chaque commit de référence et dans l'en-tête de chaque fichier de référence (section 4.5), sous la clé « plateforme consignée » pour les grandeurs consignées ; la version de la glibc y figure depuis la régénération du 24/10/2026 (ligne « glibc (libm) »), à côté de `fiche$plateforme$bibliotheque_c`, consignée par la fiche elle-même (section 15.1).

> **Ce que la plateforme ne garantit pas.** Le modèle de processeur varie d'une exécution à l'autre sous le même libellé d'exécuteur (relevé de l'ADR 0008, annotation du 16/10/2026). La BLAS Netlib n'a pas de noyau choisi selon le processeur ; la réserve sur la bibliothèque mathématique du système (section 3.1) reste en revanche ouverte pour un exécuteur sans FMA : elle n'a pas été observée sur les trois modèles rencontrés, tous munis de FMA et d'AVX2 (ADR 0008, annotation du 19/10/2026). Les références sont donc comparées à tolérance, structure contrôlée à part (E4.7 ; section 4.5).

4.4 Jobs de la CI

`.github/workflows/ci.yml` définit les jobs suivants, exécutés sur chaque pull request, sur chaque push vers `main` et sur demande. Aucun job n'installe de package du CRAN ni ne configure de dépôt CRAN pour une installation : `tablexp` est installé depuis l'arbre de travail par chaque batterie.

Table 4 : Jobs de la CI (`.github/workflows/ci.yml`).

Job	Exécution	Contenu
« Contrôles du dépôt »	hôte `ubuntu-24.04`, sans R	réglages de Claude Code valides (JSON) ; fiches d'agents complètes ; fiches `-approfondi` à jour ; aucun fichier sensible versionné
« Tests (R 4.3.3) »	conteneur de la plateforme de référence	bascule vers la BLAS et la LAPACK de référence ; script de plateforme en mode strict ; les trois batteries ; liste blanche de `DESCRIPTION` ; ordre des modules ; identité des batteries et de l'image. Les étapes des batteries et des contrôles, sous la condition `${{ !cancelled() }}`, s'exécutent même si une étape antérieure échoue, pour que tous les écarts soient visibles ; le job reste en échec. L'étape du script de plateforme, sans condition, est sautée si la bascule vers la BLAS de référence échoue
« Absence de Java »	même conteneur, réseau requis	contrôle de l'absence de Java (section 5.2) et son cas témoin

Job	Exécution	Contenu
« Documentation »	hôte `ubuntu-24.04`, sans R	installation des paquets TeX par `apt-get` (versions affichées), compilation du présent document selon `docs/latex/CONVENTIONS.md` (section 6 : passes jusqu'à stabilisation, cinq au plus ; échec sur une erreur ou un renvoi indéfini), PDF et journal publiés en artefact

Ces quatre jobs sont les contrôles requis que liste le ruleset de `main` (`.github/ruleset-main.json`, appliqué au dépôt par le mainteneur ; ADR 0008, point 6) : une pull request ne peut alors être fusionnée que s'ils sont verts. Le job « Documentation » ne produit aucun résultat ; le PDF versionné reste celui que commite la session principale, et la version des paquets TeX suit l'image de l'exécuteur. Les batteries et les scripts des jobs conteneurisés ne dépendent pas de git, absent de l'image ; un contrôle qui lit l'historique reste dans « Contrôles du dépôt », sur l'hôte (ADR 0008, point 11).

4.5 Références de non-régression

Une *référence de non-régression* fige, sur la plateforme de référence, ce que le moteur rend pour un *plan de référence* appliqué à l'*entrée de référence* ; toute modification ultérieure qui change une grandeur est détectée par la batterie de reproductibilité et appelle un tableau avant / après et un visa (E4.7 ; ADR 0009, accepté le 17/10/2026, décision M52, et ses annotations). L'outillage, en R de base, est sous `tests/reference/` : `outils_reference.R` (aplatissement, écriture, lecture, comparaison, tolérances, plans et entrée), `produire_references.R` (script du workflow de production) et `comparer_references.R` (comparaison de deux fichiers en ligne de commande).

Ce qui est figé

— **L'objet `construction` entier**, aplati, **moins** `execution` (jamais écrit), `fiche$versions` et `fiche$plateforme`, écrits dans l'en-tête du fichier et jamais comparés : une montée de version de `tablexp` n'est pas un changement de résultat. `fiche$version_fiche`, le plan et les rôles sont comparés : le plan de référence est lui-même une grandeur.

— **Les empreintes du résultat et du code** (`fiche/empreintes/resultat` et `fiche/empreintes/code`), **comparées par leur forme** (ADR 0009, seconde annotation du 20/10/2026 ; décision M69, point 4 ; issue #67). La première dépend de la plateforme (section 15.3) et la seconde change à chaque commit qui modifie `R/` : comparées en valeur, la première ferait échouer la comparaison sur tout autre exécuteur, et la seconde demanderait une régénération à chaque commit `moteur:`. L'aplatissement (`tests/reference/outils_reference.R`) remplace donc la valeur de ces deux chemins, et d'eux seuls (`CHEMINS_JETON_MD5`), par le jeton `"<md5>"` (`JETON_MD5`), **si et seulement si** c'est une chaîne de longueur 1 de la forme `^[0-9a-f]{32}$` ; toute autre valeur (`NA`, majuscules, autre longueur, autre type) est gardée telle quelle et relevée en écart contre le jeton de la référence, si bien qu'une empreinte redevenue `NA` est détectée. Le chemin, le type et la longueur restent comparés. La substitution se fait dans l'aplatissement, commun au producteur et à la batterie, et jamais par une règle générique sur la forme hexadécimale, qui masquerait deux grandeurs exactes et portables : `fiche/empreintes/donnees`, comparée en premier, et les lignes `@empreinte` des tables résumées. `statut_resultat_code`, `representation_resultat` et les autres champs de `fiche/empreintes` restent comparés exactement. Les valeurs réelles sont écrites dans l'en-tête, hors comparaison (ci-dessous). Le jeton ne vérifie que la forme : les propriétés des deux empreintes sont testées par les batteries (section 15.5). Options écartées : retirer les deux chemins de l'aplatissement, ce qui perdrait

le contrôle de structure et la détection d'un `NA` ; les comparer en valeur. Le format reste `tablexp reference 1` : le jeton est une chaîne valide, sans grammaire nouvelle.

— **Les tables de lignes de l'étape `donnees`** (`lignes_conservees`, `numeros_conserves`, `lignes_exclues`, `observations`, `observation_des_lignes` ; constante `RESUMES_PAR_EMPREINTE`) sont **résumées** par leur nombre de lignes et l'empreinte MD5 de leur représentation canonique (colonnes écrites par `ecrire_valeurs()` en forme « texte », section 15.2 ; sans colonne double, ces tables s'écrivent de même dans les deux formes, si bien que le passage à la représentation canonique 2 a laissé leurs lignes `@empreinte` inchangées), non recopiées : elles ne contiennent que des entiers, des dates et des chaînes, sans arithmétique flottante, si bien que le résumé exact ne perd aucune détection, et la table « avant » se reconstitue en rejouant le plan sur le commit de référence. Les attributs autres que les noms, la classe et les noms de lignes n'y entrent pas.

— **Première grandeur** : l'empreinte des données (`fiche/empreintes/donnees`), qui identifie l'entrée.

Entrée de référence (décision M50, Q6 ; ADR 0009, point 4.1) : un jeu tiré par `simuler_portefeuille()` sous la loi vraie des tests (Gompertz-Makeham $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$, $c = 1{,}1$, facteurs H 1 et F 0,6 ; rachat constant 0,05), portefeuille uniforme de 10^5 contrats, proportion d'hommes 0,5, âges 20 à 75, origine 2005-01-01, période 2015-01-01 au 2024-12-31, graine 20261016 (constante `GRAINE_REFERENCE`), rôles `id_contrat`, `id_tete`, `sexe`, `date_naissance`, `date_entree`, `date_sortie`, `cause_sortie`. Le simulateur est ainsi figé pour cette graine : une modification qui change le jeu tiré change l'empreinte et met toutes les références en « entrée différente ».

Table 5 : Plans de référence (`plans_reference` de `tests/reference/outils_reference.R` ; tous sur la période de l'entrée, sans exclusion ; un identifiant n'est jamais réemployé).

Id	Description	Segm.	Délai	Convention	Par année	Estimateurs, niveau
R1	vivant → décédé, référence principale	`sexe`	0	`plus_fine`	non	Poisson, binomial ; 0,95
R2	sans segmentation, délai, niveau 0,99	aucune	90	`plus_fine`	non	Poisson, binomial ; 0,99
R3	convention forcée	`sexe`	0	`age_atteint_annee`	non	Poisson, binomial ; 0,95
R4	par année (surface)	`sexe`	0	`plus_fine`	oui	Poisson ; 0,95
R5	en cours → racheté, unité contrat	`sexe`	0	`plus_fine`	non	Poisson, binomial ; 0,95
R6	R1 avec l'estimateur produit-limite	`sexe`	0	`plus_fine`	non	Poisson, binomial, `kaplan_meier` ; 0,95

Format des fichiers

Un fichier texte par plan, `tests/reference/<id>.tsv`, en UTF-8, lignes terminées par LF, écrit octet par octet :

— un **en-tête** de lignes `# <clé> : <valeur>`, hors comparaison : version du format (`tablexp reference 1`), identifiant et description du plan, identifiant de plateforme écrit par `tests/controles/plateforme.R` (grandeurs contrôlées et grandeurs consignées), `fiche$versions` et `fiche$plateforme`, valeurs réelles des empreintes du résultat et du code (clés `fiche$empreintes/resultat` et `fiche$empreintes/code`, qui disent de quel résultat et de quel code la référence a été produite sur la plateforme de référence), commit calculé, exécution et adresse, mode, date UTC, empreinte des données en rappel ;

— un **corps** d'une ligne par vecteur atomique, `chemin<TAB>type<TAB>longueur<TAB>valeurs`, dans l'ordre de parcours de l'objet ; le chemin joint les noms par `/` (`etapes/brut/resultat/poisson/taux`) ;

— des **lignes de structure** : `@noms` de chaque liste et de chaque vecteur atomique nommé, `@colonnes` et `@lignes` de chaque `data.frame`, `@classe` de l'objet, `@lignes` et `@empreinte` d'une table résumée ;

— des **types** `double`, `integer`, `logical`, `character`, `Date`, `factor`, `NULL`, aux valeurs écrites par l'écrivain de la représentation canonique, `ecrire_valeurs()`, en forme « texte » (section 15.2) : doubles en `%.17g` (et non en octets, forme réservée à l'empreinte des données), dates sur quatre chiffres d'année, jeton `NaN` distinct de `NA`, relues par l'outillage, sans installer `tablexp`, à la condition que la réécriture de la valeur lue redonne le texte (issue #19), chaînes échappées entre guillemets, sous la même règle de codage, par **refus** et jamais par conversion ;

— des **refus d'écriture**, avec le chemin fautif, pour toute autre classe (`POSIXct`, `difftime`…), tout attribut autre que les noms, la classe et les noms de lignes, et des noms de lignes non automatiques : rien n'est converti ni ignoré en silence. Les niveaux d'un facteur ne sont pas écrits (limite sans portée : l'objet n'en contient aucun ; en ajouter demanderait une version 2 du format).

Lecture : en R de base, sans évaluation de code. La lecture des doubles dépend de la plateforme. *Hors Windows*, plateforme de référence comprise, le texte lu doit être égal à l'écriture `%.17g` de la valeur lue : la bibliothèque C du système arrondissant correctement, c'est une **preuve d'aller-retour**. *Sous Windows*, le texte doit seulement avoir la forme de `%.17g` : l'écriture du système n'y arrondit pas toujours correctement le dix-septième chiffre, et la lecture n'y rend pas toujours le double désigné (constats sur le poste du mainteneur, ADR 0009, annotation du 18/10/2026) ; une comparaison exécutée sous Windows est donc un **constat de mesure**, non une preuve, et une référence n'est jamais produite hors de la CI.

Comparaison et tolérances

1. **Entrée, d'abord** : l'empreinte des données, exigée des deux côtés, est comparée en premier ; si elle diffère, le plan est en échec « entrée différente », et la comparaison se poursuit et s'imprime sous cette mention, ses écarts n'étant pas interprétables comme un tableau avant / après. L'impression nomme les causes possibles : le simulateur a changé pour cette graine (ADR 0009, point 4.2), ou la plateforme a produit un tirage différent (ADR 0008, hypothèse sur la bibliothèque mathématique du système) ; et, quand la représentation canonique (`fiche/empreintes/representation`) diffère entre la référence et l'objet, une troisième : la définition de la représentation a changé, l'entrée pouvant alors être inchangée, ce que seule prouve la garde « entrée différente » des transitions (« Règles durables », ci-dessous ; ADR 0009, troisième annotation du 19/10/2026, point (f)).
2. **Structure, exacte** : même ensemble de chemins, mêmes types et longueurs, mêmes positions des `NA` ; tout écart (chemin manquant, en trop, type, longueur) est structurel et fait échouer le plan.
3. **Valeurs, élément par élément** : deux éléments sont égaux s'ils sont identiques (`Inf`, 0, `NA` et `NaN` compris, sans confondre `NA` et `NaN` ; ± 0 confondus par la représentation canonique, qui écrit $x + 0$, soit `0` pour `-0`, et par l'égalité == de la comparaison, ce qui est voulu : aucune grandeur du moteur ne porte de sens par le signe d'un zéro ; ADR 0009, complément du 18/10/2026, point (e)) ; sinon, deux valeurs finies, non nulles, de même signe, sont égales à la tolérance τ de la grandeur si $|a-b| \leq \tau \max(|a|, |b|)$; tout autre cas est un écart. Cette règle, symétrique et relative, est propre aux références ; `proche()` reste celle des tests unitaires.

Les grandeurs de type entier, logique, chaîne, date et les lignes de structure sont exactes par leur type, sans clé de tolérance : c'est le cas du nombre des avertissements, stocké en `integer`, si bien qu'un retour au type `double` serait relevé deux fois, comme écart structurel de type et comme double

sans classe (ADR 0009, quatrième annotation du 17/10/2026) ; c'est aussi le cas de la version de la fiche, de la graine, des colonnes entières de `kaplan_meier_sauts` (`numerateur`, `ensemble_a_risque`), de `kaplan_meier/instants_evenement`, de `denominateur_instants` (paramètres et fiche), des cinq diagnostics de `kaplan_meier` et, depuis l'issue #37, du nombre de cellules de l'étape `exposition` (`etapes/exposition/diagnostics/cellules`), toutes stockées en `integer`. Les treize clés exactes qui les nommaient dans `TOLERANCES` sont retirées : inertes, la tolérance n'étant cherchée que pour un double, elles auraient en outre masqué un retour au type `double` au contrôle de complétude des classes. Douze l'ont été sur avis d'`actuary` (ADR 0009, complément du 18/10/2026, point (f)) ; la treizième, la clé nue `cellules`, l'a été à son tour, sur avis d'`actuary`, parce qu'elle classait aussi `etapes/brut/diagnostics/cellules` et `etapes/brut/diagnostics/etats/cellules`, déjà `integer`, et aurait masqué leur retour au double (ADR 0009, seconde annotation du 19/10/2026, point (a)). Un double reçoit sa tolérance de la table `TOLERANCES` (tableau 6), cherchée par le **suffixe le plus long** du chemin : une clé qualifiée `<table>/<colonne>` l'emporte sur le nom nu, ce qui donne au `taux` de `kaplan_meier` une autre classe qu'au `taux` de Poisson ou du binomial sans changer le nom commun de la colonne. **Un double sans classe déclarée fait échouer la comparaison** : une grandeur nouvelle reçoit sa classe d'`actuary` avant d'entrer dans une référence, et toute modification de la table est annotée dans l'ADR 0009, datée, avec l'avis d'`actuary`.

Table 6 : Classes de tolérance des doubles (`TOLERANCES`, `tests/reference/outils_reference.R` ; avis d'`actuary`, ADR 0009, point 3.4, seconde annotation, complément du 18/10/2026, point (f), et seconde annotation du 19/10/2026, point (a)).

Classe	Grandeurs	Justification
exacte	`exposition_jours`, `niveau`, `delai_declaration`, `tolerance_effectif`	doubles à valeur entière ou déclarée : aucun bruit possible
10^{-14}	`exposition_centrale`, `complement_initial_cellule`, `complement_initial_fenetre`, `exposition_initiale`, `taux` ; `kaplan_meier_sauts/age_exact`, `kaplan_meier_sauts/variance_log_survie_cellule`	arithmétique IEEE de base, ordre des opérations fixé par le code
10^{-12}	les bornes exactes et asymptotiques, `probabilite` et ses quatre bornes ; `kaplan_meier/taux`, `kaplan_meier/variance`, `kaplan_meier_sauts/survie_cellule`	fonctions spéciales (`qgamma`, `qbeta`) et fonctions mathématiques de la bibliothèque du système (`exp`, `log`, `log1p`, `expm1`), dont la variante peut dépendre du processeur (sans écart mesuré sur trois modèles munis de FMA, non mesurée sans FMA ; ADR 0009, annotation du 19/10/2026, conséquence 3)

Les tolérances restent plusieurs ordres de grandeur sous le plus petit changement significatif estimé par `actuary` (un jour d'exposition dans la cellule la plus exposée, un événement de plus ; ADR 0009, point 3.4). **Impression**, pour chaque plan, dans la batterie comme dans le workflow : une ligne de bilan (grandeurs comparées, identiques, à écart sous tolérance, en écart, écarts structurels) ; l'écart relatif maximal observé par classe, toujours imprimé, même nul ; une ligne par grandeur dont l'écart n'est pas nul, avec sa position et les valeurs avant et après en `%.17g`. Ces lignes sont la matière du tableau avant / après ; `comparer_references.R` produit la même impression à partir de deux fichiers.

Batterie, production et commit

Dans la batterie (`tests/test_reproductibilite.R`, job « Tests (R 4.3.3) », donc sur la plateforme de référence) : le jeu est tiré une fois, chaque plan de référence est exécuté, aplati, écrit puis relu (aller-

retour `identical()`), et comparé à son fichier ; un plan sans fichier est un échec attendu renvoyant à l'issue de création tant qu'il figure dans la liste `SANS_REFERENCE_17`, vide depuis la création des six fichiers ; hors de cette liste, un fichier manquant est un échec, de même qu'un fichier sans plan (référence orpheline). Des témoins, sur des copies temporaires et jamais sur `tests/reference/`, vérifient que la comparaison échoue quand elle le doit (valeur altérée au-delà de sa tolérance, grandeur retirée ou ajoutée, type ou longueur changés, empreinte altérée ou absente, double sans classe, classe ou attribut interdits), qu'une valeur altérée juste en deçà de sa tolérance passe, et que les chaînes non ASCII font l'aller-retour au bit près sous les locales C et UTF-8, l'aplatissement de R1 donnant les mêmes octets sous les deux. Pour le jeton (ADR 0009, seconde annotation du 20/10/2026, (i)), des témoins sur des copies de R6 vérifient que l'aplatissement porte le jeton aux deux seuls chemins et que l'aller-retour reste `identical()` ; qu'une empreinte du résultat ou du code `NA`, en majuscules ou de longueur 2 est gardée et relevée en écart ; qu'une empreinte des données altérée n'est pas remplacée et reste « entrée différente » ; qu'une ligne `@empreinte` altérée reste un écart ; que l'en-tête porte les deux valeurs réelles, et qu'en changer la valeur dans l'en-tête ne change pas la comparaison.

Production, par le workflow `.github/workflows/references.yml`, distinct de `ci.yml` : déclenché à la main seulement (`workflow_dispatch`), sur ordre du mainteneur, droits en lecture seule, sans secret, dans le même conteneur épinglé que le job « Tests (R 4.3.3) », avec la même étape de bascule vers la BLAS et la LAPACK de référence et le script de plateforme en mode strict avant toute production (invariants contrôlés par `tests/controles/identite_batteries.R`, contrôle 4). Le script `produire_references.R` produit et compare toujours tous les plans ; le **mode** fixe ce qui est admis : `creation` (aucun fichier existant pour les plans visés, comparaison réussie pour les autres), `regeneration` (écarts admis sur les seuls plans visés), `bascule` (écarts admis sur tous, l'identifiant de plateforme de l'exécution devant différer de celui des en-têtes commités) ; un mode `releve`, refusé sur la CI, sert aux essais sur un poste, dont les fichiers ne sont jamais commités. Symétriquement, les modes `creation`, `regeneration` et `bascule` sont **refusés hors de la CI**, c'est-à-dire quand la variable d'environnement `GITHUB_ACTIONS` ne vaut pas `true` : le refus intervient avant le contrôle des autres options et avant toute écriture, si bien qu'aucun dossier de sortie n'est créé (ADR 0009, seconde annotation du 19/10/2026, point (d)). Cette garde protège contre l'erreur, **pas contre la malveillance** : la variable se pose à la main ; la garantie de fond reste l'appariement des MD5 des fichiers commités à ceux de l'artefact de l'exécution citée, vérifié par la session principale au commit puis en revue finale (ADR 0009, point 5.5). Le dossier de sortie ne peut être `tests/reference/` ni s'y trouver, quelle que soit l'écriture du chemin (séparateurs, casse, lecteur, chemins réseau, liens, noms courts). Le job ne commite rien : il publie un artefact (références, leurs MD5, identifiant de plateforme, exécution, écarts), conservé 90 jours. La session principale seule, après visa (`actuary` pour les taux bruts ; mainteneur dès qu'une table finale ou un verdict change), vérifie les MD5, copie les fichiers sans modification et les commite avec l'identifiant de plateforme, le tableau avant / après et le visa.

Références existantes : R1 à R6, créées le 18/10/2026 en mode `creation` (exécution 39140635363 du workflow, commit `1019d8e`, visa d'`actuary`). Les grandeurs des étapes `donnees` et `exposition` sont rattachées à l'état de la première branche (`f9ab269`) par une chaîne de mesures (différences de code vides, ou objets `identical()` hors `execution`, un changement de type sans changement de valeur, visé, étant admis ; ADR 0009, annotation du 18/10/2026). Les références R1 à R6 ont été **régénérées une fois**, le 19/10/2026, en mode `regeneration` sur tous les plans (exécution 39248539956 du workflow, calcul sur `99272a4`, commit `763da20`, visa d'`actuary`), pour le seul changement de type du nombre de cellules de l'étape `exposition`, passé de `double` à `integer` à valeur inchangée (issue #37) : le corps de chaque fichier ne change que de cette ligne, et toutes les autres grandeurs sont restées identiques au bit près (ADR 0009, seconde annotation du 19/10/2026, point (a)). Elles l'ont été **une seconde fois**, le 19/10/2026, en mode `regeneration` sur tous les plans (exécution 39315172327 du workflow, calcul sur `d18c269`, commit `50837f6`, visa d'`actuary`, sur ordre du mainteneur), pour les seuls changements de la fiche annoncés

en transition par les issues #35 (représentation canonique et, par son en-tête, empreinte des données, l'entrée étant prouvée inchangée par la garde « entrée différente ») et #42 (version de la fiche, champs retirés) : toutes les autres grandeurs sont restées identiques au bit près. Elles l'ont été **une troisième fois**, le 20/10/2026, en mode `regeneration` sur tous les plans (exécution 39435673257 du workflow, calcul sur `5757c30`, commit `192180c`, visa d'`actuary`, sur ordre du mainteneur), régénération unique de la branche `claude/fiche-rapport`, pour les seuls changements de la fiche annoncés en transition par les issues #57 (`version_fiche` de 2 à 3, champ `nombre_lignes_exclues`), #64 (ordre des champs) et #67 (champ `representation_resultat`, empreintes du résultat et du code comparées par jeton, statut `statut_resultat_code`) : toutes les autres grandeurs sont restées identiques au bit près, et l'empreinte des données est inchangée. Elles l'ont été **une quatrième fois**, le 24/10/2026, en mode `regeneration` sur tous les plans (exécution 40111528681 du workflow, calcul sur `a96cf7f`, commit `26f7a29`, visa d'`actuary`, sur ordre du mainteneur), régénération unique de la branche `claude/rapport-fige`, pour le seul passage de `version_fiche` de 3 à 4 (issue #77, champ `fiche$plateforme$bibliotheque_c`, section 15.1) : `fiche/version_fiche` est la seule grandeur en écart ; l'empreinte du résultat, comparée par jeton, change en conséquence, ainsi que l'empreinte du code, `R/` ayant changé (en-tête `0be71b0b...` devenu `2cc03eab...`), et toutes les autres grandeurs sont restées identiques au bit près ; l'en-tête gagne la ligne `fiche$plateforme/bibliotheque_c`, hors comparaison.

Ce que les références ne couvrent pas.

— La couverture est celle des plans : une recette ou une convention absente des plans n'est protégée qu'à partir du plan qui l'introduira. En particulier, l'estimateur `kaplan_meier` n'est figé que sous `dernier_anniversaire` (R6) ; sa branche `age_atteint_annee` n'a pas de référence (avis d'`actuary` au visa).

— L'entrée n'exerce pas la fusion des contrats par tête : le simulateur minimal tire une tête par contrat (section 16). La fusion n'est couverte que par les tests unitaires (section 7.6).

— R6 comprend une cellule définie à $\hat{q}_x = 1$ avec un seul exposé à risque : la référence fige ce cas d'instabilité de l'estimateur produit-limite sous troncature (section 9.4), qui n'est pas une anomalie.

— Absence de bruit numérique sur la plateforme de référence : *constat* sur cinq exécutions de la CI sur la plateforme de référence, sur trois modèles de processeur (AMD EPYC 7763, 9V74 et 9V45, tous avec FMA et AVX2), où toutes les grandeurs des six plans sont identiques au bit près (ADR 0008, annotation du 19/10/2026) ; la réserve de l'ADR 0008 sur les variantes de la bibliothèque mathématique du système reste ouverte pour un exécuteur sans FMA, non observé.

— Le signe d'un zéro : la représentation canonique et la comparaison confondent 0 et `-0`, si bien qu'un passage de l'un à l'autre n'est pas détecté (limite admise ; aucune grandeur actuelle n'en dépend ; ADR 0009, complément du 18/10/2026, point (e)).

Règles durables (ADR 0009, point 9). Ajouter une grandeur à l'objet résultat est visible par la non-régression (écart structurel « en trop ») : les plans concernés sont régénérés, avec un tableau avant / après où les lignes nouvelles sont marquées « création », et la classe de tolérance d'un double nouveau est déclarée dans le même commit. Changer le format incrémente sa version et régénère tous les plans. Élargir une tolérance pour faire passer un écart est interdit sans tableau avant / après et visa. Un fichier de `tests/reference/` ne change que par un commit de la session principale qui cite une exécution du workflow et un visa. **Transition d'un changement de résultat annoncé** (ADR 0009, point 9.5, règle retenue par le mainteneur le 19/10/2026 ; mise en œuvre générique par l'issue #47, ADR 0009, troisième et quatrième annotations du 19/10/2026, cette dernière actant l'extension de la garde « entrée différente », décision M64) : quand un commit change un résultat avant la régénération des références, il déclare sa *transition* dans la liste unique `TRANSITIONS_EN_COURS` de `tests/test_reproductibilite.R`, vide hors transition. Une transition porte trois champs, et eux seuls : l'issue (de la forme `#N`), les plans de référence concernés et les changements admis (au moins un). Une issue peut porter plusieurs transitions, à condition

que leurs plans soient deux à deux disjoints : un même retrait n'a pas toujours la même structure sur tous les plans (avant l'issue #42, `fiche/segmentation` était de longueur 0 dans R2 et de longueur 1 ailleurs), alors qu'une transition en déclare une seule. Chaque changement nomme un chemin et une **nature**, prise dans une liste fermée de cinq et déclarée exactement :

— **type** : conversion sans perte de `double` vers `integer`, ou l'inverse ; la référence est du type de départ déclaré ;

— **valeur** : valeurs numériques nouvelles d'une grandeur `double` ou `integer`, avec ou sans positions. Avec positions, la longueur est conservée et chaque position déclarée change ; sans positions, le vecteur déclaré remplace le vecteur entier. C'est ainsi, et seulement ainsi, que se déclare un changement de longueur : une nature « longueur » admettrait des valeurs quelconques. La longueur nouvelle est au moins 1 (valeurs non vides exigées) : le passage d'un vecteur à la longueur 0 n'est pas déclarable, et il est exclu jusqu'à décision contraire ;

— **valeur de chaîne** : de même pour une grandeur `character` (version, empreinte) ;

— **ajout** : un chemin absent de la référence, présent dans l'objet avec le type et la longueur déclarés ; ses valeurs, sans « avant », entrent dans le tableau avant / après de la régénération, sous visa ;

— **retrait** : un chemin présent dans la référence seule, avec le type et la longueur déclarés.

Un test ordinaire refuse une liste mal formée (champs, forme de l'issue, plans, natures et leurs champs, valeurs, positions). Pour chaque plan listé, les changements de toutes les transitions qui le portent **s'unissent** : deux déclarations identiques s'appliquent une fois ; deux valeurs différentes à une même position, un vecteur entier et une autre déclaration de valeurs sur le même chemin, deux conversions de type, un ajout ou un retrait combiné à une autre nature sur le même chemin sont des erreurs. Puis :

1. un **test ordinaire « seul écart »** aligne la référence commitée sur les changements déclarés, dans l'ordre type, valeur, valeur de chaîne, ajout, retrait, et exige que la comparaison ne relève alors aucun autre écart, aux tolérances près ; il exige aussi la réciproque : **tout changement déclaré est observé** sur la référence commitée telle qu'elle est lue (type égal au type de départ, valeurs différentes, chemin ajouté absent, chemin retiré présent), l'observation des valeurs se jugeant après la seule conversion de type, qui ne change aucune valeur ;
2. la **comparaison stricte** est un échec attendu, un seul par plan, rattaché à l'issue de la première transition de la liste qui porte le plan : une comparaison est un seul test, la régénération porte sur un plan entier et jamais sur une issue seule, et le harnais attend une issue par échec attendu. Le test des doubles sans tolérance déclarée n'est un échec attendu que si la référence commitée en porte ; un test ordinaire exige alors que ces doubles soient tous admis par une transition (type converti depuis `double`, ou retrait déclaré de type `double`) ;
3. **garde « entrée différente »** (point 3.2 de l'ADR 0009, étendu) : un changement de l'empreinte des données (`fiche/empreintes/donnees`) n'est admis qu'en valeur de chaîne sans positions, avec un changement de `fiche/empreintes/representation` dans l'union du plan, car seule une définition nouvelle de la représentation canonique change l'empreinte d'une entrée inchangée (section 15.2) ; et un test ordinaire de garde doit prouver que l'entrée est inchangée. Il calcule par le package le texte canonique de l'entrée de référence (données et rôles déclarés), vérifie que l'en-tête de sa première ligne (texte avant la première tabulation) est la représentation de l'objet, le remplace par la représentation de la référence, et exige que le MD5 obtenu soit égal à l'empreinte des données de la référence ; il exige aussi que les deux représentations diffèrent, et que l'objet ait été construit sur l'entrée qu'on lui passe : le MD5 du texte canonique de cette entrée, sous la représentation courante, doit être égal à l'empreinte des données de l'objet (réserve M2 de l'audit de l'issue #47, levée par l'issue #35 ; ADR 0009, troisième annotation du 19/10/2026, point (f) 4, et quatrième, point (a)). Lors de la transition

de l'issue #35, la garde a prouvé l'entrée de référence inchangée : le texte de la représentation 2, sous l'en-tête de la représentation 1, redonne l'empreinte de la référence commitée, preuve refaite hors de R dans le commit de l'issue (ADR 0011, point 2.5). Sans cette garde, la transition absorberait le contrôle de l'entrée : l'entrée de référence aux lignes permutées, réellement différente, ne s'écarte des références de R1 et de R6 que par l'empreinte des données et celles des deux tables résumées, les autres écarts restant sous tolérance, si bien que déclarer ces trois chaînes suffit à faire passer le test « seul écart ». C'est un constat sur une permutation par plan : pour R1, la permutation tirée par `set.seed(1)` et `sample.int()` (ADR 0009, troisième annotation du 19/10/2026, point (f) 1) ; pour R6, la permutation tirée sous la graine 20261019 (`GRAINE_PERMUTATION`), dont un témoin de la batterie rejoue le constat et exige l'échec de la garde (section 15.2). Si la preuve échoue, aucune transition n'est possible : c'est un changement de l'entrée de référence (ADR 0009, point 4.2), qui régénère tous les plans, sous visa. **Limite de la garde** (ADR 0009, annotation du 20/10/2026) : elle ne remplace que l'en-tête de la première ligne et suppose donc qu'une représentation nouvelle ne change rien d'autre dans le texte de l'entrée de référence. C'était vrai pour l'issue #35, l'entrée n'ayant aucune colonne double, et pour l'issue #19, ses dates étant entre 1930 et 2024. Une représentation qui changerait l'écriture d'un type présent dans l'entrée (entier, chaîne, date) ou la forme de la première ligne ferait échouer la garde *à tort* : l'erreur est conservatrice, la garde ne pouvant conclure à tort à une entrée inchangée (hors collision MD5), mais son diagnostic « l'entrée a changé » serait faux. La branche qui fera un tel changement choisira d'abord, par annotation de l'ADR 0009, sa voie : de préférence un écrivain figé de la représentation précédente, dans les tests, pendant la transition.

Le commit des références de la session principale retire la transition : un plan resté listé après la régénération fait échouer la batterie, le test « seul écart » relevant un changement déclaré non observé et la comparaison stricte devenant un succès inattendu. Une tolérance élargie ou un chemin admis « quel que soit l'écart » n'est pas une transition mais un changement de résultat déguisé ; la règle ne dispense ni du tableau avant / après, ni du visa. Elle a servi une première fois pour l'issue #37, sous une forme propre à cette issue (`A_REGENERER_37`, vidée par le commit des références, puis remplacée par la liste unique), puis pour les issues #35 et #42, dont les transitions ont été closes par la seconde régénération, puis pour les issues #57, #64 et #67, dont les transitions ont été closes ensemble par la troisième (« Références existantes », ci-dessus) : la liste est de nouveau vide. Chaque nature, l'union et ses conflits, l'observation, la forme de la liste, le rattachement de l'échec attendu, les doubles restants et la garde sont exercés par des témoins de la batterie, sur des objets en mémoire et jamais sur `tests/reference/`.

Contrôles du dépôt

5.1 Scripts de contrôle

Tout contrôle mécanique est un script, jamais une relecture (`CLAUDE.md`, « Workflows », principe 7). Les scripts sont en R de base, sous `tests/controles/`, et sortent avec un code non nul en cas d'écart. Chacun est testé sur des témoins dans `tests/unitaires/test_controles.R` : il doit échouer sur un cas fautif et passer sur le dépôt. Fait exception `ecriture_doubles.R`, recoupement hors CI dont la batterie unitaire garde un échantillon (section 15.5) ; pour `locale_c.R`, dont l'exécution complète (deux installations, six plans) est une étape de la CI, la batterie ne rejoue que le chemin d'échec bon marché : une racine sans package sort en échec et nomme l'installation sous `LC_ALL=C`. Elle charge en outre le script sans le lancer (`sys.source()`), vérifie que `LOCALES_UTF8`, dupliquée de `tests/harnais.R`, y est identique

à celle du harnais et que `locale_utf8_disponible()` y rend la même locale que celle du harnais, et teste la fonction `ecart_octets` du script, qui rédige le message d'un écart de la comparaison octet à octet et que le chemin d'échec bon marché n'atteint pas, sur des attendus écrits à la main (textes identiques, premier octet différent à la deuxième ligne, longueurs inégales à préfixe commun, dans les deux sens).

Table 7 : Scripts de contrôle (`tests/controles/`).

Script	Règle contrôlée	Source
`liste_blanche_description.R`	`Depends` : exactement `R (>= 4.3.3)`, plus d'éventuels packages de priorité `base` ou admis ; `Imports` : packages de priorité `base` de R 4.3.3, ou admis ; `LinkingTo`, `Suggests`, `Enhances` : packages admis seulement. Un package est admis s'il figure, avec son champ et l'ADR de méthode qui l'introduit, dans la constante `ADMIS`, vide à cette date	ADR 0002, point 3 et A8 ; M44
`graphe_modules.R`	graphe d'appels autorisés et appels dynamiques interdits (section 1.3)	ADR 0007, points 2.1 à 2.3
`identite_batteries.R`	mêmes batteries, dans le même ordre, dans `CLAUDE.md`, dans `circuit-technique.js` et dans les étapes « Batterie : » du job « Tests (R 4.3.3) » ; ce job ne porte ni `continue-on-error`, ni condition `if:` de job, et une étape de batterie n'a pas d'autre condition que `${{ !cancelled() }}` ; une seule chaîne d'image dans tous les workflows ; invariants du workflow `references.yml` (contrôle 4, section 4.5)	`CLAUDE.md` ; ADR 0008, points 1 et 8 ; ADR 0009, point 6
`plateforme.R`	plateforme de référence (section 4.3)	ADR 0008, points 3 à 5
`absence_java.R`	aucune dépendance à Java (section 5.2)	ADR 0002, A1 ; ADR 0008, point 7 ; issue #3
`locale_c.R`	`tablexp` installé dans deux bibliothèques temporaires, l'une sous `LC_ALL=C`, l'autre sous la première locale UTF-8 disponible, et exécuté dans un sous-processus sous la locale de son installation, qui vérifie d'abord la locale effective ; pour R1 à R6, fiche et plan écrits, relus `identical()` et réécrits à l'identique, et, pour R1, bilan du rejeu outillé (chaînes non ASCII marquées UTF-8), puis ces textes, deux par plan et le bilan, comparés octet à octet entre C et UTF-8, sans écart admis (section 4.2) ; étape bloquante « Locale C (installation et exécution) » du job « Tests (R 4.3.3) », hors liste des batteries	ADR 0007, point 11 ; M54 ; issue #66
`ecriture_doubles.R`	écriture des doubles de l'empreinte en octets IEEE 754 identique, ligne à ligne, à celle d'un juge indépendant (Python, `struct.pack`) sur 10^6 doubles tirés sous graine et des cas limites ; **hors CI** (Python requis), non exécuté sous Windows (section 15.2)	ADR 0011, points 2.1 et 2.4 ; issue #35

La liste des packages de priorité `base` est écrite en dur dans `liste_blanche_description.R` pour R 4.3.3 (`PACKAGES_BASE_R433`) : le verdict ne dépend pas de la machine ; le script signale, sans échouer, un écart avec les packages de priorité `base` effectivement installés.

5.2 Absence de Java

Règle. Aucun code du projet ne dépend de Java, ni directement ni indirectement : ni le moteur, ni l'application, ni la contre-vérification (ADR 0002, annotation du 16/10/2026, A1, décision M40). Java n'est pas utilisable sur le poste de travail du mainteneur ; c'est pour cette raison que le package `ELT`, qui dépend de `xlsx` et donc de Java, est exclu de la contre-vérification. Sa source ne peut être exécutée qu'hors du dépôt, sans Java, en appoint, sans qu'aucun code en soit copié (ADR 0002, A5 et troisième annotation).

Contrôle. `tests/controles/absence_java.R` (issue #3) contrôle trois ensembles de packages :

— le *moteur* : les packages déclarés dans les champs `Depends`, `Imports` et `LinkingTo` de `DESCRIPTION`, hors R ; aucun à cette date ;

— l'*application* : `shiny`, `htmltools`, et `openxlsx` à titre facultatif ;

— la *contre-vérification* : les packages `survival`, `mgcv`, `WH`, `StMoMo`, `demography`, `MortalityLaws`, `lifecontingencies`, `survPen`, `MortCast`, `locfit`, et `flexsurv` à titre facultatif ; seul `survival` a, à cette date, produit des valeurs versionnées (version 3.5.8, sous R 4.3.3, section 9.8).

Les deux derniers ensembles sont lus dans une liste unique, le fichier `packages_hors_moteur.csv` du dossier `tests/controles/` (colonnes `ensemble`, `package`, `statut`, `source`), qui est la référence ; la liste ci-dessus en est la transcription à la date du document. Le script lit les métadonnées d'un **instantané daté** du CRAN chez Posit Package Manager, par `available.packages()`, sans rien installer ; la date est la constante `INSTANTANE` du script (2026-10-15, un jour ouvré : selon la documentation de Posit citée par l'ADR 0008, aucun instantané n'est publié les week-ends et jours fériés), qui ne change que par un commit `repo:`. Il calcule, pour chaque package, la fermeture récursive de ses dépendances `Depends`, `Imports` et `LinkingTo`, par la fonction `tools::package_dependencies()` appelée avec `recursive = TRUE`, et **échoue** (code 1) :

— si un package de `PACKAGES_JAVA` (`rJava`, `xlsx`, `xlsxjars`) figure dans un ensemble ou dans la fermeture de l'un de ses packages ; la chaîne de dépendances la plus courte qui y mène est affichée ;

— si un package listé, ou une dépendance récursive hors priorité `base`, manque aux métadonnées : un package introuvable n'a pas de dépendances connues, il ne passe pas en silence ;

— si la liste est mal formée (en-tête, ensembles et statuts connus, noms valides, sans doublon, ensembles non vides).

Des métadonnées illisibles (panne réseau, adresse fausse) donnent le code 2 : aucun verdict n'est rendu, l'échec du job n'est jamais masqué et une relance le lève. Seul le filtre `duplicates` d'`available.packages()` est appliqué : le filtre par version de R écarterait les packages qui exigent un R plus récent que celui qui exécute le contrôle, alors que la contre-vérification peut tourner sur un autre poste muni d'un R récent, et ces packages seraient alors signalés absents (faux échec) ; le filtre par système d'exploitation est écarté aussi, le poste étant sous Windows et la CI sous Linux.

Cas témoin. Le job « Absence de Java » exécute aussi le script avec `--ajouter ELT` : il doit rendre le code 1 (un verdict, et non le code 2 d'une panne) et afficher la chaîne de `ELT` à `xlsx`. Les tests du script s'exécutent sans réseau, sur des dépôts locaux témoins (`tests/unitaires/test_controles.R`).

> **Limite.** La détection porte sur les seuls noms de `PACKAGES_JAVA`. Le champ `SystemRequirements`, qui pourrait déclarer Java, n'est pas renseigné dans les métadonnées `PACKAGES` du CRAN (relevé consigné dans l'en-tête du script) : un package qui exigerait Java sans dépendre de l'un de ces noms échapperait au contrôle. Étendre la détection relève du mainteneur.

PARTIE II

Point d'entrée et plan de construction

Point d'entrée unique : R/construire_table.R

Le package a un point d'entrée unique, `construire_table(donnees, roles, plan, provenance = NULL)`, qui exécute un *plan de construction* d'un bloc et retourne un objet complet (ADR 0004 ; ADR 0005 ; ADR 0007, point 3.1). `R/construire_table.R` se limite à l'orchestration : il valide le plan, appelle une fonction d'étape par module dans l'ordre du plan et assemble l'objet, sans aucun calcul. Le quatrième argument, `provenance`, facultatif, est la voie par laquelle la couche de lecture transmet le nom du fichier lu, son empreinte et l'algorithme de celle-ci ; `NULL` signifie « données construites en mémoire, aucun fichier », ce que la fiche écrit (section 15.1).

6.1 Plan de construction et validation

Le plan est une liste R nommée, construite en mémoire par l'appelant, et **validée entièrement avant toute exécution, sans aucun défaut implicite** (ADR 0005, point 3 ; ADR 0007, point 5) : un paramètre absent est un refus, jamais une valeur supposée. Il porte six champs de premier niveau (constante `CHAMPS_PLAN` ; décisions M48 et M49) :

— `transition` : `list(depart = , arrivee = )`, deux noms d'états ; dans cette branche, `vivant` → `deces` ou `en_cours` → `rachat` ; les autres causes de sortie sont des sorties concurrentes ;

— `periode` : `list(debut = , fin = )`, deux dates, fin incluse ;

— `delai_declaration` : entier de jours ≥ 0, requis (zéro explicite admis) ;

— `segmentation` : vecteur de rôles pris dans `sexe` et `segment`, sans doublon, requis, vide explicite admis (E2.9). Elle est au premier niveau, et non parmi les paramètres de l'étape `exposition`, parce que l'étape `donnees`, qui précède, en a besoin pour ses contrôles (rôle requis, `segment_manquant`, `tete_segment_incoherent`) : une étape antérieure ne doit pas dépendre d'un paramètre d'une étape postérieure, et un plan réduit à l'étape `donnees` doit rester défini sans défaut (décision M49) ;

— `graine` : entier, requis dès qu'une étape du plan tire un aléa, interdit sinon. Aucune étape de cette branche ne tire d'aléa : la graine y est donc refusée, et la fiche écrit « aucune étape aléatoire » ;

— `etapes` : liste nommée ordonnée ; chaque nom est une étape, au plus une fois, dans l'ordre des modules, et chaque valeur la liste nommée des paramètres de l'étape, tous requis.

`transition`, `periode`, `delai_declaration` et `segmentation` sont déclarés une seule fois et transmis par le point d'entrée, en arguments nommés, à chaque fonction d'étape qui les emploie ; **une fonction d'étape ne reçoit jamais le plan entier** (ADR 0007, annotation du 17/10/2026, point c). Deux étapes ne peuvent

donc pas lire deux périodes ou deux segmentations différentes, et le chemin d'un champ dans le plan n'est connu que du point d'entrée et du validateur.

Les étapes de la branche et leurs paramètres sont des constantes du package (ETAPES, PARAMETRES_ETAPES), que le validateur applique :

Étape	Fonction d'étape	Préalable	Paramètres (tous requis)
donnees	preparer_donnees()	—	exclusions : vecteur de codes de motif sans doublon, vide explicite admis (section 7.3)
exposition	calculer_exposition()	donnees	convention : plus_fine, dernier_anniversaire ou age_atteint_annee ; par_annee : TRUE ou FALSE (section 8.1)
brut	calculer_taux_bruts()	donnees, exposition	estimateurs : sous-ensemble non vide, sans doublon ni NA, de poisson, binomial et kaplan_meier, restitué dans cet ordre ; binomial et kaplan_meier refusés sous par_annee = TRUE ; niveau : niveau de confiance, nombre double unique, fini, $0 <$ niveau < 1 (section 9.1)

Aucune des trois ne tire d'aléa. La recette d'exposition et l'unité d'observation ne sont pas des paramètres : le moteur les déduit des rôles (sections 8.1 et 7.4) ; seule une convention d'âge plus grossière peut être forcée.

Règle commune de forme (issue #23) : chaque paramètre d'étape a exactement le type de base que déclare la colonne type de PARAMETRES_ETAPES (character pour exclusions, convention et estimateurs, logical pour par_annee, double pour niveau) et **ne porte aucun attribut** : ni noms, ni dimension, ni classe (un facteur, une matrice ou un vecteur nommé sont refusés), sinon refus sous le motif propre au paramètre. Il en va de même des champs du plan : transition et periode sont des listes nues, dont le seul attribut admis est celui des noms, exactement autant d'éléments que de noms attendus (un data.frame est refusé) ; les deux bornes de la période n'ont d'autre attribut que la classe Date, stockées en double ou en integer ; delai_declaration est un integer ou un double sans attribut ; segmentation un vecteur character sans attribut, vide compris. La forme est contrôlée par forme_exacte(), directement ou par liste_aux_noms() et chaine_unique() (section 2.3), sous le motif existant de chaque paramètre : aucun motif n'est propre à la forme. Hors du plan, la même règle s'applique notamment à la graine reçue par sous_graine() (refus graine_invalide, section 3.3), aux rôles reçus par controler_roles() (seul attribut admis : les noms ; refus roles_mal_formes, section 7.3 ; issue #62), au mode du portefeuille simulé (refus portefeuille_invalide, section 16.2 ; issue #60) et, à l'appel direct de la fonction d'étape calculer_taux_bruts() (argument controler à sa valeur par défaut, TRUE ; section 6.2), à la segmentation, contrôlée dans sa forme et dans son catalogue par le même contrôle que dans le plan, controler_segmentation() (refus segmentation_invalide du module brut, section 9.1 ; issues #43 et #59). Pour chaque paramètre, elle l'est avant sa valeur, si bien qu'un vecteur nommé de codes inconnus est refusé sous exclusions_invalides ou estimateurs_invalides, et non sous exclusion_inconnue ou estimateur_inconnu.

valider_plan(plan), exportée parce que l'application l'appellera comme contrôle de saisie (E4.3), rend le plan inchangé, ou refuse. Elle porte les codes *structurels* du plan, module construire_table : plan_mal_forme, champ_inconnu, champ_manquant, etapes_mal_formees, aucune_etape, etape_en_double, etape_inconnue, etapes_hors_ordre, etape_prealable_manquante, graine_inattendue, graine_manquante, parametres_mal_formes, parametre_inconnu et

parametre_manquant. Les préalables d'une étape forment une liste (colonne prealable de ETAPES) ; chacun est contrôlé, et le refus etape_prealable_manquante ne cite que les préalables absents, au singulier ou au pluriel selon leur nombre. Son message et ses champs etape et prealable de details rassemblent les étapes fautives d'une part, les préalables absents d'autre part ; le champ prealables_par_etape, liste nommée par étape fautive dans l'ordre du plan, donne pour chacune ses propres préalables absents, dans l'ordre de ETAPES, si bien que le refus dit quel préalable manque à quelle étape (issue #27). Le plan, son champ etapes et la liste des paramètres de chaque étape sont des listes nommées **nues**, dont le seul attribut admis est celui des noms : un data.frame, un objet de classe ou une liste munie d'un autre attribut est refusé sous plan_mal_forme, etapes_mal_formees ou parametres_mal_formes (issue #23). Leurs noms sont présents, non vides et non manquants (liste_noms_complets(), qui leur applique la règle noms_complets() du module outils, section 2.3) ; un nom en double est refusé sous plan_mal_forme ou parametres_mal_formes (liste_nommee()), mais, dans etapes, sous etape_en_double, qui nomme les étapes répétées, et non sous etapes_mal_formees (issue #60). Elle ne réimplémente pas les contrôles de valeur : elle appelle ceux des modules (controler_parametres_donnees(), controler_parametres_exposition(), controler_parametres_brut()), dont les codes et le module sont ainsi identiques, quelle que soit la fonction qui détecte la condition (ADR 0007, annotation du 17/10/2026, point d). Dans une construction, ces contrôles ne sont faits que là : les fonctions d'étape ne les refont pas (section 6.2).

Exemple de plan (vivant → décédé, par sexe, sans exclusion) :

```
plan <- list(transition = list(depart = "vivant", arrivee = "deces"),
             periode = list(debut = as.Date("2015-01-01"),
                            fin = as.Date("2024-12-31")),
             delai_declaration = 0L, segmentation = "sexe",
             etapes = list(donnees = list(exclusions = character(0)),
                           exposition = list(convention = "plus_fine",
                                             par_annee = FALSE),
                           brut = list(estimateurs = c("poisson", "binomial"),
                                       niveau = 0.95)))
res <- construire_table(donnees, roles, plan)
```

6.2 Exécution

construire_table() procède dans cet ordre : validation du plan, puis sa mise sous forme canonique, une seule fois (canoniser_plan(), section 15.1) ; les étapes et la fiche ne voient que le plan canonique ; contrôle des rôles (controler_roles()) ; déduction de la recette et, si le plan comprend l'étape exposition, de la convention retenue ; contrôle de la provenance ; calcul de l'empreinte des données reçues (section 15.2). Tout refus possible avant le calcul a donc lieu avant la première étape. Chaque contrôle n'y est exécuté qu'une fois (issue #22) : construire_table() appelle chaque fonction d'étape avec son argument interne controler = FALSE, sous lequel elle ne refait aucun de ces contrôles (ses paramètres et, selon l'étape, les rôles, la recette, la convention et la segmentation), et reprend les valeurs qui en dérivent par des fonctions qui ne contrôlent rien (valeurs_parametres_donnees(), recette_branche(), convention_retenue()). Appelée directement, avec la valeur par défaut controler = TRUE, une fonction d'étape fait elle-même ses contrôles, chacun une fois, et en lève les refus ; seule la valeur FALSE les omet. Les étapes s'exécutent ensuite dans l'ordre du plan, chacune par sa fonction d'étape, qui rend ses paramètres effectifs, son résultat, ses diagnostics et ses avertissements ; l'étape exposition lit la table des observations de l'étape donnees ; l'étape brut lit la table d'exposition et les paramètres effectifs de l'étape exposition (découpage par année, convention retenue, unité de l'exposition) et, pour l'estimateur kaplan_meier, les fenêtres des observations calculées par l'étape exposition, avec la transition, la période et le délai du plan. Ces fenêtres sont la seule donnée qu'une étape transmet à une autre hors de

l'objet résultat (issue #38 ; ADR 0007, seconde annotation du 19/10/2026) : `calculer_exposition()` les rend dans un élément `fenetres` à côté de son résultat (section 8.2) ; `construire_table()` ne les conserve que si le plan retient l'estimateur `kaplan_meier`, et les abandonne sinon dès la fin de l'étape `exposition` ; conservées, elle les garde dans une variable locale, les passe à `calculer_taux_bruts()` par son argument `observations`, puis les libère après l'étape `brut`. Seuls les paramètres effectifs, le résultat et les diagnostics de chaque étape entrent dans `etapes` ; les fenêtres n'entrent ni dans l'objet résultat ni dans la fiche, qui sont ceux de la section 6.3. L'étape `exposition` précède toujours l'étape `brut`, préalable que `valider_plan()` contrôle ; dans une construction, l'étape `brut` ne lit donc pas la table des observations de l'étape `donnees` elle-même, mais ses fenêtres. L'objet assemblé, `construire_table()` calcule l'empreinte du résultat sur cet objet (`empreinte_resultat()`, section 15.3), la range dans `fiche$empreintes$resultat`, puis ajoute `execution`, hors du périmètre de l'empreinte.

6.3 Objet résultat

L'objet retourné est une liste nommée de classe S3 `construction` (ADR 0007, point 6.1), lue par `$` et jamais recalculée par ses lecteurs :

— `table` : la table finale ; `NULL` dans cette branche, dont les plans s'arrêtent aux taux bruts ;

— `etapes` : une entrée par étape exécutée, dans l'ordre, avec `parametres` (paramètres effectifs, après résolution par le moteur), `resultat` et `diagnostics` ;

— `avertissements` : le `data.frame` (`etape`, `code`, `message`, `nombre`, ce dernier en `integer`) des avertissements de toutes les étapes, repris tel quel dans la fiche (section 2.2) ;

— `fiche` : la fiche de construction (section 15.1) ;

— `execution` : horodatage de début, durée, version de R, plateforme, système, paramètres régionaux et packages attachés ; **hors de toute comparaison** (rejeu, reproductibilité, non-régression) : c'est le seul champ qui change entre deux exécutions identiques.

Ajouter un champ est compatible ; renommer ou retirer un champ change le contrat partagé « objet résultat » (ADR 0007, point 6.3). C'est à ce titre que le retrait des quatre copies de premier niveau de la fiche (section 15.1) a été décidé par le mainteneur (décision M64, question Q2 ; issue #42).

6.4 Rejeu outillé

Le *rejeu* est l'exécution de `construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance)` sur les données qu'une fiche de construction identifie, pour retrouver la construction qu'elle décrit (E3.7). La note de lecture d'E3.7 du 19/10/2026 (décision M64 ; ADR 0011, point 12) en fait un critère en quatre temps : le relecteur (1) vérifie `version_fiche`, la représentation canonique et l'empreinte des données de la fiche ; (2) exécute la construction sur le plan relu du texte de la fiche ; (3) compare les grandeurs à tolérance par classes (section 4.5) sur une autre machine, au bit près sur la même (E4.6) ; (4) n'emploie l'empreinte du résultat que comme témoin sur la même machine (section 15.3). Le *rejeu outillé*, fonction exportée `rejouer_fiche(donnees, fiche)` de `R/construire_table.R` (issue #86 ; ADR 0012, point 3, précisé par sa seconde annotation du 24/10/2026), outille les temps (1), (2) et (4) — pour le temps (2), la relecture du texte de la fiche reste à l'appelant, par `lire_fiche()` —, et le cas « même machine » du temps (3) par le seul témoin de l'empreinte du résultat, qui ne localise pas un écart ; la comparaison à tolérance par classes sur une autre machine reste hors de l'outil (ci-dessous, « Limites »).

Une fonction de contrôle, non un second point d'entrée (ADR 0012, points 3.2 et 3.3). `rejouer_fiche()` n'exécute aucune étape et n'appelle aucune fonction d'étape : elle contrôle la fiche et les données, puis appelle `construire_table()` **exactement une fois**, avec les rôles, le plan et la provenance

lus dans la fiche, sans en recevoir d'autres ; construire_table() reste la seule fonction qui exécute un plan, et sa signature est inchangée. Elle vit dans R/construire_table.R parce que le module rapport, qui porte la fiche et ses empreintes, ne peut pas appeler construire_table() (section 1.3) ; un troisième fichier transversal, écarté, aurait changé la forme du package pour une seule fonction. Elle reçoit un data.frame, jamais un fichier (E4.4) : la fiche est relue de ses octets par lire_fiche() chez l'appelant, ou prise dans x$fiche.

Étapes, dans cet ordre ; le rejeu s'arrête au premier refus, sans appeler construire_table() :

1. **nature des arguments**, la fiche d'abord : un objet construction ou rejeu reçu à la place de la fiche, ou une fiche qui n'est pas une liste nommée aux noms uniques, sans autre attribut que ses noms (liste_nommee()), est refusé sous fiche_non_conforme ; puis des données qui ne sont pas un data.frame sont refusées par controler_roles(), sous le motif donnees_mal_formees du module donnees, avec le message même qu'en donnerait construire_table() ;
2. **forme des champs que le rejeu lit lui-même** : version_fiche, integer de longueur 1, non NA, sans attribut (forme exacte : un double relu d'un texte édité est refusé) ; empreintes, liste nommée aux noms uniques, sans autre attribut que ses noms, et, si elle l'est, ses éléments algorithme, representation et donnees, chacun chaîne unique, non vide, non NA, sans attribut (chaine_unique()) ; noms roles, plan et provenance présents (provenance peut valoir NULL). **Tous** les champs en défaut sont cités dans un seul refus fiche_non_conforme ; aucun n'est complété ni converti. Le contenu de roles, plan et provenance est laissé à l'étape 6 ;
3. **version admise** : version_fiche appartient à VERSIONS_FICHE_REJEU, sinon refus version_fiche_non_admise. Le contrôle précède ceux du contenu, parce que la version donne le sens des autres champs ;
4. **algorithme de l'empreinte des données** : empreintes$algorithme égal à ALGORITHME_EMPREINTE, sinon algorithme_empreinte_inconnu ;
5. **représentation canonique** : empreintes$representation égale à ENTETE_CANONIQUE, sinon representation_non_admise : un recalcul sous une autre définition échouerait toujours, sous un motif trompeur ;
6. **entrées de la construction** : la séquence de contrôles de construire_table() (section 6.2), refaite en entier, dans son ordre, sur les champs de la fiche : valider_plan(), canoniser_plan(), controler_roles() sur la segmentation canonique, deduire_recette(), resoudre_convention() si l'étape exposition est au plan, controler_provenance(). Ces refus gardent leur module, leur motif et leur message (colonne_absente, type_incompatible, plan_mal_forme, convention_invalide, provenance_invalide…). La séquence est dupliquée, non factorisée, pour ne pas modifier construire_table() ; un test vérifie qu'elle donne le même refus par les deux chemins. Elle précède l'empreinte des données, qui, calculée la première, nommerait mal la cause : une colonne de rôle absente donnerait type_non_canonique (module rapport) et une date en character une empreinte discordante, au lieu de colonne_absente et de type_incompatible (ADR 0012, « Contexte », mesures d'actuary) ;
7. **empreinte des données** : recalculée par empreinte_donnees(donnees, fiche$roles) et comparée à fiche$empreintes$donnees, sinon empreinte_donnees_discordante. Elle est donc calculée deux fois par rejeu, ici et dans construire_table(), qui ne reçoit pas d'empreinte : la lui transmettre aurait changé la signature du point d'entrée ;
8. **construction** : construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance), une fois. Si l'empreinte des données de l'objet rendu diffère de celle de l'étape 7, le rejeu s'arrête sur une erreur de programmation (garde-fou, non un refus) ;

9. **témoins**, consignés dans le bilan, jamais motifs de refus (ci-dessous).

Versions admises : la version courante seule (ADR 0012, point 3.5). `VERSIONS_FICHE_REJEU` vaut `VERSION_FICHE`, écrite comme une expression et non comme un littéral (un test exige leur identité) ; elle est rangée dans `R/rapport.R`, à côté de `VERSION_FICHE`, parce qu'une constante de premier niveau de `R/construire_table.R` ne peut pas lire une constante de `R/rapport.R`, évalué après lui à l'installation (section 15.1, « Ordre des valeurs des champs ensembles ») ; `rejouer_fiche()` la lit à l'appel. Trois raisons : aucune conversion silencieuse d'une fiche d'une autre version (ADR 0011, point 10 (d)) ; une telle fiche donnerait de toute façon un témoin « différente », même sur la même machine, `version_fiche` étant dans le périmètre de l'empreinte du résultat ; aucune fiche d'une version antérieure n'a été livrée hors des tests. Admettre une version antérieure demandera de montrer, version par version, que les champs lus par le rejeu ont la même structure et le même sens. Une fiche de version 3, relue sans refus par `lire_fiche()`, qui ne contrôle pas la version, est ainsi refusée par `rejouer_fiche()`.

Textes des refus. Les cinq motifs propres au rejeu sont levés sous le module `construire_table`, comme les refus de `valider_plan()` (tableau 8) ; les refus délégués (étape 1 pour les données, étape 6) gardent leur module. Chaque message cite la valeur lue et la valeur attendue. La version de `tablexp` et les empreintes du code y sont écrites par `texte_lu()` : « absente » si la valeur manque, elle-même si c'est une chaîne unique sans attribut (`NA` écrit « NA »), « illisible (*type* de longueur *n*) » sinon. Dans toute valeur lue dans la fiche et insérée dans un message, les caractères de contrôle (U+0001 à U+001F, U+007F) sont remplacés par une espace (`sans_controle()`) ; les `details` et le bilan gardent la valeur brute. Les formes lues de l'étape 2 sont écrites par `decrire_forme()` : « champ absent », ou « *type* de longueur *n* », complété au besoin par « , NA », « , vide », « , attribut(s) *noms* » (les noms d'une liste ne sont pas cités comme attribut), « , noms en double : *noms* » et « , noms incomplets ». Dans le tableau, un crochet désigne une valeur insérée.

Table 8 : Refus propres au rejeu outillé (module `construire_table`).

Motif et étape	Message	`details`
`fiche_non_conforme`, étape 1 (objet `construction`)	« Rejeu : objet ‘`construction`‘ reçu à la place de sa fiche ; passer `x$fiche`, ou la fiche relue de son texte par `lire_fiche()`. »	`champ` « (fiche) », `lu` « construction », `attendu` « liste nommée »
`fiche_non_conforme`, étape 1 (objet `rejeu`)	« Rejeu : objet ‘`rejeu`‘ reçu à la place d'une fiche ; passer `r$construction$fiche`. »	idem, `lu` « rejeu »
`fiche_non_conforme`, étape 1 (autre objet)	« Rejeu : la fiche doit être une liste nommée aux noms uniques, sans autre attribut que ses noms ; reçu : objet de classe [classes] (type [type]). Le rejeu ne lit aucun fichier : relire la fiche de ses octets par `lire_fiche(readBin(fichier, "raw", file.size(fichier)))`, ou passer `x$fiche`. »	idem, `lu` les classes
`fiche_non_conforme`, étape 2	« Fiche non conforme au rejeu : « [champ] » : [forme lue] au lieu de [forme attendue] ; … Le rejeu ne lit que ces champs avant `construire_table()` ; aucun n'est complété ni converti. » Formes attendues : « integer de longueur 1, non NA, sans attribut » (`version_fiche`) ; « liste nommée aux noms uniques, sans autre attribut que ses noms » (`empreintes`) ; « chaîne unique, non vide, non NA, sans attribut » (ses trois éléments) ; « champ présent » (`roles`, `plan`) ; « champ présent (NULL admis) » (`provenance`)	`champ`, `lu`, `attendu` : un élément par champ en défaut, dans l'ordre de l'étape

Motif et étape	Message	details
version_fiche_non_admise, étape 3	« Version de fiche non admise au rejeu : version_fiche [version lue] lue, version(s) admise(s) par ce code : [VERSIONS_FICHE_REJEU]. Fiche produite par tablexp [version de la fiche], empreinte du code [empreinte du code de la fiche] ; code installé : tablexp [version installée], empreinte du code [EMPREINTE_CODE]. Aucune conversion de fiche n'est faite : rejouer cette fiche avec le code dont l'empreinte est celle de la fiche. » La version de la fiche est « absente » si versions manque ou est une liste sans élément tablexp	lu, admises, tablexp, code, code_installe
algorithme_empreinte_inconnu, étape 4	« Algorithme d'empreinte « [lu] » lu dans la fiche, « [ALGORITHME_EMPREINTE] » attendu : ce code ne calcule l'empreinte des données que par cet algorithme ; aucune comparaison n'est possible. »	lu, attendu
representation_non_admise, étape 5	« Représentation canonique « [lue] » lue dans la fiche, « [ENTETE_CANONIQUE] » attendue : ce code calcule l'empreinte des données sous cette seule définition, et une empreinte d'une autre définition ne lui est jamais égale ; aucune conversion n'est faite. »	lu, attendu
empreinte_donnees_discordante, étape 7	« Empreinte des données discordante : [empreinte de la fiche] lue dans la fiche, [empreinte recalculée] recalculée sur les données reçues ([nombre] lignes). L'empreinte porte sur la représentation « [ENTETE_CANONIQUE] » des colonnes des rôles de la fiche ([colonnes, entre guillemets]), lignes dans l'ordre reçu : un autre ordre des lignes, une valeur ou un type de colonne différents suffisent à la changer ; les colonnes hors rôles n'y entrent pas. »	lu, attendu, lignes, colonnes

Le message de version_fiche_non_admise cite l'empreinte du code, et non la seule version du package, parce que celle-ci, déclarée dans DESCRIPTION, est immobile (0.0.0.9000) et ne permet pas de retrouver le code d'une fiche ; le moyen de passer d'une fiche au code qui l'a produite reste à choisir (issue #89, ouverte).

Objet rendu et bilan de rejeu (ADR 0012, points 3.1 et 3.6). rejouer_fiche() rend une liste nommée de classe S3 rejeu à deux éléments : construction, l'objet rendu par construire_table(), **sans modification ni attribut** — il reste identical() hors execution à un original, et le périmètre de l'empreinte du résultat n'est pas touché —, et bilan, un data.frame de treize lignes et cinq colonnes character, sans NA (controle, attendu, lu, verdict, statut) : sept contrôles, puis six témoins (tableau 9). La colonne lu porte ce que reçoit le rejeu (la fiche, et les données pour la première ligne), la colonne attendu ce que le code installé exige ou obtient. Le bilan **n'entre pas dans l'objet** : un champ ou un attribut de plus ferait que l'objet rejoué ne serait plus identical() à un original, changerait le périmètre de l'empreinte du résultat, et empreinte_resultat() s'arrête sur une erreur de programmation devant tout attribut inconnu. Un bilan n'existe que pour un rejeu abouti : ses sept lignes de contrôle valent donc toujours « conforme » ; elles disent ce qui a été contrôlé, sur quelles valeurs. Un refus est une condition R (section 2.1) et ne rend aucun bilan.

Table 9 : Bilan de rejeu : une ligne par contrôle, puis par témoin.

controle	attendu / lu	verdict	statut
nature_arguments	« fiche : liste nommée ; donnees : data.frame » / « fiche : liste nommée de [n] champs ; donnees : classe [classes], [n] lignes, [m] colonnes »	conforme	contrôle de forme

controle	attendu / lu	verdict	statut
forme_fiche	formes exigées à l'étape 2 / « champs lus de la forme attendue »	conforme	contrôle de forme
version_fiche	VERSIONS_FICHE_REJEU / version_fiche de la fiche	conforme	comparaison exacte
algorithme_empreinte	ALGORITHME_EMPREINTE / empreintes$algorithme	conforme	comparaison exacte
representation	ENTETE_CANONIQUE / empreintes$representation	conforme	comparaison exacte
entrees_construction	« valider_plan(), controler_roles(), recette et convention, controler_provenance() : aucun refus » / « étapes : [étapes du plan] ; rôles : [rôles de la fiche] ; recette : [recette déduite] ; provenance : [fichier, ou NULL] »	conforme	contrôles délégués aux modules
empreinte_donnees	empreinte recalculée / empreintes$donnees	conforme	exacte ; sans rôle de sécurité
temoin_code	EMPREINTE_CODE / empreintes$code	V1	exacte ; sans rôle de sécurité
temoin_versions	versions de la fiche rendue / de la fiche reçue, en paires « nom=valeur » jointes par « ; »	V2	comparaison exacte
temoin_plateforme	idem pour plateforme	V2	comparaison exacte
temoin_fiche	« fiche rendue par construire_table(), hors versions, plateforme, empreintes/resultat, empreintes/code, empreintes/statut_resultat_code » / « fiche reçue, même périmètre »	V3	comparaison exacte
temoin_empreinte_resultat	empreinte du résultat de l'objet rendu / empreintes$resultat de la fiche	V1	témoin, même machine seulement
temoin_empreinte_fichier	« non recalculée par le moteur (couche de lecture, E4.4) » / empreintes$fichier	non vérifiée	reprise de la fiche, non vérifiée par le moteur

Les libellés forment des ensembles fermés, au féminin singulier (ADR 0012, seconde annotation du 24/10/2026, (a) 4). Verdicts : « conforme », « identique », « différente », « différente : *chemins* », « non déterminable », « non vérifiée ». Statuts : « contrôle de forme », « comparaison exacte », « contrôles délégués aux modules », « exacte ; sans rôle de sécurité », « témoin, même machine seulement », « reprise de la fiche, non vérifiée par le moteur ». Les verdicts des témoins suivent trois règles :

— **V1** (verdict_chaines(), empreintes du code et du résultat) : « non déterminable » si l'un des deux côtés n'est pas une chaîne unique, non vide, non NA, sans attribut (chaine_unique()) — deux NA ne prouvent pas une identité —, sinon « identique » ou « différente » ;

— **V2** (verdict_liste(), versions et plateforme) : « non déterminable » si le côté de la fiche reçue n'est pas une liste sans classe aux noms complets, sinon la règle V3 sur ces deux listes, champ par champ, les noms étant lus dans la fiche et dans l'objet rendu (le champ bibliotheque_c y entre ainsi sans retouche) ;

— **V3** (verdict_chemins(), fiche) : « identique » si les deux côtés sont identical(), sinon « différente : » suivi des chemins qui diffèrent, joints par « , ». Les chemins sont ceux de champs_differents() : deux listes nommées sans classe, aux noms complets et uniques, sont parcourues sur l'union de leurs noms (ordre de la fiche rendue, puis noms propres à la fiche reçue) ; un nom présent d'un seul côté est cité « (absent de la fiche reçue) » ou « (absent de la fiche rendue) » ; toute autre valeur est une feuille, citée par son chemin ; un nœud qui diffère sans qu'aucun de ses enfants diffère est cité « (ordre des champs ou attributs) ». Pour le témoin de la fiche, les deux

côtés sont d'abord privés de versions, plateforme, empreintes/resultat, empreintes/code et empreintes/statut_resultat_code (perimetre_temoin_fiche()).

Ce que dit chaque ligne (E2.20). Les lignes 1 et 2 sont des contrôles de forme, exacts ; les lignes 3 à 5 comparent exactement des libellés (version, nom de l'algorithme, en-tête de la représentation), non des empreintes ; la ligne 6 dit seulement que les contrôles des modules n'ont rien refusé, sans rien ajouter à ce qu'ils vérifient ; la ligne 7 est l'égalité exacte de deux MD5 : l'empreinte des données est portable (exacte sous glibc, non mesurée ailleurs ; section 15.2) et, comme toute empreinte du projet, sans rôle de sécurité : elle identifie un contenu, elle ne le protège pas contre une modification délibérée. Les témoins **signalent et ne refusent jamais** :

— **code** : égalité exacte de l'empreinte du code de la fiche et de celle du code installé (section 15.3, « Empreinte du code »), qui identifie le texte source et non le comportement numérique ; rejouer une fiche avec un code corrigé est un usage légitime (analyse de sensibilité d'un certificateur), d'où un signal et non un refus ; « non déterminable » si l'une des deux vaut NA (sources introuvables à l'installation) ;

— **versions** et **plateforme** : comparaison exacte des chaînes ; un écart dit que l'environnement diffère, sans juger de son effet ;

— **fiche** : un écart révèle un code au comportement différent sur ce que la fiche consigne (recette, convention, fin effective, hypothèses, décomptes d'exclusion, avertissements) ; la fiche ne portant ni exposition ni taux, un écart de dernier bit des grandeurs calculées n'y apparaît que par l'empreinte du résultat. La stabilité de ce témoin d'une machine à l'autre n'est pas mesurée directement ; elle a un *constat* indirect sur deux machines Linux x86_64, par les références de non-régression (glibc 2.35 et processeur AMD EPYC 7763 ; glibc 2.39 et processeur Intel Xeon : R1 à R6 identiques sur tout le périmètre du témoin), et n'est pas mesurée hors glibc ;

— **empreinte du résultat** : témoin d'identité sur la même machine seulement (section 15.3) ; « différente » quand les témoins du code, des versions, de la plateforme et de la fiche sont « identique » est un **signal d'écart à E4.6** si la machine est la même, ce que le bilan ne peut pas établir seul : la fiche ne consigne pas le processeur ; sinon c'est un écart possible et attendu (les empreintes du résultat de R4 et R5 diffèrent entre la plateforme de référence et un hôte hors de celle-ci, par un ulp d'une grandeur, section 15.3), et le critère reste la comparaison à tolérance par classes, hors de l'outil ;

— **empreinte du fichier** : recopiée de la provenance par la couche de lecture et non recalculée par le moteur, qui ne lit aucun fichier : toujours « non vérifiée ».

Limites (ADR 0012, point 3.9, et seconde annotation du 24/10/2026, (a) 7 et 8 ; constats de tests/unitaires/test_rejeu.R).

— **L'identité de l'objet ne vaut qu'à colonnes identiques.** Une colonne hors rôles ajoutée aux données ou retirée ne change pas l'empreinte des données, qui ne porte que sur les colonnes des rôles : le rejeu passe, les témoins du code, des versions, de la plateforme, de la fiche et de l'empreinte du résultat valent « identique » (celui du fichier, toujours « non vérifiée »), mais l'objet rendu n'est pas identical() à l'original, qui ne diffère que par etapes/donnees/resultat/lignes_conservees, copie de l'entrée hors du périmètre de l'empreinte du résultat (décision M71).

— **Un plan modifié dans la fiche ne se voit que par ses échos.** La fiche ne porte pas d'empreinte d'elle-même (voulu) : le plan de la fiche reçue est celui que la construction emploie, si bien que la fiche rendue porte le même plan. Une modification du plan n'apparaît au témoin de la fiche que par ce qu'elle change ailleurs dans la fiche (un niveau modifié se lit dans hypotheses_brut/niveau), et à l'empreinte du résultat sur la même machine ; une modification sans écho dans la fiche passerait inaperçue de ce témoin.

— **Lignes permutées** : refusées (empreinte des données discordante) ; le refus est fondé, l'exposition et les taux n'étant pas `identical()` sous un autre ordre de sommation (ADR 0012, « Contexte », mesure d'`actuary`).

— **Colonne de rôle renommée** : refusée (`colonne_absente`) si les rôles de la fiche ne sont pas adaptés ; adaptés, le rejeu passe, le témoin de la fiche vaut « identique » et celui de l'empreinte du résultat « différente », les rôles étant dans son périmètre.

— **Provenance** : une fiche sans provenance et la même avec provenance se rejouent toutes deux, avec des empreintes du résultat distinctes, `fiche$provenance` étant dans le périmètre.

— **`construire_table()` appelée directement ne contrôle pas la version** : une fiche de version non admise s'y exécute sans refus, et `lire_fiche()` ne la contrôle pas non plus. Seul le rejeu outillé applique ce contrôle.

— **Le temps (3) n'est pas outillé** : la comparaison à tolérance par classes sur une autre machine demande un artefact de valeurs et relèverait d'une issue à part, si le besoin est confirmé (ADR 0012, « Conséquences »).

Les textes fixes et composés du bilan sont du texte du moteur, marqué UTF-8 (`texte_moteur()`) ; les comparaisons portent sur les valeurs brutes. Le bilan du rejeu de R1 est identique octet à octet sous la locale C et sous une locale UTF-8 (constat du contrôle `tests/controles/locale_c.R`, section 5.1).

6.5 Vérifications

`tests/unitaires/test_construire_table.R` vérifie la liste des exports, chaque refus de `valider_plan()` (noms exacts des champs de transition et de période compris), la règle commune de forme (pour chaque paramètre d'étape et chaque champ du plan, une valeur valide munie de noms, d'une dimension ou d'un autre attribut est refusée sous le motif du paramètre, porté par le module qui le contrôle ; issue #23), la cohérence entre `ETAPES` et les fonctions d'étape appelées, la structure de l'objet et de la fiche, et des identités exactes sur un portefeuille simulé (section 8.4). Il vérifie aussi que les contrôles ne sont pas répétés (issue #22) : le nombre d'exécutions de chaque contrôle, compté par `trace()` sur l'espace de noms installé, vaut un par construction (plan avec ou sans `kaplan_meier`, plan réduit à l'étape `donnees` ; aucun contrôle n'est exécuté au-delà d'un refus de `valider_plan()`), et un par contrôle que fait chaque fonction d'étape appelée directement, `taux_kaplan_meier()` n'en faisant aucun ; les refus de l'appel direct gardent leur module et leur motif ; seule la valeur `FALSE` de `controler` omet les contrôles ; sur des entrées valides, chaque fonction d'étape rend sous `controler = FALSE` un objet `identical()` à celui de l'appel avec contrôle, et chaque valeur dérivée sans contrôle est égale à celle que rend le contrôle correspondant ; une segmentation dupliquée passée à `taux_kaplan_meier()` arrête le calcul par une erreur de programmation, non par un refus. `tests/test_reproductibilite.R` vérifie que deux constructions identiques sont `identical()` hors `execution`, dans le même processus et dans un autre, et que le rejeu de la fiche redonne le même objet.

Rejeu outillé (section 6.4 ; issue #86). `tests/unitaires/test_rejeu.R` vérifie le contrat (fonction exportée, arguments `donnees` et `fiche`, `VERSIONS_FICHE_REJEU` `identical()` à `VERSION_FICHE`) ; la structure d'un rejeu réussi (classe, colonnes, lignes, libellés et valeurs du bilan ; objet rendu `identical()` hors `execution` à l'original ; données et fiche inchangées ; même rejeu avec `x$fiche` qu'avec la fiche relue de son texte) ; chaque refus, avec son module, son motif, ses `details` et les valeurs lue et attendue de son message ; l'ordre des contrôles, sur des fiches qui en violent deux à la fois ; pour chaque refus délégué, l'égalité du module, du motif, du message et des `details` avec ceux de `construire_table()` appelée directement sur les champs de la fiche ; le nombre d'appels de `construire_table()`, compté par `trace()` sur l'espace de noms installé : aucun sur un refus, un sur un rejeu réussi ; les verdicts des témoins sur des fiches modifiées ; les cas adverses de la section 6.4, « Limites », testés tels que

constatés. tests/test_reproductibilite.R rejoue, pour les six plans de référence, la fiche écrite puis relue : objet identical() hors execution à l'objet de référence, témoins du code, des versions, de la plateforme, de la fiche et de l'empreinte du résultat « identique », empreinte du fichier « non vérifiée ». tests/controles/locale_c.R compare les octets du bilan du rejeu de R1 sous les deux locales.

6.6 Fonctions du module

Table 10 : Définitions de premier niveau de R/construire_table.R.

Nom	Statut	Entrées	Sortie et rôle
CHAMPS_PLAN	constante	—	les six champs de premier niveau du plan
CHAMPS_PLAN_REQUIS	constante	—	les cinq champs toujours requis (tous sauf graine)
ETAPES	constante	—	étapes : module, fonction d'étape, préalables (colonne liste), tirage d'un aléa
PARAMETRES_ETAPES	constante	—	paramètres de chaque étape : nom, type de base exigé (règle commune de forme, section 6.1), domaine
liste_noms_complets()	interne	objet	TRUE pour une liste nue (seul attribut admis : les noms, par forme_exacte()) aux noms complets au sens de noms_complets() (module outils : présents, non vides et non manquants), ou vide (sans noms admis) ; doublons non contrôlés ; partagée par liste_nommee() et par le contrôle du champ etapes de valider_plan(), qui admet les doublons pour que le refus etape_en_double les nomme (issues #22, #23 et #60)
liste_nommee()	interne	objet	TRUE pour une liste nue (seul attribut admis : les noms) aux noms uniques, non vides et non manquants, ou vide ; liste_noms_complets() sans doublon (issues #23 et #60)
valider_plan()	exportée	plan	plan inchangé, ou refus (section 6.1)
ordres_canoniques()	interne	—	vecteurs d'ordre des trois champs ensembles du plan, passés à canoniser_plan() : segmentation (ROLES_SEGMENTATION), etapes/donnees/exclusions (c(MOTIFS_LIGNES, MOTIFS_TETE)), etapes/brut/estimateurs (ESTIMATEURS_BRUT) ; fonction et non constante, construite à l'appel (section 15.1)

Nom	Statut	Entrées	Sortie et rôle
construire_table()	exportée	données, rôles, plan, provenance	objet construction (section 6.3) ; empreinte du résultat posée avant execution
texte_lu()	interne	valeur lue	« absente » (NULL), la chaîne (chaîne unique sans attribut, NA écrit « NA ») ou « illisible (type de longueur n) », pour un message ou le bilan du rejeu
sans_controle()	interne	texte	caractères de contrôle remplacés par une espace, marque d'encodage conservée ; messages de refus du rejeu seulement
decrire_forme()	interne	conteneur, nom	forme lue d'un champ pour fiche_non_conforme : « champ absent », ou type et longueur complétés de NA, vide, attributs, noms en double, noms incomplets (section 6.4)
paires_lues()	interne	liste	paires « nom=valeur » jointes par « ; », ou texte_lu() d'une autre valeur ; colonnes du bilan pour versions et plateforme
champs_differents()	interne	fiche rendue, fiche reçue, racine	chemins des feuilles qui diffèrent, par descente sur l'union des noms ; absent d'un côté, ou « (ordre des champs ou attributs) » ; vide si identical()
verdict_chemins()	interne	fiche rendue, fiche reçue, racine	verdict V3 : « identique » ou « différente : » et les chemins
verdict_chaines()	interne	attendu, lu	verdict V1 : « non déterminable » si l'un n'est pas une chaîne unique, sinon « identique » ou « différente »
verdict_liste()	interne	liste rendue, liste reçue, racine	verdict V2 : « non déterminable » si la liste reçue n'est pas une liste sans classe aux noms complets, sinon verdict_chemins()
perimetre_temoin_fiche()	interne	fiche	fiche privée de versions, plateforme et des trois dernières empreintes, périmètre du témoin de la fiche
rejouer_fiche()	exportée	données, fiche	objet de classe rejeu (construction, bilan) ; contrôles, puis construire_table() une fois ; refus nommés du module construire_table (section 6.4)

PARTIE III

Module donnees

Données, rôles et contrôles : R/donnees.R

Le module donnees reçoit la base d'observation, un data.frame, et la déclaration des rôles de ses colonnes. Il contrôle les rôles, évalue les motifs d'invalidité de chaque ligne, applique les règles d'exclusion déclarées par le plan, déduit l'unité d'observation et fusionne les intervalles d'observation d'une même tête (E2.2, E2.4, E2.7). Il ne modifie jamais les données : une ligne invalide arrête la construction, sauf règle d'exclusion déclarée, et rien n'est corrigé (E2.7). Sa fonction d'étape est preparer_donnees() ; il porte aussi le catalogue des rôles (catalogue_roles()) et la représentation commune d'une table de référence (table_reference()), toutes deux exportées. Le contenu de fond (catalogue, motifs, règles de fusion) vient de la spécification d'actuary du 16/10/2026, versée dans l'issue #11 (parties A.0 et A.1).

7.1 Conventions de temps

Trois conventions de temps sont communes à toutes les recettes (E2.3) ; elles sont appliquées par donnees et par exposition (section 8), qui ajoute l'unité de l'exposition et la fenêtre d'observation.

— **Une date est un jour** (classe Date, sans heure ni fuseau). Un intervalle d'observation est l'intervalle d'instants semi-ouvert $[\text{entrée}, \text{sortie})$: date_entree est l'instant où l'observation commence, date_sortie celui où elle cesse (jour du décès ou du rachat, ou lendemain du dernier jour couvert pour une fin de garantie). Un événement daté du jour s survient à l'instant s : la tête est exposée jusqu'à s, jour s exclu. Une ligne en_cours est ouverte à droite. *Choix de conception* : il donne des décomptes entiers et additifs, au prix d'une sous-estimation de l'exposition d'un demi-jour par événement en moyenne.

— **Âge au dernier anniversaire.** L'âge au jour t d'une tête née le jour n est le nombre d'années révolues

$$\text{age}(n,t) = \text{an}(t) - \text{an}(n) - \mathbb{1}\{(\text{mois}(t), \text{jour}(t)) < (\text{mois}(n), \text{jour}(n))\},$$

comparaison lexicographique, appliquée sans cas particulier (age_revolu()). Pour une naissance le 29 février, l'anniversaire tombe donc le 1er mars des années non bissextiles : le 28 février d'une telle année, $(2, 28) < (2, 29)$ et l'année n'est pas révolue. Une année d'âge dure ainsi 365 ou 366 jours. Convention de calcul, conséquence de la formule (spécification, A.0, T2), validée par actuary le 17/10/2026, d'impact négligeable (un jour d'exposition déplacé entre deux âges, pour les seules naissances du 29 février) ; aucune source juridique n'est établie à son sujet. Elle est appliquée par trois arithmétiques de dates écrites indépendamment (modules donnees et exposition, simulateur), dont l'accord est vérifié (section 8.4).

— **Période d'observation et délai de déclaration.** La période [début, fin] est déclarée par le plan, fin incluse, jamais déduite des données (E2.5). Le délai de déclaration est un nombre entier de jours ≥ 0, sans arithmétique de mois. La fin effective est fin $-$ délai et la borne d'instant B = fin effective $+ 1$: l'observation s'arrête à l'instant B. Une période dont le début est postérieur à la fin effective est refusée (`periode_vide`).

7.2 Catalogue des rôles

Le moteur ne lit une colonne que par son *rôle*, déclaré par l'utilisateur dans un catalogue fermé (E2.2 ; ADR 0003, point 2). Les rôles sont transmis à `construire_table()` sous la forme d'un vecteur de chaînes nommé : noms = rôles du catalogue, valeurs = noms de colonnes de la base (ADR 0007, point 4.1). Les colonnes sans rôle sont ignorées. `catalogue_roles()`, exportée pour que l'application propose la déclaration des rôles, rend le catalogue sous la forme d'un `data.frame` de colonnes `role`, `famille`, `type_admis`, `usage` et `recettes` (tableau 11).

La colonne *usage* décrit l'emploi du rôle par la seule recette d'exposition disponible, `lignes_dates_completes` (section 8.1) :

— `requis` : rôle sans lequel aucune recette ne s'applique (refus `aucune_recette`) ;

— `facultatif` : rôle lu pour des contrôles (unicité du contrat, cohérence de l'année de naissance) ou, pour `id_tete`, pour l'unité d'observation ;

— `segmentation` : rôle requis si la segmentation du plan l'emploie (refus `role_requis_manquant`), contrôlé s'il est seulement déclaré, avec l'avertissement `role_non_employe` ;

— `non_pris_en_charge` : rôle qu'aucune recette disponible n'emploie et dont l'omission changerait le sens du résultat (pondération, agrégats, états, date de survenance distincte de la sortie, date d'inventaire, âge fourni) ; sa déclaration est **refusée** (`role_non_pris_en_charge`) plutôt qu'ignorée en silence. Le rôle `montant` est de ceux-là : la pondération par les montants attend la spécification de ses intervalles (E2.8).

Les types sont contrôlés sans conversion (E4.4) : une colonne de dates doit être de classe `Date`, exactement, à valeurs entières et finies ; une colonne `POSIXct`, une chaîne ou un nombre sont refusés (`type_incompatible`), la conversion revenant à la couche de lecture. Au contrôle de type, une valeur manquante, stockée `NA` ou `NaN`, est admise (les motifs de lignes en décident ensuite), et les deux restent distinctes dans l'empreinte (section 15.2). Une date de type conforme doit en outre appartenir au **domaine** [0001-01-01, 9999-12-31] : hors de lui, aucune écriture ISO 8601 à quatre chiffres d'année n'existe, et la date est refusée (`date_hors_domaine`, tous les rôles en cause cités avec leurs lignes) avant tout calcul, empreinte comprise (`dates_hors_domaine()` ; ADR 0011, point 4 ; décision M64, Q7 ; issue #19). Les années 1 à 999 sont admises. Le motif `age_aberrant` mesure l'âge à la plus petite des dates de sortie et de fin de période : une ligne née avant l'an 1000 et observée dans une période postérieure à l'an 1129 a au moins 130 ans à cette date et est relevée ; une ligne entièrement antérieure à la période ne l'est pas, mais elle n'a aucune intersection avec la période. Les modalités des rôles catégoriels sont comparées après conversion en chaîne, en respectant la casse : `H` et `F` pour `sexe` ; `deces`, `rachat`, `terme`, `autre` et `en_cours` pour `cause_sortie` (constantes `MODALITES_SEXE` et `MODALITES_CAUSE`). Un recodage (par exemple `1/2` vers `H/F`) relève de la couche de lecture.

Table 11 : Catalogue fermé des rôles (`catalogue_roles()` ; constantes `CATALOGUE_ROLES` et `TYPES_ADMIS`). La colonne `recettes` du catalogue vaut `lignes_dates_completes` pour tout rôle pris en charge, vide sinon.

Rôle	Famille	Types R admis	Usage
`id_contrat`	identification	`character`, `integer`, `double` entier fini	facultatif
`id_tete`	identification	`character`, `integer`, `double` entier fini	facultatif
`sexe`	état civil	`character`, `factor`	segmentation
`date_naissance`	état civil	`Date` (valeurs entières finies)	requis
`annee_naissance`	état civil	`integer`, `double` entier fini	facultatif
`age_fourni`	état civil	`integer`, `double` entier fini ≥ 0	non pris en charge
`date_age_fourni`	état civil	`Date` (valeurs entières finies)	non pris en charge
`date_entree`	observation	`Date` (valeurs entières finies)	requis
`date_sortie`	observation	`Date` (valeurs entières finies ; manquante admise pour une ligne `en_cours`)	requis
`cause_sortie`	observation	`character`, `factor`	requis
`date_evenement`	observation	`Date` (valeurs entières finies)	non pris en charge
`date_inventaire`	observation	`Date` (valeurs entières finies)	non pris en charge
`age`	agrégats	`integer`, `double` entier fini	non pris en charge
`annee`	agrégats	`integer`, `double` entier fini	non pris en charge
`exposition`	agrégats	`double` fini ≥ 0	non pris en charge
`evenements`	agrégats	`double` fini ≥ 0	non pris en charge
`montant`	pondération	`double` fini > 0	non pris en charge
`segment`	segmentation	`character`, `factor`, `integer`	segmentation
`etat_depart`	multi-états	`character`, `factor`	non pris en charge
`etat_arrivee`	multi-états	`character`, `factor`	non pris en charge
`date_changement`	multi-états	`Date` (valeurs entières finies)	non pris en charge

7.3 Contrôles, refus et exclusions

Refus au niveau des rôles et des paramètres

`controler_roles()` contrôle la déclaration des rôles dans cet ordre, et chaque refus nomme tous les rôles et colonnes en cause : `donnees_mal_formees` (l'entrée n'est pas un `data.frame`) ; `roles_mal_formes` (les rôles ne sont pas un vecteur de chaînes nommé, de forme exacte `character` sans autre attribut que ses noms — une classe, `noquote` ou `AsIs` comprises, une dimension ou un attribut quelconque sont refusés, par `forme_exacte()` —, sans valeur ni nom manquant ni nom vide ; `details$classe` et `details$attributs` donnent la classe et les noms des attributs reçus, qui désignent la cause quand elle tient à une classe, une dimension ou un attribut, mais non quand elle tient à une valeur ou à un nom manquant ou vide ; issue #62) ; `role_inconnu` ; `role_declare_deux_fois` ; `role_non_pris_en_charge` ; `colonne_absente` ; `colonne_ambigue` (le nom de colonne figure plusieurs fois dans la base : lire la première sans le dire serait deviner) ; `colonne_declaree_deux_fois` (une colonne pour deux rôles) ; `type_incompatible` ; `date_hors_domaine` (une ou plusieurs colonnes de date portent une date hors de [0001-01-01, 9999-12-31] ; le refus nomme tous les rôles et colonnes en cause et, dans `details$lignes`, les lignes de chacun, comme `ligne_invalide`) ; `aucune_recette` (un rôle requis manque ; le message nomme la recette et les rôles absents) ; `role_requis_manquant` (un rôle employé par la segmentation n'est pas déclaré).

`controler_parametres_donnees()` contrôle les champs du plan que l'étape reçoit, les cinq premiers motifs ci-dessous par `controler_transition_periode()`, qui contrôle la transition, la période et le délai et que la fonction d'étape `brut` appelle aussi pour `kaplan_meier` en appel direct (section 9.1), les suivants

par elle-même : transition_inconnue (liste qui n'est pas nue — un data.frame ou une liste munie d'un autre attribut que ses noms —, de champs autres qu'exactement depart et arrivee, ou d'un élément de plus, même nommé NA ; état qui n'est pas une chaîne unique sans attribut, ni NA ni vide, refusé dès ce contrôle de forme ; ou transition non prise en charge : seules vivant → décédé et en cours → racheté le sont, constante TRANSITIONS) ; periode_invalide (liste qui n'est pas nue, de champs autres qu'exactement debut et fin, ou d'un élément de plus ; bornes manquantes, non Date, non entières ou munies d'un autre attribut que la classe Date) ; date_hors_domaine (borne de la période hors de [0001-01-01, 9999-12-31], même domaine que les données ; details$bornes nomme les bornes en cause) ; delai_invalide (délai qui n'est pas un nombre unique, integer ou double, sans attribut, entier et ≥ 0) ; periode_vide ; segmentation_invalide (autre chose qu'un vecteur character sans attribut, sans NA ni doublon, pris dans sexe et segment, vide explicite admis ; contrôle controler_segmentation(), partagé avec la fonction d'étape calculer_taux_bruts() en appel direct, section 9.1) ; exclusions_invalides (autre chose qu'un vecteur character sans attribut, sans NA ni doublon, vide explicite admis) ; exclusion_inconnue (code absent de la liste fermée des motifs). Les deux derniers sont contrôlés dans cet ordre : exclusions_invalides l'emporte, si bien qu'un vecteur nommé ou en double de codes inconnus est refusé sous ce motif et non sous exclusion_inconnue. Les noms de champs sont lus exactement, jamais par appariement partiel. Le même contrôle est appelé par valider_plan(), avant toute étape : un code désigne une condition, quelle que soit la fonction qui la détecte, et le refus porte le module donnees (ADR 0007, annotation du 17/10/2026, point d). Dans une construction, il n'est appelé que là, une fois : preparer_donnees(), appelée avec controler = FALSE, ne contrôle ni ces paramètres ni les rôles, déjà contrôlés, et reprend la transition retenue, les bornes de la période et la borne B par valeurs_parametres_donnees() ; appelée directement, elle appelle ce contrôle, puis controler_roles() (section 6.2).

Motifs d'invalidité des lignes

Les motifs forment une liste fermée (E2.7), évaluée par evaluer_motifs() (tableau 12).

— **Exhaustivité** : tous les motifs de ligne sont évalués sur toutes les lignes, sans court-circuit ; un motif de comparaison n'est pas levé si l'une de ses dates est manquante (le motif de la valeur manquante l'est) ;

— **motifs de tête** : évalués, quand id_tete est déclaré, sur les lignes exemptes de motif de ligne (lignes *propres*), ils marquent les lignes propres de la tête, et elles seules : une ligne déjà porteuse d'un motif de ligne n'est pas comptée de nouveau sous le motif de tête. Les lignes conservées n'en dépendent pas, puisqu'une telle ligne est de toute façon refusée ou exclue au titre de son motif de ligne ; seul le décompte du motif de tête, dans le refus et dans les diagnostics, l'omet (constat : une tête de trois lignes dont la première porte annee_naissance_incoherente et dont les naissances diffèrent donne tete_naissance_incoherente sur les lignes 2 et 3 seulement, et le refus annonce deux lignes pour ce motif) ; le rapport est ainsi complet en un seul passage ;

— **décompte** : une ligne peut porter plusieurs motifs ; le décompte par motif compte les lignes porteuses, et le nombre de lignes distinctes est donné à part.

Table 12 : Motifs d'invalidité, dans l'ordre de restitution (constantes MOTIFS_LIGNES et MOTIFS_TETE). Ce sont aussi les codes admis comme règles d'exclusion.

Code	Condition
Motifs de ligne	
id_contrat_manquant	id_contrat déclaré et manquant
id_tete_manquant	id_tete déclaré et manquant
naissance_manquante	date de naissance manquante

Code	Condition
entree_manquante	date d'entrée manquante
sortie_manquante	date de sortie manquante et cause autre que en_cours
cause_manquante	cause de sortie manquante
cause_hors_modalites	cause hors des cinq modalités
sexe_manquant	sexe déclaré et manquant
sexe_hors_modalites	sexe déclaré, hors de H et F
segment_manquant	segment déclaré et manquant
naissance_apres_entree	naissance postérieure à l'entrée (l'égalité est admise)
sortie_avant_entree	sortie antérieure à l'entrée (l'égalité est admise : durée nulle)
en_cours_avant_fin	cause en_cours et date de sortie renseignée antérieure à la fin déclarée : la base ne couvre pas la période déclarée. Une ligne en_cours dont la date de sortie renseignée précède l'entrée relève de sortie_avant_entree et, si sa sortie précède la fin déclarée (toujours le cas quand son entrée ne la dépasse pas), de ce motif-ci : elle se contredit, le moteur ne choisit pas entre la cause et les dates et refuse (E2.7) ; elle s'écarte par une exclusion déclarée de chacun des motifs qu'elle porte (exclure le seul sortie_avant_entree laisse le refus sous en_cours_avant_fin si elle le porte ; issue #20)
annee_naissance_incoherente	annee_naissance déclarée et différente de l'année de la date de naissance
age_aberrant	âge au dernier anniversaire au moins égal à AGE_ABERRANT (130) à la plus petite des dates de sortie et de fin de période (à la fin si la sortie manque) ; constante de catalogue, *choix de conception* : la borne laisse passer toute durée de vie humaine connue, elle n'est pas un paramètre du plan
contrat_duplique	id_contrat déclaré et présent sur plusieurs lignes (toutes les lignes du groupe sont listées)
Motifs de tête (si id_tete est déclaré)	
tete_naissance_incoherente	dates de naissance différentes entre lignes d'une même tête
tete_sexe_incoherent	sexes différents au sein d'une tête (sexe déclaré)
tete_segment_incoherent	segments différents au sein d'une tête, si la segmentation emploie segment
tete_observee_apres_deces	le premier décès s_d de la tête est antérieur à B et une ligne de la tête a une fin d'observation postérieure à s_d (lecture B de E2.4, ci-dessous)

Refus par défaut et exclusions déclarées

Une ligne qui porte un motif **non déclaré** dans les règles d'exclusion du plan arrête la construction (refus ligne_invalide), même si elle porte aussi un motif déclaré. Le message nomme, pour chaque motif en cause, le nombre de lignes et leurs dix premiers numéros (NB_LIGNES_CITEES, numéros 1 à n des lignes du data.frame reçu), puis le total des lignes distinctes ; la condition porte la liste complète (E2.7, E5.5).

Le plan peut déclarer des *règles d'exclusion*, paramètre exclusions de l'étape donnees : un vecteur de codes de motif sans doublon, vide explicite admis. Une ligne est exclue si l'un de ses motifs est déclaré. Les exclusions sont comptées et listées par motif dans le résultat de l'étape (resultat$exclusions, resultat$lignes_exclues) et chacune donne un avertissement exclusion ; une ligne exclue sous plusieurs motifs est comptée sous chacun, et le nombre de lignes distinctes exclues est donné à part (resultat$nombre_lignes_exclues, repris par la fiche, section 15.1). Si toutes les lignes sont exclues, la construction est refusée (aucune_ligne_conservee) ; une base sans ligne l'est aussi (base_vide). Le diagnostic diagnostics$motifs restitue, pour chaque motif, sa nature (ligne ou tête), son décompte et s'il est exclu.

Les lignes conservées sont celles de l'entrée : donnees lui-même s'il n'y a aucune exclusion, sinon le sous-ensemble de ses lignes, sans aucune modification de valeur ; les quantités dérivées (observations fusionnées) sont rangées à part. Une réserve technique : l'extraction des lignes conservées par l'opérateur d'indexation de R retire des colonnes les attributs autres que la classe et les niveaux d'un facteur (par exemple tzone, label ; constat sur R 4.3.3) ; les valeurs et les classes ne changent pas, et le moteur ne lit pas ces attributs.

7.4 Unité d'observation et fusion par tête

Unité d'observation, déduite des rôles

L'unité d'observation n'est pas un paramètre du plan : le moteur la déduit des rôles et de la transition, comme il déduit la recette (décision M48 ; ADR 0007, annotation du 17/10/2026, point a), et l'inscrit dans les paramètres effectifs de l'étape et dans la fiche :

— **tête**, si l'état de départ de la transition est un état de la personne (vivant → décédé) et que id_tete est déclaré ;

— **contrat** sinon : transition dont l'état de départ est un état du contrat (en cours → racheté, décision M46), ou transition de tête sans id_tete. Dans ce second cas, l'avertissement unite_contrat est inscrit dans l'objet et repris dans la fiche : « une tête titulaire de plusieurs contrats est comptée plusieurs fois, ce qui surestime l'exposition et peut compter plusieurs fois un même décès » (cas F4 des tests : deux décès comptés au lieu d'un).

Un paramètre d'unité serait soit redondant avec les rôles, soit en contradiction avec eux : une tête déclarée « contrat » alors que id_tete est présent compterait plusieurs fois un même décès. Pour le rachat, l'unité est le contrat ; l'identifiant de tête sert alors aux contrôles de cohérence et au futur rééchantillonnage par têtes (E2.4, E2.14) : il est gardé dans la table des observations.

Fusion des intervalles d'une tête

En unité tête, les intervalles $[\text{entrée}, \text{fin})$ des lignes d'une même tête (fin $= +\infty$ pour une ligne en_cours) sont triés par entrée et fusionnés quand entrée suivante $\leq$ fin courante, ce qui couvre le chevauchement et la contiguïté exacte (fusionner_intervalles()). **Aucune tolérance « à un jour près »** : un écart d'un jour reste un trou non exposé, et la tête est de nouveau tronquée à gauche à sa réentrée ; combler l'écart inventerait une exposition. Un tel écart trahit souvent une date de fin inclusive dans la base : il est relevé et décompté à part, non corrigé (*choix de conception*). La cause de fin d'une composante fusionnée est deces si une ligne de décès se termine à cette fin, en_cours si la fin est $+\infty$, censure sinon ; le décès n'est compté qu'une fois.

Le calcul de la fusion est vectorisé par un maximum cumulé décalé par tête, exact en double tant que (nombre de têtes) $\times$ (largeur du décalage) reste inférieur à 2^{52}, la largeur valant $\max - \min + 2$ jours sur les dates d'entrée et de sortie (fusionner_intervalles()). Au-delà, le moteur refuse (dates_hors_plage, dont le message cite l'étendue et le nombre de têtes) au lieu de fusionner inexactement. Depuis le refus des dates hors du domaine (issue #19), qui compte 3 652 059 jours, la largeur est au plus de 3 652 060 : par construire_table(), le refus n'est atteint qu'au-delà d'environ $1{,}23 \cdot 10^9$ têtes ($2^{52}/3\,652\,060$), sous le plus grand entier de R ; la garde est conservée (avis d'actuary), une étendue plus grande restant possible par un appel direct de la fonction de fusion. C'est une limite d'implémentation, non un motif de ligne : elle n'est pas excluable.

Observation après le décès : lecture B de E2.4

Une tête dont un contrat est observé après son décès a une trajectoire indéterminée, et ses lignes sont refusées (E2.4). Le décès doit cependant être connu : le mainteneur a retenu une *lecture B* (décision M48) : le motif `tete_observee_apres_deces` n'est levé que si le premier décès s_d de la tête est **antérieur à la borne** B de la période effective et qu'une fin d'observation d'une ligne de la tête le dépasse. Une ligne `en_cours` compte pour $+\infty$, quelle que soit sa date de sortie, et une sortie manquante aussi (`deces_par_tete()`). Deux lignes closes par le décès le même jour sont cohérentes.

Si $s_d \geq B$, le décès est hors de la période effective : aucune ligne n'est refusée, mais le relevé `observations_apres_deces_hors_periode` compte ces têtes et donne un avertissement du même code. Ce relevé est calculé **sur les seules lignes conservées** : une tête écartée par une exclusion déclarée n'y est pas comptée, une tête dont une ligne seulement est exclue l'est sur ses lignes conservées. Il est calculé dès que `id_tete` est déclaré, **quelle que soit l'unité**, donc aussi en unité contrat pour le rachat (contrôle par identifiant de tête, E2.4, M46, M48) ; sans `id_tete`, il n'est pas calculable et vaut `NA`.

Relevés et avertissements

L'étape restitue dans `diagnostics$releves`, sans bloquer, le nombre de têtes et de têtes multi-contrats, de chevauchements, de contiguïtés, d'intervalles disjoints, d'écarts d'un jour, de fenêtres vides (observations sans intersection avec la période), d'observations, et le relevé après décès hors période. Les relevés de fusion valent `NA` sans `id_tete`. Ses avertissements sont : `unite_contrat` ; `role_non_employe` ; `exclusion` ; `chevauchements`, `disjoints` et `ecarts_un_jour`, en unité tête seulement ; `fenetres_vides` et `observations_apres_deces_hors_periode`, dans toutes les unités. Les contiguïtés sont décomptées sans avertissement : une contiguïté exacte est fusionnée sans perte.

Le résultat de l'étape (`resultat`) contient les lignes conservées et leurs numéros, les exclusions, l'unité retenue, la table des observations (une ligne par composante fusionnée ou par contrat : identifiant, numéro de la première ligne, nombre de lignes, date de naissance, sexe s'il est déclaré, segment s'il est employé, début, fin, cause) et, pour chaque ligne conservée, le numéro de son observation. Les paramètres effectifs (`parametres`) sont le code interne de la transition (`vivant_decede` ou `en_cours_rachete`), l'unité d'observation, la segmentation, les exclusions et la fin effective.

Copie des colonnes dans la table des observations (issue #67 ; décisions M71, point 4, et M72 ; ADR 0011, septième annotation du 20/10/2026, (e), et huitième). La table des observations est une quantité dérivée : les colonnes qu'elle recopie de l'entrée (identifiants de contrat et de tête, date de naissance, sexe, segment) y sont **normalisées**, l'entrée et sa copie `lignes_conservees` restant intactes, si bien que rien n'est corrigé dans les données (E2.7). Une chaîne, un entier ou une date est recopié par sous-indiçage (première ligne de chaque composante en unité tête, chaque ligne conservée en unité contrat), dans les deux unités, ce qui garde valeurs, type et classe et retire les attributs de colonne de l'entrée (par exemple `label`). Un facteur `sexe` ou `segment` est reconstruit par la fonction locale `categorie` de `preparer_donnees()` : codes sous-indicés, niveaux tels quels (ordre et niveaux inutilisés compris), classe ramenée à `factor`, ou à `c("ordered", "factor")` s'il est ordonné ; `contrasts`, sous-classe et tout autre attribut sont retirés. La table des observations porte des noms de lignes automatiques, même si une colonne d'identifiant porte l'attribut `names` (par exemple dans un `data.frame` construit par `list2DF()`), dont `data.frame()` tirerait sinon ses noms de lignes (`row.names = NULL` dans les deux unités). Motif : l'empreinte du résultat écrit la table des observations et arrête le calcul, par une erreur de programmation, sur tout attribut ou toute classe hors de sa grammaire (section 15.3) ; sans cette normalisation, `construire_table()` échouait sur des entrées valides : attribut `label` sur `id_contrat`, `id_tete` ou `date_naissance` en unité contrat, où ces colonnes étaient recopiées entières, ou facteur muni de `contrasts` ou de sous-classe, que le contrôle des types des rôles admet (section 7.2). Option écartée : refuser ces facteurs à la validation, ce qui resserrerait

le contrat d'entrée pour une métadonnée sans effet sur le résultat. Pour les entrées déjà acceptées, la table des observations est inchangée : celles des plans R1 à R6 sont restées identiques à la régénération des références qui a suivi (section 4.5).

7.5 Table de référence

Toute table de référence — loi vraie synthétique du simulateur, table embarquée, table importée — est lue par le moteur sous une seule forme : la classe S3 `table_reference`, sur un `data.frame` en format long (ADR 0007, point 7.1, et annotation du 17/10/2026, point b), que construit et contrôle `table_reference(x, nom, statut, source, parametres, forme_origine)`, exportée pour l'import de tables par l'application :

— colonnes `age` (entier ≥ 0), `sexe` (`H`, `F`, ou `ensemble` pour une table sans distinction de sexe, modalité qui ne coexiste jamais avec `H` ou `F`, dans la table ou au sein d'une même année), `qx` (double fini dans $[0, 1]$) et, facultative, `annee` (entier), pour les tables prospectives de la v2 ;

— âges sans doublon, croissants dans l'ordre reçu et contigus, au sein de chaque sexe (et de chaque année) ;

— attributs `nom`, `statut` (`synthetique`, `embarquee` ou `importee`), `source` (arrêté, URL, fichier de provenance), `forme_origine` (`qx` ou `lx`, défaut `qx`), chacun une chaîne unique de forme exacte — sans attribut, noms compris, ni `NA` ni vide (`chaine_unique()`, section 2.3 ; une chaîne nommée est refusée sous le motif du champ ; issue #48) —, et `parametres` : une liste non vide est **requise** pour une table synthétique, qui n'a d'autre provenance que sa loi, et **interdite** pour une table embarquée ou importée, qu'elle ferait passer pour dérivée d'une loi.

Le constructeur refuse au lieu de corriger, avec un code par contrôle : `table_nom_manquant`, `table_statut_inconnu`, `table_source_manquante`, `table_parametres_manquants`, `table_parametres_inattendus`, `table_forme_inconnue`, `table_mal_formee`, `table_colonne_manquante`, `table_colonne_inconnue`, `table_vide`, `table_colonne_mal_typee`, `table_sexe_hors_modalites`, `table_sexes_melanges`, `table_qx_non_fini`, `table_qx_hors_bornes`, `table_age_double`, `table_ages_non_croissants` et `table_ages_non_contigus`.

7.6 Vérifications

`tests/unitaires/test_donnees.R` vérifie le catalogue, chaque refus de rôle et de paramètre, chaque motif et les règles d'évaluation, les exclusions (lignes conservées identiques à l'entrée), la lecture B de E2.4 au jour près de la borne B (délais 0 et 30 jours) et le relevé après décès hors période (lignes conservées, deux unités), la table de référence et ses refus. Les cas de fusion F1 à F7 de la spécification (chevauchement, contiguïté, écart d'un jour, décès sur deux contrats, observation après décès, deux décès, naissances discordantes) ont des attendus calculés à la main ; la fusion est vérifiée en outre contre un second algorithme, une boucle naïve écrite dans le test. Le domaine des dates est vérifié à ses bornes : 0001-01-01 et 9999-12-31 admis, la veille et le lendemain (années 0 et 10 000) refusés (`date_hors_domaine`), pour les colonnes de date comme pour les bornes de la période ; la garde `dates_hors_plage` l'est à l'étendue maximale du domaine, de part et d'autre du nombre de têtes limite.

7.7 Fonctions du module

Table 13 : Définitions de premier niveau de R/donnees.R.

Nom	Statut	Entrées	Sortie et rôle
CATALOGUE_ROLES	constante	—	catalogue fermé des rôles (rôle, famille, code de type, usage)
TYPES_ADMIS	constante	—	libellé des types R admis par code de type
RECETTE_BRANCHE	constante	—	seule recette d'exposition disponible : lignes_dates_completes
MODALITES_SEXE	constante	—	H, F
MODALITES_CAUSE	constante	—	deces, rachat, terme, autre, en_cours
ROLES_SEGMENTATION	constante	—	sexe, segment
AGE_ABERRANT	constante	—	130 (motif age_aberrant)
dates_hors_domaine()	interne	jours depuis 1970-01-01	vrai hors de $[-719162, 2932896]$, soit hors de [0001-01-01, 9999-12-31], infinis compris, faux pour NA et NaN ; fonde le refus date_hors_domaine
TRANSITIONS	constante	—	transitions prises en charge : départ, arrivée, code interne, porteur de l'état de départ (tête ou contrat)
MOTIFS_LIGNES	constante	—	motifs de ligne (tableau 12)
MOTIFS_TETE	constante	—	motifs de tête (tableau 12)
NB_LIGNES_CITEES	constante	—	10 numéros de ligne cités par motif dans un message de refus
catalogue_roles()	exportée	—	catalogue des rôles (tableau 11)
composantes_date()	interne	jours depuis 1970-01-01	année, mois, jour
age_revolu()	interne	naissance, date	âge au dernier anniversaire (section 7.1)
entiers_finis()	interne	vecteur	valeurs non manquantes entières et finies
type_conforme()	interne	colonne, code de type	conformité au type du catalogue, sans conversion
liste_guillemets()	interne	chaînes	liste entre guillemets pour les messages
message_aucune_recette()	interne	rôles requis, manquants	message du refus aucune_recette, partagé avec exposition
controler_roles()	interne	données, rôles, segmentation	refus au niveau des rôles, dont roles_mal_formes (forme exacte : seul attribut admis, les noms ; issue #62) et date_hors_domaine ; rend les rôles inchangés

Nom	Statut	Entrées	Sortie et rôle
controler_segmentation()	interne	segmentation, étape appelante	seul endroit où s'écrivent la forme et le catalogue de la segmentation : vecteur character sans attribut, sans NA ni doublon, dont chaque clé est prise dans ROLES_SEGMENTATION, vide explicite admis ; sinon refus segmentation_invalide au nom de l'étape appelante (donnees ou brut) ; rend NULL ; appelée par le contrôle des paramètres de l'étape donnees et, en appel direct (controler = TRUE), par la fonction d'étape brut (issues #22 et #59)
ligne_transition()	interne	transition (list(depart, arrivee), de forme déjà contrôlée)	ligne de TRANSITIONS de la transition, zéro ligne si elle n'est pas prise en charge ; seul endroit où s'écrit la recherche (issue #22)
valeurs_parametres_donnees()	interne	transition, période, délai, déjà contrôlés	transition retenue (ligne_transition()), début, fin, borne B (jours depuis 1970-01-01) ; ne contrôle rien ; employée par les fonctions d'étape appelées avec controler = FALSE et par taux_kaplan_meier() (issue #22)
controler_transition_periode()	interne	transition, période, délai	refus de la transition, de la période (dont date_hors_domaine pour ses bornes), du délai et de la période vide, dans cet ordre ; rend la transition retenue, le début, la fin et la borne B, calculés par la fonction sans contrôle de la ligne précédente ; appelée par le contrôle des paramètres de l'étape donnees et, pour kaplan_meier en appel direct, par la fonction d'étape brut (issue #22)
controler_parametres_donnees()	interne	transition, période, délai, segmentation, exclusions	refus de la transition, de la période et du délai par le contrôle de la ligne précédente, puis de la segmentation et des exclusions ; transition retenue, début, fin, borne B
groupes_heterogenes()	interne	groupes, valeurs	groupes où la valeur n'est pas constante
extremes_par_groupe()	interne	groupes, valeurs	minimum et maximum par groupe
deces_par_tete()	interne	têtes, sorties, causes	premier décès de chaque tête et observation au-delà
evaluer_motifs()	interne	données, rôles, période, borne, segmentation	numéros des lignes porteuses de chaque motif
fusionner_intervalles()	interne	têtes, entrées, fins	composantes fusionnées et relevés ; refus dates_hors_plage

Nom	Statut	Entrées	Sortie et rôle
preparer_donnees()	interne	données, rôles, transition, période, délai, segmentation, exclusions, controler (TRUE par défaut)	fonction d'étape donnees : paramètres effectifs, résultat, diagnostics, avertissements ; avec controler = FALSE, réservé à construire_table(), ni les paramètres ni les rôles ne sont recontrôlés (issue #22)
STATUTS_TABLE	constante	—	synthetique, embarquee, importee
SEXES_TABLE	constante	—	H, F, ensemble
FORMES_ORIGINE	constante	—	qx, lx
table_reference()	exportée	table, nom, statut, source, paramètres, forme d'origine	objet table_reference contrôlé (section 7.5)

Partie IV

Module exposition

Exposition et événements : R/exposition.R

Le module exposition calcule, pour la transition étudiée, l'exposition au risque et le nombre d'événements par cellule (segment, âge et, sur demande, année civile), en traitant explicitement la troncature à gauche et la censure à droite (E2.3, E2.5, E2.6 ; ADR 0003, points 3, 6 et 8). Sa fonction d'étape est calculer_exposition() ; elle reçoit la table des observations de l'étape donnees (composantes fusionnées ou contrats, section 7.4), les rôles, les champs du plan et ses deux paramètres. Le contenu de fond vient de la spécification d'actuary (issue #11, parties A.0 et A.3) et de son erratum du 17/10/2026. Les fenêtres et les événements, les segments des observations et la position d'un jour dans sa cellule sont calculés par trois fonctions (fenetres_observations(), segments_observations(), position_dans_cellule()) que l'estimateur kaplan_meier du module brut emploie aussi (section 9.4) : les deux modules lisent ainsi les mêmes fenêtres, sans seconde implémentation. Dans une construction, les fenêtres et les segments ne sont calculés qu'une fois : l'étape exposition les rend à côté de son résultat, et construire_table() les transmet à l'étape brut, si le plan retient l'estimateur kaplan_meier, sans les inscrire dans l'objet résultat (section 8.2, « Fenêtres transmises à l'étape brut » ; issue #38). Leur extraction de calculer_exposition() n'a changé aucun résultat (constat : objets identical() hors execution avant et après l'extraction, sur les cas de test et sur des plans sans kaplan_meier).

8.1 Recette d'exposition et conventions d'âge

Recette déduite des rôles

La *recette d'exposition* n'est pas un paramètre du plan : le moteur retient la plus fine que les rôles permettent, la nomme dans les paramètres effectifs et dans la fiche, et **refuse si aucune ne s'applique**, sans jamais deviner (E2.3 ; ADR 0003, point 3 ; ADR 0007, point 5.3). Une seule recette existe dans cette branche (tableau 14) ; les recettes « agrégats » et « stocks successifs » (E2.1) viendront avec les branches qui les créent. deduire_recette() retient la recette dont tous les rôles requis sont déclarés, ou refuse (aucune_recette, le message nommant la recette et les rôles absents) ; dans construire_table(), le même manque est détecté plus tôt par le contrôle des rôles, avec le même code et le module donnees. construire_table() appelle deduire_recette() une fois, avant toute étape ; la fonction d'étape, appelée avec controler = FALSE (section 6.2), ne la rappelle pas et lit la recette de la branche, sa convention native et ses rôles requis par recette_branche(), qui ne contrôle rien ; appelée directement, elle appelle deduire_recette(). Cette lecture sans contrôle vaut tant que la branche n'a qu'une recette.

Table 14 : Recettes d'exposition (constantes RECETTE_BRANCHE et CONVENTION_NATIVE).

Recette	Rôles requis	Convention native
lignes_dates_completes	date_naissance, date_entree, date_sortie, cause_sortie (lignes individuelles, dates complètes)	âge au dernier anniversaire (dernier_anniversaire)

Conventions d'âge

Les conventions d'âge sont ordonnées de la plus fine à la plus grossière (ORDRE_CONVENTIONS) : âge au dernier anniversaire (dernier_anniversaire), âge atteint dans l'année (age_atteint_annee), âge fourni (age_fourni). Le paramètre convention de l'étape vaut plus_fine, qui retient la convention native de la recette, ou désigne une convention :

— une convention **plus grossière** que la native est un *forçage*, admis : il est inscrit dans les paramètres effectifs et dans la fiche (convention demandée, native, retenue, forcée ou non) et donne l'avertissement convention_forcee (E2.3) ;

— une convention **plus fine** que la native est refusée (convention_plus_fine) : les rôles ne la permettent pas ;

— age_fourni, non mise en œuvre (ses rôles ne sont pas pris en charge), est refusée (convention_non_mise_en_oeuvre) ; toute autre valeur aussi, de même qu'une convention qui n'est pas une chaîne unique sans attribut ni NA (convention_invalide) ; par_annee doit valoir TRUE ou FALSE, valeur logical unique sans attribut (par_annee_invalide). Ces deux contrôles appliquent la règle commune de forme du plan (section 6.1 ; issue #23).

Aux refus des paramètres s'ajoute un refus de capacité, trop_de_cellules, levé à la construction de la table quand elle compterait plus de .Machine$integer.max cellules (issue #37), ou, plus tôt, quand le décompte de présence des âges compterait plus de .Machine$integer.max couples (segment, âge) (issue #45) (section 8.2, « Paramètres effectifs, diagnostics, avertissements »).
Chaque convention définit ses cellules d'âge et l'unité de l'exposition (UNITES_EXPOSITION, inscrite dans la fiche) :

— dernier_anniversaire : la cellule d'âge x d'une tête est $[\text{anniv}(x), \text{anniv}(x+1))$, l'âge étant celui de la section 7.1 (29 février compris) ; l'unité est l'**année d'âge** : un morceau de j jours dans la cellule x vaut j/L_x, avec $L_x = \text{anniv}(x+1) - \text{anniv}(x)$, soit 365 ou 366 jours ;

— age_atteint_annee : l'âge au jour t est $\text{an}(t) - \text{an}(n)$, la cellule est l'année civile $[1^{\text{er}}$ janvier, 1er janvier suivant), et l'unité la fraction d'année civile ($j/365$ ou $j/366$).

Pourquoi l'année d'âge. Avec cette unité, une année d'âge entière vaut exactement 1 ; c'est la seule pour laquelle l'hypothèse « force constante par âge entier » donne exactement $q_x = 1 - e^{-\mu_x}$ pour toute année d'âge (résultat exact). Avec une division par 365,25, une année d'âge pèserait $365/365{,}25$ ou $366/365{,}25$ selon les années, et la correspondance ne serait plus exacte. L'unité est un *choix de conception* de la spécification (A.0, T3), validé par actuary le 17/10/2026 (validation de fin de branche).

Statut des valeurs. L'exposition est produite en jours, entiers exacts stockés en double : l'exposition totale en jours d'un portefeuille de 10^6 lignes sur 30 ans (environ $1{,}1 \cdot 10^{10}$) dépasse .Machine$integer.max ; résultat exact. L'exposition en unité d'exposition est une somme de quotients en virgule flottante : approximation numérique, vérifiée à 10^{-12} près en relatif. L'arithmétique des dates du module (années bissextiles, anniversaires, 1er janvier) est entière et vectorisée, écrite indépendamment de celle du simulateur ; elle est recoupée par as.Date() et comparée à age_revolu() du module donnees, écrite indépendamment, dans les tests. Trois implémentations de la convention du 29 février (section 7.1) sont **volontairement** conservées (décision M89, issue #22, point 3) : celle du module

donnees (composantes_date(), enveloppe de as.POSIXlt, sur laquelle repose age_revolu()), celle-ci, et les jours civils du simulateur (jours_civils(), jour_anniversaire(), age_anniversaire(), section 16), qu'aucun module du moteur n'appelle (ADR 0007, point 2.1). Le motif de ne pas les unifier, qui relève de la présente rédaction et non de la décision M89, est double : elles se contrôlent l'une l'autre par comparaison directe pour la décomposition d'un jour, les jours civils et les âges, et par l'intermédiaire de as.Date(), oracle commun, pour les anniversaires et les bornes du domaine ; et le simulateur, qui produit les jeux sur lesquels le moteur est vérifié, ne partage pas avec lui une erreur de calendrier que la comparaison du moteur à la loi vraie des jeux simulés ne révélerait pas. Leur accord sur le domaine [0001-01-01, 9999-12-31] est vérifié, avec as.Date() pour oracle des anniversaires et des bornes (section 8.4) : exhaustivement pour la décomposition d'un jour en année, mois et jour et pour les jours civils, partiellement pour les anniversaires et les âges.

8.2 Troncature à gauche et censure à droite

Fenêtre d'observation et décompte

Pour une observation d'entrée e et de fin s ($s = +\infty$ si elle est en cours), la période effective étant $[D, B)$, avec D le début de la période et B la borne de la section 7.1 :

— la **fenêtre** est $[a, b)$, avec $a = \max(e, D)$ et $b = \min(s, B)$, vide si $a \geq b$: une entrée antérieure à D est tronquée à gauche à D, et l'observation est censurée à droite à B (fin de période et délai de déclaration) ;

— l'**exposition centrale** de la cellule (g, x) est

$$E^{c}_{g,x} = \sum_{i \in g} \big| [a_i, b_i) \cap [\mathrm{anniv}_i(x), \mathrm{anniv}_i(x+1)) \big|,$$

en jours, et en unité d'exposition par division de chaque morceau par la longueur de sa cellule ;

— l'**événement** de la transition étudiée est compté si et seulement si la cause est la cible et $D \leq s < B$; il est attribué à la cellule du jour s. Un décès le jour d'un anniversaire est donc attribué à l'âge $x + 1$, où l'exposition de la tête est nulle (cas H8 des tests), et un événement le jour de l'entrée est compté avec une exposition nulle (cas H11) ; le relevé evenement_sans_exposition compte les cellules à exposition nulle qui comptent un événement.

Sorties concurrentes

Une sortie dans la fenêtre dont la cause n'est ni la cible ni en_cours est une *sortie concurrente* : rachat ou décès pour l'autre transition, terme, autre cause, composante de tête close sans décès. Elle est traitée comme une **censure à droite à son instant**, et comptée dans la colonne sorties_concurrentes de sa cellule, censures programmées comprises. Ce traitement suppose une **censure non informative** (risques concurrents indépendants) : l'hypothèse est nommée dans les paramètres effectifs et dans la fiche (HYPOTHESE_CENSURE ; E2.6).

Compléments de l'exposition initiale

L'exposition initiale n'est pas choisie dans cette branche : pour chaque événement compté au jour s dans une cellule qui se termine en $\mathrm{fin}(x)$, deux compléments, en unité d'exposition, sont restitués :

— complement_initial_cellule : $c_1 = \mathrm{fin}(x) - s$, jusqu'à la fin de la cellule (anniversaire suivant sous dernier_anniversaire, 1er janvier suivant sous age_atteint_annee), convention actuarielle classique ;

— `complement_initial_fenetre` : $c_2 = \min(\text{fin}(x), B) - s$, jusqu'à la fin de la fenêtre, la censure par la période étant connue d'avance.

D'où deux expositions initiales, $E^{i,\text{cellule}} = E^c + \sum c_1$ et $E^{i,\text{fenêtre}} = E^c + \sum c_2$. Aucune n'est sans biais indépendamment de l'hypothèse de répartition dans l'année. *Résultat exact* : pour une cohorte entrée à l'âge exact x, sans sortie concurrente, censurée par la période à un instant $c \sim \mathcal{U}(0,1)$ indépendant de la durée de vie, la limite presque sûre de $D/E^{i,\text{fenêtre}}$ quand le nombre de têtes croît (rapport des espérances) vaut $2\big(1 - (1 - e^{-\mu})/\mu\big)$ sous force constante, soit environ $0{,}05043$ pour $q = 0{,}05$, et exactement q sous l'hypothèse de répartition uniforme ; dans le même schéma, la limite de $D/E^{i,\text{cellule}}$ vaut exactement q sous l'hypothèse ${}_{1-t}q_{x+t} = (1-t)\,q_x$ (calcul direct). À effectif fini, le rapport a un biais que ce résultat ne couvre pas. Le choix de l'exposition initiale revient donc à l'estimateur binomial, qui doit nommer son hypothèse de répartition : le module `brut` retient $E^{i,\text{cellule}}$, sans paramètre, et la colonne `complement_initial_fenetre` reste un relevé de l'étape `exposition`, que le binomial n'emploie pas (preuve et hypothèse : section 9.3). Dans les deux définitions, une sortie concurrente reste à son instant réel : elle est traitée comme une censure, et la portée de l'estimateur binomial sous censure est décrite avec lui.

Table d'exposition

`resultat$exposition` a une ligne par cellule : clés de segmentation, `age` et, avec `par_annee`, `annee` ; colonnes `exposition_jours`, `exposition_centrale` (unité nommée dans `parametres$unite_exposition`), `complement_initial_cellule` et `complement_initial_fenetre` (sans `par_annee` seulement), `evenements` et `sorties_concurrentes`. Par segment, la table couvre la plage **contiguë** des âges dont les cellules ont une exposition non nulle, un événement ou une sortie concurrente ; les zéros sont explicites. Inclure les sorties concurrentes dans la plage garde exacte l'identité « somme de `sorties_concurrentes` = nombre de sorties concurrentes dans la fenêtre » ; la plage de validité de la table finale se fixera à la fermeture (erratum A.3, point 1).

Clés de segmentation. Leur ordre et leur type ne dépendent ni des niveaux d'un facteur ni de la classe reçue : `sexe`, et `segment` reçu en chaîne ou en facteur, sont rendus en chaînes et triés en ordre d'octets ; un `segment` entier est rendu en entier et trié par valeur. L'empreinte des données ignore les niveaux d'un facteur (section 15.2) : sans cette règle, deux entrées de même empreinte pourraient donner des tables ordonnées différemment (erratum A.3, point 4).

Paramètres effectifs, diagnostics, avertissements. Les paramètres effectifs sont la recette, la convention (demandée, native, retenue, forcée), l'unité d'exposition, `par_annee`, le code de la transition, la segmentation, la période, le délai, la fin effective et l'hypothèse de censure. Les diagnostics donnent le nombre d'observations et d'observations exposées, l'exposition totale en jours, le nombre d'événements et de sorties concurrentes, le nombre de cellules, stocké en `integer` (issue #37), et le nombre de cellules à exposition nulle qui comptent un événement. Le nombre de cellules est calculé par `nombre_cellules()` : somme, sur les segments, du nombre d'âges de la plage par le nombre d'années (une seule sans `par_annee`), formée en `double` ; s'il dépasse `.Machine$integer.max` ($2^{31} - 1$), l'étape **refuse** sous le motif `trop_de_cellules` (refus de capacité : segmentation trop fine ou plage d'âges et d'années trop étendue), avant toute allocation par cellule, au lieu de laisser `tabulate()` échouer sur une erreur R. Le même motif est levé plus tôt, par la même fonction, avant le décompte de présence des âges (issue #45) : ce décompte, fait par `tabulate()`, porte sur une matrice qui croise les segments et la plage globale des âges présents (du plus petit au plus grand âge présent, tous segments confondus), soit le produit du nombre de segments par cette étendue, en couples (segment, âge) ; au-delà de `.Machine$integer.max`, l'étape **refuse**, avec un message qui compte des couples (segment, âge) et non des cellules, et un décompte inscrit dans `details` sous le nom `couples_presence` au lieu de `cellules`.

Ce produit n'est pas un nombre de cellules mais en est un majorant quand il n'y a pas de découpage par année : sans par_annee, c'est donc la garde de présence qui refuse, et le refus sur le nombre de cellules n'est atteignable qu'avec par_annee = TRUE. Ce majorant peut être lâche : la garde de présence peut refuser alors que la table elle-même tiendrait sous la limite, par exemple quand les plages d'âges des segments sont disjointes, la plage globale couvrant alors, pour chaque segment, les âges de tous les autres. Une base de cette taille étant hors de portée d'un test, la garde est vérifiée par appel direct de nombre_cellules() sous ses deux libellés, et son placement avant le tabulate() de présence par une copie instrumentée de calculer_exposition() (tests/unitaires/test_exposition.R). Avertissements : convention_forcee, evenement_sans_exposition et aucune_exposition (aucune exposition ni aucun événement dans la période).

Fenêtres transmises à l'étape brut (issue #38 ; ADR 0007, seconde annotation du 19/10/2026). Outre ses paramètres effectifs, son résultat, ses diagnostics et ses avertissements, calculer_exposition() rend un cinquième élément, fenetres, objet de classe fenetres_exposition. Il contient les fenêtres des observations et leurs événements (composantes, calendrier, a, b, evenement, issus de fenetres_observations()), le segment de chaque observation et la table des clés de segmentation (g, segments, issus de segments_observations()), les paramètres dont ils dépendent : début D (debut) et borne B (borne) de la période, code de la transition (transition) et segmentation (segmentation), et la convention retenue par l'étape (convention). La convention n'entre pas dans le calcul des fenêtres ni des segments (fenetres_observations() et segments_observations() ne la prennent pas) : elle est jointe pour que l'étape brut vérifie qu'elle emploie la même convention que l'étape exposition qui a produit la table d'exposition. Ce sont les valeurs mêmes qui ont servi à construire la table d'exposition, non recalculées. Cet élément est **interne à l'exécution** : construire_table() ne le conserve que si le plan retient l'estimateur kaplan_meier ; elle le passe alors à l'étape brut, puis le libère ; il n'entre ni dans etapes de l'objet résultat ni dans la fiche (section 6.2). L'estimateur kaplan_meier le reprend après un contrôle d'égalité exacte de ces cinq paramètres (section 9.4, « Données »). Ainsi, les deux étapes lisent les fenêtres d'une seule source, calculées une seule fois. Les positions des jours dans leur cellule ne sont pas transmises : seul kaplan_meier les emploie et il les calcule lui-même. Les calculer ici coûterait à toute construction, y compris sans cet estimateur, et ferait dépendre le module exposition des besoins d'un estimateur du module brut.

8.3 Découpage par année

Avec par_annee = TRUE, chaque morceau d'exposition est en outre découpé au 1er janvier, et un événement est attribué à l'année civile de son jour. La grille est **rectangulaire** : elle croise la plage d'âges du segment et toutes les années civiles de l'année du début de la période à celle de la fin effective, avec des zéros explicites ; la première et la dernière année peuvent être partielles (erratum A.3, point 2). La somme sur les années redonne la table par âge seul, exactement en jours. Les compléments de l'exposition initiale ne sont pas produits dans ce découpage.

8.4 Vérifications

Les cas construits à la main de la spécification (H1 à H13 pour une ligne, F1 à F4 pour la fusion) sont vérifiés par tests/unitaires/test_exposition.R, en jours exactement et en unité d'exposition à 10^{-12} près. Quelques-uns illustrent les règles ci-dessus (période du 2015-01-01 au 2024-12-31, délai nul, vivant $\rightarrow$ décédé) :

Table 15 : Exemples de cas calculés à la main (spécification, A.4 ; valeurs reproduites par le moteur sur R 4.3.3).

Cas	Ligne	Exposition par âge (jours / L_x)	Événement
H2	née 1945-10-15 ; 2016-05-20 → 2019-02-10, décès	70 : 148/366 ; 71 : 365/365 ; 72 : 365/365 ; 73 : 118/365 ; total 996 jours	à 73 ans ; $c_1 = c_2 = 247/365$
H6	née 1964-02-29 ; 2021-01-01 → en cours	56 : 59/366 ; 57 à 59 : 1 chacun ; 60 : 307/366 ; total 1 461 jours, 4 années d'âge	aucun
H8	née 1955-04-12 ; 2019-09-01 → 2020-04-12, décès	64 : 224/366 ; 65 : 0	à 65 ans, exposition nulle à 65 ; $c_1 = 365/365$
H11	née 1980-01-01 ; 2018-06-15 → 2018-06-15, décès	aucune	à 38 ans ; $c_1 = 200/365$
H12	H2 avec `age_atteint_annee`	71 : 226/366 ; 72 : 1 ; 73 : 1 ; 74 : 40/365	à 74 ans ; $c_1 = 325/365$

Le cas F4 (décès porté par deux contrats d'une même tête) donne un décès en unité tête et deux en unité contrat, avec l'avertissement `unite_contrat`. Sur un portefeuille simulé, `tests/unitaires/test_construire_table.R` vérifie des *identités exactes* : exposition totale en jours égale à la somme des $\max(0, b - a)$, égalité entière ; décès comptés égaux aux décès simulés tels que $D \leq s \leq$ fin effective ; même identité pour les rachats, décès censurés ; décès par sexe et par âge égaux aux décès simulés par âge au dernier anniversaire ; avec `par_annee`, somme sur les années égale à l'exposition par âge seul, en jours. La segmentation est vérifiée avec des facteurs aux niveaux permutés, en chaîne et en entier.

Accord des trois arithmétiques de dates (décision M89, issue #22, point 3 ; section 8.1, « Statut des valeurs »). `tests/unitaires/test_exposition.R` compare les implémentations des modules `donnees`, `exposition` et du simulateur, sans les modifier, avec `as.Date()` pour oracle externe des anniversaires et des bornes :

— **décomposition**, exhaustive : pour chaque jour du 0001-01-01 au 9999-12-31, `composantes_date()` et `composantes_jour()` donnent la même année, le même mois et le même jour ; les jours voisins des bornes tombent dans les années 0 et 10 000 ;

— **jours civils**, exhaustifs : pour chaque jour du domaine, `jours_civils()` appliquée aux composantes rend le jour de départ, `annee_du_jour()` et `annee_calendrier()` rendent son année, et `premier_janvier()` égale `jours_civils()` au 1er janvier de chacune des années 1 à 10 001 ;

— **anniversaires**, partiels : `anniversaire_age()` et `jour_anniversaire()` égalent l'oracle (le 29 février d'une année commune remplacé par le 1er mars, convention T2) pour toutes les naissances du domaine aux âges 0, 1, 3, 4, 100 et 130, et pour les naissances des 28 février, 29 février et 1er mars de chaque année bissextile à tous les âges de 0 à 130, tant que l'anniversaire reste dans le domaine ;

— **âges au dernier anniversaire**, partiels : les trois fonctions d'âge, `age_revolu()`, `age_anniversaire()` et `age_dernier_anniversaire()`, concordent sur un tirage de couples (naissance, jour) sous graine locale (`sous_graine()`), naissances du 29 février comprises, et sur un balayage des 28 février, 29 février et 1er mars postérieurs aux naissances du 29 février des années 4, 1964 et 2000 ;

— **témoin écrit à la main** : pour une naissance le 29 février de l'an 4, anniversaires et âges aux années séculaires communes (le 1er mars) et bissextiles (le 29 février), les anniversaires par les deux fonctions qui les calculent (`anniversaire_age()` et `jour_anniversaire()`), les âges par les trois fonctions d'âge.

L'accord exhaustif couvre tous les jours du domaine (vérification par énumération, sur la plateforme où la batterie s'exécute) ; l'accord partiel n'est constaté que sur les cas énumérés.

8.5 Fonctions du module

Table 16 : Définitions de premier niveau de R/exposition.R.

Nom	Statut	Entrées	Sortie et rôle
ORDRE_CONVENTIONS	constante	—	conventions d'âge, de la plus fine à la plus grossière
CONVENTIONS_MISES_EN_OEUVRE	constante	—	dernier_anniversaire, age_atteint_annee
VALEURS_CONVENTION	constante	—	valeurs admises du paramètre convention
CONVENTION_NATIVE	constante	—	convention native de chaque recette
UNITES_EXPOSITION	constante	—	unité de l'exposition par convention
HYPOTHESE_CENSURE	constante	—	hypothèse de censure non informative, inscrite dans la fiche
recette_branche()	interne	—	recette de la branche, convention native, rôles requis, sans contrôle des rôles ; employée par deduire_recette() et par calculer_exposition() appelée avec controler = FALSE (issue #22)
deduire_recette()	interne	rôles	recette, convention native, rôles requis (recette_branche()) ; refus aucune_recette
controler_parametres_exposition()	interne	convention, par_annee	refus des valeurs
convention_retenue()	interne	convention demandée, native	convention demandée, native, retenue (la native pour plus_fine, la demandée sinon) et forçage, sans contrôle ; employée par resoudre_convention() et par calculer_exposition() appelée avec controler = FALSE (issue #22)
resoudre_convention()	interne	convention demandée, native	convention retenue et forçage (convention_retenue()) ; refus convention_plus_fine
an_bissextile()	interne	années	bissextilité grégorienne
bissextiles_anterieures()	interne	années	nombre d'années bissextiles antérieures
premier_janvier()	interne	années	jour du 1er janvier
JOURS_AVANT_MOIS	constante	—	jours écoulés avant chaque mois, année commune
annee_du_jour()	interne	jours	année civile
composantes_jour()	interne	jours	année, mois, jour, quantième commun
calendrier()	interne	années extrêmes	table calendaire
indice_annee()	interne	table, années	indice dans la table calendaire
annee_calendrier()	interne	table, jours	année civile, par la table
anniversaire_age()	interne	table, naissances, âges	jour de l'anniversaire
age_dernier_anniversaire()	interne	table, naissances, jours	âge au dernier anniversaire

Nom	Statut	Entrées	Sortie et rôle
debut_cellule()	interne	table, naissances, âges, convention	début de la cellule d'âge
cellule_du_jour()	interne	table, naissances, jours, convention	cellule d'âge du jour
extraire_composantes()	interne	composantes, indices	sous-ensemble
decouper_cellules()	interne	fenêtres, convention	morceaux contenus chacun dans une cellule
decouper_annees()	interne	morceaux	morceaux découpés au 1er janvier
somme_par_cellule()	interne	cellules, valeurs	sommes par cellule, zéros explicites
nombre_cellules()	interne	nombres d'âges par segment, nombre d'années ; libellé et nom du décompte, facultatifs	nombre de cellules de la table, en integer ; refus trop_de_cellules au-delà de .Machine$integer.max (issue #37) ; appelée aussi, avant le décompte de présence des âges, sur le produit du nombre de segments par l'étendue des âges, avec son propre libellé et le décompte couples_presence (issue #45)
fenetres_observations()	interne	observations, début de la période, borne B, cause cible	fenêtres $[a, b)$, événements et sorties concurrentes (section 8.2) ; composantes des naissances, table calendaire
segments_observations()	interne	observations, segmentation	segment de chaque observation et table des clés, triées sans dépendre des niveaux d'un facteur
position_dans_cellule()	interne	table calendaire, composantes, jours, convention	cellule x du jour, rang j dans la cellule, longueur L de la cellule, début de la suivante
calculer_exposition()	interne	observations, rôles, transition, période, délai, segmentation, convention, par_annee, controler (TRUE par défaut)	fonction d'étape exposition : paramètres effectifs, résultat, diagnostics, avertissements, et fenêtres transmises à l'étape brut (objet fenetres_exposition, hors de l'objet résultat ; issue #38) ; avec controler = FALSE, réservé à construire_table(), ni les paramètres, ni la recette, ni la convention ne sont recontrôlés (issue #22)

Partie V

Module brut

Taux bruts : R/brut.R

Le module brut estime, par cellule de la table d'exposition (segment, âge et, sur demande, année civile), le taux de la transition étudiée et ses intervalles de confiance exacts et asymptotiques, chacun restitué avec son statut (E2.12, E2.14, E2.20). Sa fonction d'étape est calculer_taux_bruts() ; elle reçoit la table d'exposition et les paramètres effectifs de l'étape exposition, jamais le plan. Le module n'appelle que des modules situés avant lui (donnees, exposition, outils ; section 1.3) et ne tire aucun aléa. Le contenu de fond vient de la spécification d'actuary du 17/10/2026 (issue #14, parties A.1 à A.7) et des décisions du mainteneur sur ses questions A.8 (décision M51) ; pour l'estimateur kaplan_meier, de la spécification d'actuary du 17/10/2026 (issue #15, parties A.1 à A.12) et des décisions M52 (b) et M53.

Appellations et références. Les sources primaires des estimateurs et des intervalles de la présente partie ne sont pas encore établies (issue #6 ; décision M50, Q4). Chaque méthode est donc décrite par sa formule, sa dérivation et son statut, **sans attribution d'auteur** : référence à établir, issue #6. Les textes inscrits par le code dans la fiche emploient, pour les intervalles asymptotiques, l'appellation d'usage « Wald », dont la référence est elle aussi à établir. L'estimateur produit-limite porte dans le plan le nom kaplan_meier, celui que lui donne le cahier des charges (E2.12) : la source de cette appellation, celle de sa formule de variance et celle de ses propriétés asymptotiques sous troncature à gauche sont à établir (issue #6).

9.1 Étape brut du plan

Notations, par cellule de la table etapes$exposition$resultat$exposition (section 8.2) : D le nombre d'événements (evenements) ; E^c l'exposition centrale (exposition_centrale, dans l'unité de la fiche : année d'âge, ou fraction d'année civile sous age_atteint_annee) ; $E^i = E^c + \sum c_1$ l'exposition initiale « cellule », où c_1 est le complément complement_initial_cellule de chaque événement (section 8.2) ; $\alpha = 1 -$ niveau et $z = \Phi^{-1}(1 - \alpha/2)$. Les intervalles sont à queues égales, $\alpha/2$ de chaque côté, quel que soit D.

Paramètres, tous requis, sans défaut (ADR 0007, point 5 ; constante PARAMETRES_ETAPES), soumis à la règle commune de forme (type de base exact, aucun attribut ; section 6.1, issue #23) : estimateurs, vecteur character, sous-ensemble non vide, sans doublon ni NA, de ESTIMATEURS_BRUT ; les résultats sont toujours rendus dans l'ordre de cette constante, quel que soit l'ordre déclaré ; niveau, nombre double unique, fini, strictement compris entre 0 et 1, partagé par tous les estimateurs. L'étape a pour préalables les étapes donnees et exposition (constante ETAPES ; refus etape_prealable_manquante de valider_plan() sinon) : l'estimateur kaplan_meier lit les fenêtres des observations de la première,

calculées par la seconde et transmises par construire_table() (section 6.2), tous les estimateurs la table de la seconde.

Refus du module brut, levés par controler_parametres_brut(), que valider_plan() appelle avant toute étape et que la fonction d'étape appelle en appel direct seulement (controler = TRUE, valeur par défaut ; construire_table() l'appelle avec controler = FALSE, section 6.2) : estimateurs_invalides (pas un vecteur character, attribut — noms, dimension ou classe —, vide, NA ou doublon) ; estimateur_inconnu, contrôlé après la forme : un vecteur nommé de noms inconnus relève du motif précédent ; niveau_invalide (autre type que double, attribut, longueur différente de 1, valeur non finie, ≤ 0 ou ≥ 1) ; binomial_par_annee : sous par_annee = TRUE, l'exposition initiale n'est pas produite (section 8.3), et une cellule âge $\times$ année ne couvre pas une année d'âge entière, si bien que q_x n'y est pas défini ; kaplan_meier_par_annee, pour la même raison de couverture (un estimateur de période sur une cellule âge $\times$ année relèverait d'une autre spécification). Si le binomial et kaplan_meier sont tous deux déclarés sous par_annee, binomial_par_annee est levé en premier. En appel direct, la fonction d'étape contrôle en outre, en tête et avant controler_parametres_brut(), la forme et le catalogue de la segmentation qu'elle reçoit, par controler_segmentation() du module donnees : segmentation_invalide, module brut (autre chose qu'un vecteur character sans attribut, sans NA ni doublon, dont chaque clé est prise dans ROLES_SEGMENTATION, soit sexe et segment ; vide explicite admis ; issues #43 et #59). Une clé hors de ce catalogue est donc refusée à l'appel direct, qu'elle désigne une colonne de la table d'exposition ("age", "evenements") ou non ("region"), de même que la chaîne vide, qui n'en fait pas partie : sans ce contrôle, une clé hors du catalogue présente dans la table serait employée comme clé de segmentation sans avoir été déclarée comme rôle ("age" dupliquerait la clé d'âge dans la table résultat), et une clé absente de la table ne serait signalée que par une erreur de programmation sur les colonnes manquantes. Sur le chemin Kaplan-Meier, la fonction d'étape contrôle ensuite, en appel direct, la transition, la période et le délai par controler_transition_periode() du module donnees (refus de ce module, section 7.3), sans recontrôler la segmentation ni contrôler d'exclusions ; l'estimateur taux_kaplan_meier() ne contrôle rien lui-même (section 9.4). Une segmentation invalide y est donc refusée sous le module brut, une transition inconnue sous le module donnees. Le même code est porté par le module qui détecte la condition (ADR 0007, annotation du 17/10/2026, point d) : dans le plan, la segmentation est déjà refusée sous ce code par controler_parametres_donnees(), module donnees, que valider_plan() appelle et qui emploie le même contrôle (section 7.3) ; la forme et le catalogue de la segmentation ne sont écrits qu'à un endroit, et le message du refus est le même dans les deux modules. Ce contrôle-ci ne sert qu'à l'appel direct de la fonction d'étape, et une segmentation invalide y est signalée avant un estimateur inconnu.

Quantiles supérieurs. Ils sont calculés par la queue supérieure (qgamma(alpha / 2, D + 1, lower.tail = FALSE), de même pour qbeta() et qnorm()) plutôt que par le quantile $1 - \alpha/2$: pour un niveau très proche de 1 (par exemple $1 - 2^{-53}$), $1 - \alpha/2$ vaut exactement 1 en double et le quantile serait infini, alors que la queue supérieure garde une borne finie (spécification, A.3 ; cas couvert par les tests). Aux niveaux usuels, les deux écritures ne diffèrent que par l'arrondi (approximation numérique).

9.2 Estimateur de Poisson

— **Objet** : l'intensité μ de la transition dans la cellule, sur l'axe de l'âge (et de l'année sous par_annee) ; estimateur poisson.

— **Hypothèses** : force de la transition constante dans la cellule ; sorties concurrentes non informatives (HYPOTHESE_CENSURE, section 8.2) ; troncature à gauche traitée par la fenêtre d'observation.

— **Estimateur** : sous ces hypothèses, la log-vraisemblance de μ vaut exactement $D \log \mu - \mu E^c$; son

maximum est

$$\hat{\mu} = D/E^c.$$

— **Probabilité associée** : $\hat{q}^P = 1 - e^{-\hat{\mu}}$, calculée par `-expm1(-taux)`, sous l'hypothèse « force constante sur la cellule entière, égale au taux de la cellule » ; les bornes sont transformées par la même fonction croissante, si bien que la couverture est conservée exactement. Sa portée dépend de la cellule (`PORTEE_PROBABILITE`) : sous `dernier_anniversaire`, l'unité étant l'année d'âge, c'est le q_x exact sous force constante par âge entier (section 8.1) ; sous `age_atteint_annee`, c'est la probabilité sur l'année civile de la cellule ; sous `par_annee`, c'est le q_x de l'année t à force $\mu_{x,t}$.

— **Biais dû à la datation au jour** (spécification de #16, A.1 ; résultat exact, preuve écrite). L'exposition court jusqu'au jour de la sortie exclu (section 7.1) : il manque en moyenne un demi-jour d'exposition par sortie, quelle qu'en soit la cause. Dans une cellule de L jours où la transition a la force constante μ_d et les sorties concurrentes la force μ_r, avec $m = \mu_d + \mu_r$, une tête à risque au début d'un jour subit la transition ce jour-là avec la probabilité $h_d = (\mu_d/m)(1 - e^{-m/L})$, constante dans la cellule ; il en résulte $E[D - \mu^* E^c] = 0$ exactement, avec

$$\mu^* = \frac{\mu_d}{m} L \left(e^{m/L} - 1\right), \qquad \frac{\mu^*}{\mu_d} - 1 \approx \frac{m}{2L}.$$

$\hat{\mu}$ estime donc μ^*, et non μ_d : l'écart relatif, de l'ordre de 10^{-5} à $5 \cdot 10^{-4}$ pour les forces des constats de simulation (section 9.9), est négligeable devant l'erreur d'estimation, mais non nul. Il vaut pour des données réelles datées au jour comme pour le simulateur. Le binomial sur une cellule observée entière ($E^i = n$) et l'estimateur produit-limite sans sortie concurrente n'en sont pas affectés : leur cible exacte est q_x. Avec une sortie concurrente, la cible de l'estimateur produit-limite sur la grille des jours, $q^* = 1 - (1 - h_d)^L$, s'écarte de $1 - e^{-\mu_d}$ d'un écart relatif du même ordre (constats, section 9.9).

— **Intervalle exact** : $\left[G_D^{-1}(\alpha/2),\ G_{D+1}^{-1}(1-\alpha/2)\right]/E^c$, où G_k est la loi gamma de forme k et d'échelle 1 ; borne inférieure 0 si $D = 0$, borne supérieure alors $-\ln(\alpha/2)/E^c$.

— **Intervalle asymptotique** : normal sur l'échelle de $\log \mu$, $\mathrm{Var}(\log \hat{\mu}) \approx 1/D$, soit $\hat{\mu}\, e^{\mp z/\sqrt{D}}$; non défini à $D = 0$.

— **Calcul** : `taux_poisson()`, bornes exactes par `qgamma()`.

— **Statut** : $\hat{\mu}$, résultat exact (maximum de vraisemblance du modèle), valeur en arithmétique flottante ; $\hat{q}^P$, résultat exact (image du maximum de vraisemblance par $q = 1 - e^{-\mu}$) ; intervalle exact, exact sous la loi supposée $D \sim \mathcal{P}(\mu E^c)$, E^c tenue pour fixe (E2.14) : couverture au moins égale au niveau pour tout μ ; sous le modèle de survie, où E^c est aléatoire, sa couverture n'est qu'asymptotique ; bornes par approximation numérique (`qgamma`) ; intervalle asymptotique, asymptotique ($\mu E^c \to \infty$).

— **Références** : à établir (issue #6).

9.3 Estimateur binomial

— **Objet** : la probabilité q de la transition dans la cellule, sur l'axe de l'âge ; estimateur `binomial`, refusé sous `par_annee`.

— **Hypothèse de répartition dans la cellule** : ${}_{1-t}q_{x+t} = (1-t)\, q_x$ pour $0 \le t \le 1$, ce qui revient à interpoler linéairement $1/S$ dans la cellule ; censures (sorties concurrentes, fin de période, délai de déclaration) indépendantes de l'instant de l'événement.

— **Estimateur** : $\hat{q}^B = D/E^i$, avec l'exposition initiale « cellule » (`EXPOSITION_INITIALE_BRUT`), fixée sans paramètre (décision M51, question 1).

— **Pourquoi la définition « cellule »** (spécification, A.2 ; preuve écrite). Soit une tête présente dans la cellule à la fraction u et censurée en c ($u < c \leq 1$, c fixe ou indépendant de l'instant T de l'événement), et $\pi = P(T \in (u, c) \mid T > u)$. *Définition « fenêtre »* : le complément court jusqu'à c, donc $E^{i,\text{fenêtre}} = c - u$ quelle que soit la loi ; l'égalité $E[D] = q\,E[E^i]$ exige $\pi = q(c-u)$ pour tout (u, c) ; avec $u = 0$, cela impose la répartition uniforme $P(T \leq c) = qc$, sous laquelle $\pi = q(c-u)/(1-qu) \neq q(c-u)$ dès que $u > 0$: **aucune hypothèse de répartition ne rend cette définition exacte dès qu'une tête entre en cours de cellule**. *Définition « cellule »* : le complément court jusqu'à la fin de la cellule, donc $E[E^{i,\text{cellule}}] = (c-u)(1-\pi)+(1-u)\pi$, et l'égalité $\pi = q\,E[E^i]$ équivaut à $\pi = q(c-u)/(1-q(1-c))$, qui est exactement la valeur de π sous l'hypothèse ci-dessus. En sommant sur les têtes, $E[D] = q\,E[E^i]$ **pour toute troncature à gauche et toute censure indépendante dans la cellule** ; avec $u = 0$, cette hypothèse est la seule loi dans la cellule qui rende l'égalité vraie pour tout c. Sous une autre loi, en particulier la force constante du simulateur, l'identité ne tient plus dès qu'une entrée ou une censure tombe dans la cellule ; elle tient pour une cellule observée entière, sans entrée ni censure, où E^i est le nombre de têtes. Exemple exact (spécification de #16, A.1, point 3 ; section 9.9) : pour n têtes entrées à l'âge exact x, sans sortie concurrente, censurées par la période à la fraction c de la cellule, sous la force constante μ, $D \sim \mathcal{B}(n, \pi_c)$ avec $\pi_c = 1 - e^{-\mu c}$, et D/E^i tend vers $\pi_c/\big(c(1-\pi_c)+\pi_c\big)$ pour la définition « cellule » et vers π_c/c pour la définition « fenêtre » ; avec $q = 0{,}182091641$ et $c = 183/365$, ces limites s'écartent de q de $-4{,}13\ \%$ et de $+5{,}01\ \%$.

— **Intervalle exact** : $\big[B^{-1}_{D,\,E^i-D+1}(\alpha/2),\ B^{-1}_{D+1,\,E^i-D}(1-\alpha/2)\big]$, où $B_{a,b}$ est la loi bêta ; borne inférieure 0 si $D = 0$, borne supérieure 1 si $D = E^i$. Si E^i n'est pas entier, aucune loi binomiale ne lui correspond : la formule est prolongée continûment (décision M51, question 4).

— **Intervalle asymptotique** : normal sur l'échelle de logit q, $\operatorname{logit}^{-1}\big(\operatorname{logit}\hat{q} \mp z\sqrt{1/D + 1/(E^i - D)}\big)$; non défini à $D = 0$ et à $D = E^i$.

— **Tolérance sur l'effectif** (`TOLERANCE_EFFECTIF`, 10^{-9}, absolue) : si $|E^i - D| \leq 10^{-9}$, le calcul prend $E^i := D$, la colonne `exposition_initiale` gardant la somme calculée. Raison : E^i est une somme de fractions j/L, $L \in \{365, 366\}$, dont l'arrondi flottant peut placer une valeur rationnellement égale à D juste en dessous, ce qui la ferait déclarer à tort « effectif inférieur » ; le plus petit écart rationnel non nul entre E^i et un entier vaut $1/(365 \times 366) = 1/133\,590 \approx 7{,}5 \cdot 10^{-6}$, si bien que la tolérance ne peut absorber un écart réel (approximation numérique).

— **Calcul** : `taux_binomial()`, bornes exactes par `qbeta()`.

— **Statut** : $\hat{q}^B$, estimateur des moments : $E[D] = q\,E[E^i]$ exactement sous l'hypothèse de répartition, convergent, non sans biais à effectif fini (c'est un rapport) ; intervalle exact, exact sous la loi supposée $D \sim \mathcal{B}(E^i, q)$, E^i tenue pour fixe, **pour une cellule observée entière**, sans entrée ni censure dans la cellule (alors E^i est le nombre de têtes, entier) ; intervalle asymptotique, asymptotique ($E^i q(1-q) \to \infty$) sous la même condition ; bornes par approximation numérique (`qbeta`). **Avec une entrée ou une censure dans la cellule**, D ne suit pas $\mathcal{B}(E^i, q)$, même si E^i est entier : seule l'égalité $E[D] = q\,E[E^i]$ subsiste, et la **couverture des deux intervalles n'est pas établie** ; si E^i n'est pas entier, l'intervalle exact est en outre un prolongement continu de la formule exacte. Exemple (calcul exact de la loi de D, sans simulation : résultat exact sur un cas, bornes par `qbeta`, approximation numérique) : n têtes entrées à la fraction $u = 0{,}5$ de la cellule et observées jusqu'à sa fin, sous l'hypothèse de répartition avec $q = 0{,}2$, si bien que $E^i = n/2$ est entier et $D \sim \mathcal{B}(n, 0{,}1)$; pour un niveau de 0,95, la couverture vraie de l'intervalle exact vaut 0,9561, 0,9423, 0,9381 et 0,9364 pour $n = 200$, $1\,000$, 10^4 et 10^5, celle de l'intervalle logit 0,9245, 0,9352, 0,9357 et 0,9357 : l'intervalle « exact » peut couvrir moins que le niveau. L'écart ne disparaît pas quand l'effectif croît : $\operatorname{Var}(D)$ vaut $n \cdot 0{,}1 \cdot 0{,}9$, au lieu de $E^i q(1-q) = n \cdot 0{,}08$ supposé par les deux intervalles, et les deux couvertures

tendent, quand $n \to \infty$, vers $2\,\Phi\big(\Phi^{-1}(0{,}975)/\sqrt{1{,}125}\big) - 1 = 0{,}9354$, où $1{,}125 = 0{,}09/0{,}08$ est le rapport de ces deux variances (résultat asymptotique).

— **Références** : à établir (issue #6), y compris pour l'appellation usuelle de l'hypothèse de répartition, nommée ici par sa formule.

Choix des échelles asymptotiques (décision M51, question 2). La couverture exacte des intervalles, calculée par sommation sur la loi du décompte (spécification, A.3 ; résultat exact calculé numériquement, sans simulation), est meilleure sur l'échelle du lien canonique, $\log \mu$ pour Poisson et $\operatorname{logit} q$ pour le binomial, que sur l'échelle naturelle dès environ cinq événements attendus ; en deçà, les deux formes asymptotiques sont mauvaises, à cause du poids de $D = 0$, et c'est l'intervalle exact qui fait foi. Ces échelles gardent en outre les bornes dans l'espace des paramètres, sans troncature à 0 ou à 1.

9.4 Estimateur produit-limite (`kaplan_meier`)

— **Objet** : la probabilité q_x de la transition dans la cellule d'âge x, estimée **sans hypothèse de répartition dans la cellule**, à partir des observations individuelles de l'étape `donnees` (composantes fusionnées par tête, ou contrats ; section 7.4) ; estimateur `kaplan_meier`, refusé sous `par_annee`. Sous `age_atteint_annee`, admis (décision M52 (b), question 3), q_x est la probabilité de la transition dans l'année civile $\mathrm{an}(n) + x$ pour une tête présente au 1er janvier, même portée que le binomial sous cette convention.

— **Hypothèses** : troncature à gauche non informative (aucun effet de sélection à l'entrée) ; censure non informative (`HYPOTHESE_CENSURE`) ; unités d'observation indépendantes.

— **Échelle de temps** : celle des cellules de la convention retenue, en années d'âge (section 8.1) : à l'instant t (un jour), $\tau(t) = x + j/L$, avec x la cellule du jour, j le nombre de jours écoulés depuis le début de la cellule et $L \in \{365, 366\}$ sa longueur (`position_dans_cellule()`). Sous `dernier_anniversaire`, un âge entier est l'anniversaire de chaque tête, et un événement est dans la cellule x si et seulement si $x \le \tau(s) < x + 1$: c'est exactement l'attribution de l'étape `exposition`. L'échelle « jours depuis la naissance » est écartée : un âge entier n'y correspond pas au même instant pour toutes les têtes, et rendre q_x commun demanderait de diviser par 365,25, ce que l'unité de l'exposition a écarté. Les instants sont comparés exactement par la clé entière $K = 133\,590\,x + j \cdot 133\,590/L$ (`DENOMINATEUR_INSTANTS` $= 365 \times 366$) : deux instants distincts diffèrent d'au moins $1/133\,590$ an, et aucune tolérance n'intervient.

— **Données** : fenêtres et événements identiques à ceux de l'étape `exposition` (`fenetres_observations()`, section 8.2). Dans une construction, ce sont ceux-là mêmes : l'étape `exposition` les transmet, avec les segments des observations, dans un objet `fenetres_exposition` (« Fenêtres transmises à l'étape `brut` », section 8.2), que `taux_kaplan_meier()` reprend tel quel après avoir contrôlé à l'égalité exacte (`identical()`) les cinq paramètres qu'il porte, comparés à ceux de l'étape `brut` : début et borne de la période, transition et segmentation, dont dépendent les fenêtres et les segments ; convention retenue, qui n'entre pas dans leur calcul et dont le contrôle vérifie la cohérence avec la convention de l'étape `exposition` ; un écart est une erreur de programmation. `taux_kaplan_meier()` ne lève aucun refus : la transition, la période, le délai et la segmentation qu'il reçoit ont été contrôlés par `valider_plan()` dans une construction, par la fonction d'étape en appel direct (section 9.1), et il en reprend les valeurs par `valeurs_parametres_donnees()`, qui ne contrôle rien (issue #22). Les observations elles-mêmes ne sont pas contrôlées : seul `construire_table()` transmet cet objet, produit par `calculer_exposition()` à partir des observations de l'étape `donnees` de la même exécution. En appel direct de la fonction d'étape sur la table des observations, les fenêtres et les segments sont recalculés par les mêmes fonctions (`fenetres_observations()`, `segments_observations()`) ; les deux voies donnent des résultats `identical()` (constat de test,

section 9.8). Les positions des jours dans leur cellule (`position_dans_cellule()`) sont calculées ici, dans les deux cas. Une observation est retenue si sa fenêtre $[a, b)$ est non vide, ou si elle porte un événement ($a \leq s = b$). Sorties concurrentes, fin de période et délai de déclaration sont des censures à leur instant. Une tête dont deux composantes sont disjointes quitte l'ensemble à risque pendant l'écart et y revient ; elle n'y est jamais deux fois.

— **Ensemble à risque fermé** (`ENSEMBLE_A_RISQUE_KM`, `"ferme"` ; décision M52 (b), question 1) : $r(t) = \#\{i : \tau(a_i) \leq t \leq \tau(b_i)\}$, si bien qu'à un même instant l'ordre est **entrées, puis événements, puis censures**. Raisons : un événement du jour s survient à l'instant s et la tête y est présente, ce qui est le seul ordre possible pour une tête qui entre et subit l'événement le même jour ; une tête censurée en c a la même exposition $[a, c)$ qu'une tête dont l'événement survient en c, et appartient donc à l'ensemble à risque de cet instant. Conséquences (identités exactes, vérifiées) : $d_j \leq r_j$; la somme des d_j d'une cellule égale le nombre d'événements de la table d'exposition ; un événement à l'instant d'entrée est compté. La convention « prévisible » $\{a < t \leq b\}$, écartée, exclurait ces derniers événements ; aucune source ne tranche entre les deux (*choix de conception*, mesuré sur un cas construit qui les départage, ci-dessous).

— **Estimateur** : avec t_j les instants d'événement de $[x, x+1)$, d_j le nombre d'événements et r_j l'effectif à risque à t_j,

$$\hat{P}_x = \prod_j \Big(1 - \frac{d_j}{r_j}\Big), \qquad \hat{q}_x = 1 - \hat{P}_x,$$

calculé par `-expm1(sum(log1p(-d / r)))`, plus précis que `1 - prod(1 - d / r)` pour les petits q (spécification, A.3 ; approximation numérique). Si $d_j = r_j$ à un instant, $\hat{q}_x = 1$. $\hat{q}_x$ est $1 - S((x+1)^-)/S(x^-)$, limites à gauche : l'événement du jour de l'anniversaire appartient à $x + 1$.

— **Justification** (spécification, A.3 ; preuve écrite). Avec des risques discrets h_j aux instants d'événement, une observation contribue, conditionnellement à sa présence en a_i, $1 - h_j$ à chaque instant où elle est à risque sans l'événement et h_j à son événement ; la vraisemblance $\prod_j h_j^{d_j}(1-h_j)^{r_j - d_j}$ est maximale en $h_j = d_j/r_j$. Aux instants où personne n'est à risque, elle ne dépend pas du risque : q_x n'y est **pas identifié**.

— **Cellule définie** : la couverture d'une cellule est l'union des intervalles $[\tau(a_i), \tau(b_i))$ non vides ; soit t_0 le premier instant de la cellule où $d = r$, ou $x + 1$ s'il n'y en a pas. La cellule est définie (`defini`) si $[x, t_0)$ est entièrement couvert ; après t_0, la survie est nulle et $\hat{q}_x = 1$ quel que soit le risque, qui n'est pas identifié. Sinon, l'état est `ensemble_a_risque_vide` (couverture incomplète : taux, variance et bornes `NA`) ou, sans aucune couverture ni événement, `exposition_nulle` (section 9.6). Un produit calculé sur la seule partie couverte sous-estimerait q_x (décision M52 (b), question 4).

— **Variance** : $\Sigma_x = \sum_j d_j / \big(r_j(r_j - d_j)\big)$, calculée en `double` (le produit d'entiers $r_j(r_j - d_j)$ déborde) ; $\widehat{\mathrm{Var}}(\hat{q}_x) = \hat{P}_x^2\, \Sigma_x$, par la méthode delta sur $\log \hat{P}_x$; restituée pour $0 < \hat{q}_x < 1$, `NA` à $\hat{q}_x \in \{0, 1\}$.

— **Intervalle asymptotique** : normal sur l'échelle de $\mathrm{logit}\, q$, $\mathrm{Var}(\mathrm{logit}\, \hat{q}_x) \approx \Sigma_x / \hat{q}_x^2$, soit

$$\mathrm{logit}^{-1}\big(\mathrm{logit}\, \hat{q}_x \mp z\sqrt{\Sigma_x}/\hat{q}_x\big)\ ;$$

`NA` à $\hat{q}_x \in \{0, 1\}$.

— **Aucun intervalle exact** (décision M52 (b), question 5) : sous troncature et censure, D ne suit aucune loi binomiale, et la colonne n'existe pas.

— **Identité exacte avec le binomial** (spécification, A.10 ; preuve écrite). Si toute observation dont la fenêtre fermée rencontre $[x, x+1)$ vérifie $a_i \leq x$ et, soit $b_i \geq x + 1$, soit un événement en $b_i \in [x, x+1)$ (ni entrée ni censure dans la cellule, y compris à l'instant x), alors, avec n le nombre de ces observations, $r_{j+1} = r_j - d_j$, d'où $\hat{q}_x^{\mathrm{KM}} = D/n = \hat{q}_x^{B}$ $(E^i = n)$; par télescopage,

$\Sigma_x = 1/(n-D) - 1/n$, d'où $\Sigma_x/\hat{q}_x^2 = 1/D + 1/(n-D)$: les bornes asymptotiques sont celles du binomial.

— **Petit ensemble à risque** : sous troncature, une cellule peut être définie avec un ensemble à risque très réduit ; une seule unité à risque qui subit l'événement donne $\hat{q}_x = 1$. C'est l'instabilité de l'estimateur quand l'ensemble à risque est petit : le diagnostic `cellules_survie_nulle` compte ces cellules, et la colonne `ensemble_a_risque` de la table des sauts donne l'effectif à chaque instant.

— **Pas de survie cumulée par segment** (décision M52 (b), question 2) : le moteur restitue la survie dans la cellule (table des sauts) et q_x, non le produit sur tous les âges d'un segment ; sous troncature, ce produit tombe à zéro au premier âge où l'ensemble à risque s'épuise par un événement, et y reste (mesure de la spécification sur le cas composite C). Le chaînage des q_x en une fonction de survie relève de l'assemblage de la table.

— **Calcul** : `taux_kaplan_meier()` ; $r(t) = \#\{K_a \leq t\} - \#\{K_b < t\}$ par `findInterval()`, couverture par `cummax()`, en $O(n \log n)$. La table d'exposition est appariée **par clé** (segment, âge) : chaque instant d'événement est rattaché, par `match()`, à la ligne dont la cellule commence au même point de l'échelle des clés, sans hypothèse sur l'ordre des lignes ; les instants étant triés et les cellules étant des intervalles disjoints de cette échelle, les instants d'une même cellule forment une plage contiguë, sur laquelle se font les sommes cumulées de $\log(1 - d_j/r_j)$ et de Σ. Gardes-fous, erreurs de programmation et non refus : la segmentation ne doit pas porter de clé en double (sans quoi la table aurait une colonne parasite ; issue #22), garde-fou vérifié en premier ; la période et la transition doivent être celles de l'étape `exposition` ; les fenêtres transmises doivent porter les mêmes début et borne de période, transition, segmentation et convention (ci-dessus, « Données ») ; la clé d'un instant doit rester un entier exact en `double` ($< 2^{53}$) ; chaque clé (segment, âge) ne doit figurer qu'une fois dans la table d'exposition, sous son propre message (« clé (segment, âge) en double dans la table d'exposition »), contrôlé avant le suivant ; chaque instant d'événement doit tomber dans une cellule de la table d'exposition, et chaque segment de la table d'exposition figurer parmi les segments des observations (transmis ou recalculés), ces deux contrôles partageant le message « instant d'événement hors des cellules de la table d'exposition » ; les instants d'une cellule doivent être contigus ; enfin, la somme des d_j par cellule doit être `identical()` aux événements de la table d'exposition.

— **Statut** : $\hat{q}_x$, maximum de vraisemblance non paramétrique (risques discrets aux instants d'événement), résultat exact, valeur en arithmétique flottante ; non sans biais à effectif fini ; variance, asymptotique (ensemble à risque $\to \infty$ sur toute la cellule) ; intervalle, asymptotique, sous indépendance des unités d'observation. Sous l'unité contrat (rachat, ou décès sans `id_tete`), les contrats d'une même tête sont supposés indépendants : une dépendance positive rend l'intervalle trop étroit, ce que le relevé `tetes_multi_contrats` de l'étape `donnees` permet d'apprécier. La validité asymptotique de la variance sous troncature à gauche est une hypothèse, sans source établie.

— **Références** : à établir (issue #6) : appellation, formule de variance, résultat asymptotique sous troncature à gauche.

Cas construit qui départage les conventions d'ex aequo (cas K de la spécification, A.11 ; neuf têtes, attendus calculés à la main et vérifiés par les tests). Il réunit, à un même instant, l'entrée d'une tête et son décès ; l'entrée d'une tête et le décès d'une autre ; le décès d'une tête, la censure d'une autre et l'entrée d'une troisième : sous l'ensemble fermé, $\hat{q}_{69} = 1/6$ et $\hat{q}_{70} = 29/45$, avec $\Sigma_{70} = 3/10$; la convention prévisible et le binomial donnent d'autres valeurs de $\hat{q}_{70}$, si bien qu'une erreur de convention ne passerait pas inaperçue.

9.5 Statuts des grandeurs restituées

Le statut de chaque grandeur est restitué par le moteur, dans les paramètres de l'étape et dans la fiche (`statuts`, constante `STATUTS_BRUT` : colonnes `estimateur`, `grandeur`, `intervalle`, `statut`, `loi_supposee`, `calcul`), pour les seuls estimateurs retenus. Le tableau 17 en donne le contenu, complété pour les intervalles du binomial (note a) ; la mention « E^c fixe » ou « E^i fixe » est réservée aux intervalles exacts.

Table 17 : Statuts restitués par l'étape `brut` (constante `STATUTS_BRUT`).

Estimateur, grandeur	Intervalle	Statut	Loi supposée
`poisson`, taux	sans objet (estimation ponctuelle)	résultat exact : maximum de vraisemblance du modèle à force constante dans la cellule	force constante dans la cellule (vraisemblance de survie, proportionnelle à celle de Poisson)
`poisson`, taux	exact	exact sous la loi supposée : couverture $\geq$ niveau pour tout μ, E^c tenue pour fixe ; sous le modèle de survie (E^c aléatoire), couverture seulement asymptotique	$D \sim \mathcal{P}(\mu E^c)$, E^c fixe
`poisson`, taux	asymptotique	asymptotique ($\mu E^c \to \infty$) ; non défini à $D = 0$	$D \sim \mathcal{P}(\mu E^c)$
`poisson`, probabilité	sans objet	résultat exact : image du maximum de vraisemblance par $q = 1 - e^{-\mu}$, force constante sur la cellule entière	force constante dans la cellule
`poisson`, probabilité	exact	exact sous la loi supposée (transformation croissante des bornes exactes du taux : couverture identique)	$D \sim \mathcal{P}(\mu E^c)$, E^c fixe
`poisson`, probabilité	asymptotique	asymptotique (transformation croissante des bornes asymptotiques du taux : même régime) ; non défini à $D = 0$	$D \sim \mathcal{P}(\mu E^c)$
`binomial`, taux	sans objet	estimateur des moments : $E[D] = q\,E[E^i]$ exactement sous l'hypothèse de répartition ; convergent ; non sans biais à effectif fini	${}_{1-t}q_{x+t} = (1-t)q_x$, censure indépendante
`binomial`, taux	exact	exact sous la loi supposée pour une cellule observée entière (sans entrée ni censure, E^i entier) ; avec une entrée ou une censure dans la cellule, couverture non établie, même si E^i est entier (prolongement continu de la formule exacte si E^i n'est pas entier) [a]	$D \sim \mathcal{B}(E^i, q)$, E^i fixe
`binomial`, taux	asymptotique	asymptotique ($E^i q(1-q) \to \infty$) pour une cellule observée entière ; avec une entrée ou une censure dans la cellule, couverture non établie [a] ; non défini à $D = 0$ et à $D = E^i$	$D \sim \mathcal{B}(E^i, q)$
`kaplan_meier`, taux	sans objet	résultat exact : maximum de vraisemblance non paramétrique (risques discrets aux instants d'événement) ; non sans biais à effectif fini	troncature à gauche et censure non informatives, unités indépendantes ; aucune hypothèse de répartition dans la cellule
`kaplan_meier`, variance	sans objet	asymptotique (méthode delta sur $\log \hat{P}$, ensemble à risque $\to \infty$ sur toute la cellule) ; non définie à $q = 0$ et $q = 1$	idem

Estimateur, grandeur	Intervalle	Statut	Loi supposée
kaplan_meier, taux	asymptotique	asymptotique (ensemble à risque $\to \infty$ sur toute la cellule), sous indépendance des unités d'observation ; non défini à $q = 0$ et $q = 1$; sous l'unité contrat, contrats d'une même tête supposés indépendants (une dépendance positive rend l'intervalle trop étroit ; relevé tetes_multi_contrats)	idem
kaplan_meier, taux	exact	aucun : non restitué (sous troncature et censure, D ne suit aucune loi binomiale)	sans objet

[a] Statut documenté plus complet que le libellé inscrit dans la fiche : pour les deux intervalles du binomial, la colonne statut de STATUTS_BRUT (R/brut.R) ne pose pas encore la condition de cellule observée entière (intervalle exact : « exact sous la loi supposée si E^i est entier », suivi du seul cas non entier ; intervalle asymptotique : « asymptotique », avec son régime). Le statut qui fait foi est celui de ce tableau et de la section 9.3 (entrée « Statut ») ; la mise à jour du texte du code, donc de la fiche et des références de non-régression, est renvoyée à l'issue #31 (décision M55 du mainteneur, 18/10/2026). Aucune valeur calculée n'en dépend.
Les bornes exactes sont calculées numériquement par qgamma() ou qbeta() : leur valeur est une approximation numérique de la borne exacte, ce que la colonne calcul de la fiche écrit.

9.6 États des cellules et avertissements

Une cellule où l'estimateur n'est pas défini reçoit NA pour le taux et ses bornes, un *état* et, si elle compte un événement, un avertissement ; **jamais un refus** (décision M51, question 3) : un décès survenu le jour d'un anniversaire, au bord de la plage d'âges, ne doit pas bloquer la construction. Les états possibles sont ceux de ETATS_BRUT :

— defini, pour tous les estimateurs ;

— exposition_nulle : exposition nulle et aucun événement ; zéro explicite de la table d'exposition, compté dans les diagnostics, sans avertissement ;

— evenement_sans_exposition (Poisson) : $E^c = 0$ et $D > 0$; la vraisemblance croît sans borne et le maximum est infini (cas H8, H10b et H11 de l'étape exposition). Avertissement poisson_evenement_sans_exposition, de nombre le nombre de ces cellules, le message donnant aussi le nombre d'événements ;

— effectif_inferieur_aux_evenements (binomial) : $0 < E^i < D$, la tolérance appliquée ; le modèle binomial ne s'applique pas. C'est le cas d'une tête entrée dans la cellule puis touchée par l'événement, qui apporte moins d'une unité d'exposition initiale. Avertissement binomial_effectif_inferieur_aux_evenements, de nombre le nombre de ces cellules. Côté binomial, $E^i = 0$ avec $D > 0$ est impossible : chaque événement apporte un complément d'au moins un jour ;

— ensemble_a_risque_vide (kaplan_meier) : couverture incomplète de la cellule avant le premier instant où $d = r$ (section 9.4) ; taux, variance et bornes NA. L'avertissement kaplan_meier_ensemble_a_risque_vide n'est émis que pour les cellules de cet état **qui portent au moins un événement** ; son nombre est le nombre de ces cellules, et le message donne la somme de leurs événements (décision M53). Toutes les cellules de cet état restent comptées dans diagnostics$etats. Motif de la restriction : les cellules mal couvertes sont, en pratique, celles de l'âge minimal d'entrée, sans événement, et l'avertissement apparaîtrait sinon à presque chaque construction (mesure d'actuary sur des portefeuilles simulés).

Dans une cellule définie, les bornes asymptotiques valent en outre NA à $D = 0$ et, pour le binomial, à $D = E^i$ ($\hat{q} = 1$; borne exacte inférieure $(\alpha/2)^{1/D}$, borne supérieure 1) ; pour kaplan_meier, la variance et les bornes valent NA à $\hat{q}_x \in \{0, 1\}$.

9.7 Résultat, paramètres et diagnostics

etapes$brut$resultat contient une table par estimateur retenu, dans l'ordre de ESTIMATEURS_BRUT, aux **mêmes lignes et dans le même ordre** que la table d'exposition (zéro ligne si elle n'en a aucune) :

— poisson : clés de segmentation, age et, sous par_annee, annee ; evenements et exposition_centrale, identiques aux colonnes de l'exposition ; taux, borne_inf_exacte, borne_sup_exacte, borne_inf_asymptotique, borne_sup_asymptotique ; probabilite et ses quatre bornes, de même nom préfixé (probabilite_borne_inf_exacte…) ; etat ;

— binomial : clés, age, evenements, exposition_initiale ($E^c + \sum c_1$, avant tolérance), taux, les quatre bornes, etat ;

— kaplan_meier : clés, age, evenements et instants_evenement (entiers), taux ($\hat{q}_x$), variance, borne_inf_asymptotique, borne_sup_asymptotique, etat ;

— kaplan_meier_sauts, si kaplan_meier est retenu : une ligne par segment et par instant d'événement, triée par segment puis par instant : clés, age, numerateur ($j \cdot 133\,590/L$, entier), age_exact ($x + j/L$, quotient correctement arrondi du rationnel), ensemble_a_risque (r_j), evenements (d_j), survie_cellule (produit dans la cellule jusqu'à cet instant inclus) et variance_log_survie_cellule (Σ cumulée dans la cellule, infinie à partir d'un instant où $d = r$) ; ces deux dernières valent NA aux sauts des cellules non définies.

Paramètres effectifs : estimateurs (ordre canonique), niveau, exposition_initiale ("cellule"), tolerance_effectif, convention (retenue), unite_exposition, hypotheses (texte par estimateur retenu et, si Poisson est retenu, pour la probabilité, complété de sa portée ; constantes HYPOTHESES_BRUT et PORTEE_PROBABILITE) et statuts (tableau 17) ; si kaplan_meier est retenu, ensemble_a_risque ("ferme") et denominateur_instants (133 590, entier). **Diagnostics** : nombre de cellules, somme des événements, nombre de cellules sans événement, nombre de cellules où $D = E^i > 0$ à la tolérance près (NA si le binomial n'est pas retenu), et le tableau etats (estimateur, état, nombre de cellules) ; si kaplan_meier est retenu, la liste kaplan_meier, en fin de diagnostics, de cinq entiers : instants (nombre d'instants d'événement), evenements, evenements_a_l_instant_d_entree (événements dont l'instant coïncide avec le début d'observation retenu, $\max$(entrée, début de la période) : ceux qu'exclurait l'ensemble à risque prévisible), cellules_survie_nulle (cellules définies où $\hat{q}_x = 1$) et ensemble_a_risque_maximal (plus grand r_j, 0 sans événement). Les champs de kaplan_meier n'apparaissent que si l'estimateur est retenu : un plan qui ne le retient pas garde l'objet antérieur à son ajout. La fiche reçoit le champ hypotheses_brut (section 15.1).

9.8 Vérifications

tests/unitaires/test_brut.R vérifie, au niveau 0,95, avec z écrit en dur :

— des *formes closes*, indépendantes de qgamma(), qbeta() et pgamma() : Poisson à $D = 0$ (borne $-\ln(\alpha/2)/E$) et à $D = 1$ (borne supérieure u/E, u racine de $(1 + u)e^{-u} = \alpha/2$ obtenue par la méthode de Newton) ; binomial à E^i réel, $D = 0$ ($1 - (\alpha/2)^{1/n}$), $D = 1$ et $D = n$; séries explicites (Poisson $D \leq 5$, binomial $n \in \{2, 5, 12\}$) ;

— les cas d'une ligne et de fusion de l'étape exposition (H, F) et un cas composite à onze têtes (C), aux attendus calculés à la main : taux, bornes, états, avertissements, diagnostics ;

— chaque refus, depuis valider_plan() et depuis la fonction d'étape, avec le même code ; le niveau $1 - 2^{-53}$, admis, à bornes finies ;

— des identités exactes : evenements et exposition_centrale identical() aux colonnes de l'exposition, clés et ordre compris ; $\hat{\mu}E^c = D$ à 10^{-14} près ; exposition_initiale $= E^c +$ complement_initial_cellule ;

— l'*invariance* : un plan sans étape brut donne un objet identical() à celui du plan avec brut, privé de ses seuls ajouts (etapes$brut, fiche$hypotheses_brut, avertissements de l'étape et plan).

Portée du recoupement (CONTEXT.md). Les bornes exactes sont recoupées par stats::poisson.test() et, pour un E^i entier seulement, par stats::binom.test(), qui reposent sur les mêmes fonctions qgamma() et qbeta() que le moteur : le recoupement vérifie la formule, les branchements à $D = 0$ et $D = E^i$, la division par l'exposition et le partage $\alpha/2$, **pas** les fonctions spéciales ; ce sont les formes closes qui en font une vérification indépendante. Les bornes asymptotiques et la probabilité ne sont recoupées par aucune fonction de stats : leurs attendus sont calculés à la main.

tests/unitaires/test_brut_kaplan_meier.R vérifie l'estimateur kaplan_meier sur les cas H et F (une tête), le cas K des ex aequo, un cas Z (événement, censure et unité en cours au même instant x, censure exactement au début de la cellule) et le cas composite C : états, taux, variances et bornes calculés à la main, en formes closes ; les identités exactes (somme des d par cellule égale aux événements, clés et ordre de l'exposition, $1 \leq d \leq r$, survie au dernier saut égale à $1 - \hat{q}_x$) ; l'identité avec le binomial sur des cohortes d'âge exact simulées, qui sert aussi de test de débordement des entiers ; les refus, le message de etape_prealable_manquante, les types entiers, l'admission de age_atteint_annee ; l'invariance d'un plan sans kaplan_meier (objet identical() à celui du plan qui le retient, privé de ses seuls ajouts). Quatre tests portent sur les fenêtres transmises et l'appariement par clé (issue #38) : calculer_exposition() rend l'objet fenetres_exposition à côté de ses quatre éléments, l'étape brut donne des résultats identical() avec les fenêtres transmises et en appel direct sur les observations, et aucun objet de cette classe ne figure dans l'objet résultat ni dans la fiche ; trois permutations des lignes de la table d'exposition (l'inverse et deux tirages sous graine locale), par les deux voies, rendent la table kaplan_meier identical() à l'originale permutée, et la table des sauts et les diagnostics identical() ; chacun des cinq contrôles des fenêtres transmises arrête l'étape, un cas par contrôle, et des fenêtres inchangées ne l'arrêtent pas ; une table d'exposition dont une ligne est dupliquée arrête l'étape, en appel direct, par le message de la clé en double et non par celui de l'instant hors des cellules, et la table sans doublon ne l'arrête pas.

Contre-vérification par le package survival (ADR 0002 ; issue #7) : la production des valeurs par survfit se fait hors CI et hors batterie ; leur comparaison au moteur est un test ordinaire de la batterie unitaire, exécuté en CI. Le script tests/contre_verification/survfit_kaplan_meier.R, exécuté à la main sur un poste muni de survival et qui n'emploie pas tablexp, lit un jeu fixé (tests/contre_verification/jeu_kaplan_meier.csv : segments C, K et Z, chaque ligne en dates et en âges rationnels $x + j/L$) et appelle la fonction survfit de ce package, par cellule qui porte un événement, sur les intervalles $(a' - \delta, b']$ tronqués à la cellule, $\delta = 10^{-7} < 1/133\,590$, ce qui reproduit l'ensemble à risque fermé, intervalle sur l'échelle logit ; il écrit un fichier de valeurs dont l'en-tête porte la version de R, celle de survival, la date et sessionInfo(). Ce fichier, tests/contre_verification/valeurs_survfit_kaplan_meier.csv, est versionné ; il a été produit le 18/10/2026 sur le poste du mainteneur (Windows 11), hors CI, sous R 4.3.3 avec survival 3.5.8, versions consignées dans son en-tête, et compte onze sauts (cinq pour C, cinq pour K, un pour Z). Le test de test_brut_kaplan_meier.R compare ce fichier au moteur sur les cellules définies qui portent un événement, qui doivent être exactement celles que rend survfit : sept cellules (C 64, 65, 73 et 83, K 69 et 70, Z 70) et dix sauts ; la cellule C 38, rendue par survfit mais déclarée ensemble_a_risque_vide par le moteur, n'est pas comparée. Tolérances : âge exact à 10^{-13} près en absolu ; r et d exactement ; survie à chaque saut, $\hat{q}$ de la cellule

(comparé à $1 -$ surv au dernier saut) et écart-type de $\log \hat{P}$ à chaque saut à 10^{-13} près en relatif ; bornes de la cellule (comparées à `q_inf` et `q_sup` au dernier saut) à 10^{-12} près en relatif ; valeurs manquantes ou infinies (survie nulle, C 83 : écart-type infini, bornes manquantes) : identité exacte des positions et des valeurs. C'est un test ordinaire : un fichier absent, un en-tête incomplet (version de R, de `survival`, date ou `sessionInfo()`) ou un écart le fait échouer. Ses attendus sont des valeurs de *contre-vérification* (section 4.2) : produites par un package externe, écrites en dur, elles ne sont régénérées que sur ordre du mainteneur, avec un tableau avant / après (ADR 0002, point 4, et troisième annotation, A11). `survival` n'est chargé par aucune batterie.

`tests/test_reproductibilite.R` vérifie qu'une construction avec l'étape `brut` est `identical()` hors `execution` dans le même processus et dans un autre, que le rejeu de la fiche redonne l'objet, et l'invariance du plan sans `brut` sur deux plans de référence (R1 et R4, section 4.5) ; et de même pour le plan R6, qui retient `kaplan_meier`, avec l'invariance du plan sans `kaplan_meier`.

9.9 Constats de simulation : retrouver la loi vraie

Retrouver la loi vraie par les taux bruts est une batterie de constats de simulation (E2.19, E2.20), `tests/unitaires/test_constats_brut.R`, spécifiée par `actuary` le 17/10/2026 (issue #16, parties A.1 à A.8, et errata du même jour). Chaque test écrit, à chaque exécution, le nombre de têtes, le niveau α_k, la région d'acceptation, la fausse alarme et sa nature, la graine ; les valeurs observées ne sont pas reproduites ici (E3.4). Un constat de simulation n'est jamais un résultat exact sur le moteur : sous une graine fixée, la sortie du moteur tombe dans une région dont la probabilité de fausse alarme est écrite ; la graine étant fixe, le test est déterministe, et sa fausse alarme est la probabilité, sur les graines, qu'une implémentation juste échoue (comme pour le simulateur, section 16.3).

Cibles exactes sous le modèle simulé

Le simulateur tire des instants continus sous force constante par âge entier (section 16.1) ; le moteur ne voit que des jours. Dans une cellule de L jours, avec les forces μ_d (transition) et μ_r (sortie concurrente), $m = \mu_d + \mu_r$, une tête à risque au début d'un jour subit la transition avec la probabilité $h_d = (\mu_d/m)(1 - e^{-m/L})$, sort par la cause concurrente avec $h_r = (\mu_r/m)(1 - e^{-m/L})$ et survit au jour avec $e^{-m/L}$, probabilités constantes dans la cellule. D'où, en *résultats exacts* (preuve écrite de la spécification, A.1) :

— binomial et produit-limite sur une cellule observée entière, sans entrée ni censure et sans cause concurrente : $D \sim \mathcal{B}(n, q_x)$ exactement ; $E^i = n$ et le produit-limite égale le binomial (identité de la section 9.4, qui exige l'absence de censure dans la cellule). Avec une cause concurrente, $D \sim \mathcal{B}\big(n, (\mu_d/m)(1 - e^{-m})\big)$ reste exact, mais les sorties concurrentes censurent dans la cellule : $E^i < n$ dès qu'une sortie concurrente survient, le binomial ne vise plus q_x et le produit-limite vise q^* (point suivant) ;

— produit-limite sur la grille des jours : limite $q^* = 1 - (1 - h_d)^L$, égale à $1 - e^{-\mu_d}$ sans cause concurrente ;

— Poisson : $E[D - \mu^* E^c] = 0$, avec μ^* de la section 9.2 ;

— binomial, cohorte censurée par la période à la fraction c de la cellule : $D \sim \mathcal{B}(n, \pi_c)$, $\pi_c = 1 - e^{-\mu c}$, et les limites de la section 9.3, qui ne valent pas q.

Chaque test compare l'estimateur à sa cible exacte sous le modèle simulé, sauf en C2, où deux approximations de la cible sont faites et bornées (ci-dessous) ; l'écart de cette cible à la loi vraie est écrit à côté. Loi vraie des tests : celle du simulateur (Gompertz-Makeham $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$, $c = 1{,}1$, facteurs H 1 et F 0,6, plafond 1 ; rachat constant 0,05), convention `plus_fine`, délai nul, niveau 0,95.

Familles de tests

Hypothèse nulle, pour tous les tests sauf C3b : le moteur calcule correctement l'estimateur, si bien que sa sortie suit la loi déduite du modèle simulé (cibles ci-dessus) ; C3b a une hypothèse nulle composite, donnée avec lui. Pour chaque famille sont indiqués le jeu, la statistique et la région ; la fausse alarme et sa nature sont au tableau 19, les conditions de validité dans l'encadré qui suit.

— **C1, cohortes d'âge exact** : $n = 10^5$ têtes par cohorte, entrées le 2016-03-15 à l'âge exact x, période du 2016-03-15 au 2017-03-14, si bien que B est l'anniversaire $x+1$, que $L = 365$ et que la cellule x est observée entière (sauf C1.6, censurée au 2016-09-13, $c = 183/365$). *Statistiques* : binomial, $D \in [\text{qbinom}(\alpha_k/2, n, \pi), \text{qbinom}(1-\alpha_k/2, n, \pi)]$; Poisson, $U = D - \mu^* E^c \in [-t_{\text{lo}}, t_{\text{hi}}]$; produit-limite (C1.5), $z = (\text{logit}\,\hat{q} - \text{logit}\,q^*)\,\hat{q}(1-\hat{q})/\sqrt{\text{variance}}$, $|z| \leq \Phi^{-1}(1-\alpha_k/2)$. *Région de Poisson* : U est la somme de n variables indépendantes de même loi, de support fini (une valeur par jour de transition, de sortie concurrente, et la survie à la fin de la fenêtre) ; pour tout $\theta > 0$, $P(U \geq t) \leq \exp\bigl(-\theta t + n\Lambda(\theta)\bigr)$, Λ étant le logarithme de la fonction génératrice des moments d'un terme (inégalité $P(Y \geq a) \leq E[Y]/a$ pour $Y \geq 0$, appliquée à $Y = e^{\theta U}$) ; t_{hi} rend l'infimum en θ de cette borne égal à $\alpha_k/2$, et t_{lo} de même pour $-U$. La borne est exacte ; le calcul de t (`optimize()`, `uniroot()`) est une approximation numérique sans effet sur la validité : t est arrondi vers l'extérieur à 10^{-6} et la borne, recalculée en t, est vérifiée inférieure ou égale à $\alpha_k/2$. Le binomial n'est pas testé en C1.5, où le rachat concurrent censure dans la cellule : sa cible n'y est pas q.

— **C2, portefeuille tronqué et censuré, avec rachat** : mode uniforme, $n = 3 \cdot 10^5$, proportion d'hommes 0,5, âges 20 à 75, origine 2005-01-01, période 2015-01-01 au 2024-12-31, segmentation `sexe`, estimateurs `poisson` et `kaplan_meier`. Sélection *a priori*, sur l'exposition seule : cellules définies pour les deux estimateurs et $\lambda_x = \mu_x^* E_x^c \geq 30$; leur nombre $K = 83$ sous la graine déclarée est une constante du test, vérifiée à l'exécution ; une cellule sélectionnée où $\hat{q}$ vaut 0 ou 1 compte comme un rejet. Tests par cellule, à $a = 1{,}5 \cdot 10^{-4}/K$ chacun : Poisson, $D_x \in [\text{qpois}(a/2, \lambda_x), \text{qpois}(1 - a/2, \lambda_x)]$; produit-limite, $|z_x| \leq \Phi^{-1}(1-a/2)$, de cible $1 - e^{-\mu_{d,x}}$, et non la cible exacte q_x^* sur la grille des jours : l'écart relatif, de l'ordre de $-\mu_r/(2L)$, vaut environ $-7 \cdot 10^{-5}$ (rachat 0,05), négligeable devant l'erreur d'estimation par cellule. Cet écart de cible déplace les statistiques d'au plus des valeurs mesurées sous la graine déclarée, sur les $K = 83$ cellules : z_x d'au plus $1{,}4 \cdot 10^{-3}$ en valeur absolue par cellule, et la statistique globale $\sum_x z_x/\sqrt{K}$ de $-7{,}4 \cdot 10^{-3}$; la fausse alarme des tests produit-limite, par cellule et global, reste nominale asymptotique à ce déplacement près. Ce déplacement ne s'annule pas quand l'exposition croît : l'écart de cible est fixe et l'écart type de $\hat{q}$ décroît comme l'inverse de la racine de l'exposition, si bien que le déplacement de z_x croît comme $\sqrt{D_x}$. Tests globaux, à 10^{-4} chacun, $|z| \leq \Phi^{-1}(1 - 5 \cdot 10^{-5})$: Poisson signé, $\sum(D-\lambda)/\sqrt{\sum(1+\mu^*/365)\lambda}$, dont la variance est exacte à L fixé (scores de cellules disjointes non corrélés, preuve écrite de la spécification) ; produit-limite, $\sum_x z_x/\sqrt{K}$. *Approximation* : μ^*, donc λ_x, et la variance du test signé sont calculés à $L = 365$ pour toutes les cellules, y compris celles de 366 jours ; l'écart relatif de μ^* est alors au plus $6{,}7 \cdot 10^{-7}$ sur les 83 cellules retenues (mesuré), sans effet pratique sur les régions. Pas de test binomial : sa cible n'est pas q, et l'écart entre les deux définitions de l'exposition initiale y est indétectable à cet effectif.

— **C3, couverture des intervalles** : une simulation de $R \cdot m$ têtes indépendantes est découpée en R blocs de m têtes par un `segment` entier, $(\text{id_tete} - 1)$ div $m + 1$, ce qui donne R réplications indépendantes sous l'hypothèse usuelle sur le générateur ; un intervalle `NA` compte comme non couvrant. *C3a* : cohortes de femmes de 100 ans, $m = 50$, $R = 2\,000$; intervalles binomial exact, binomial logit, Poisson exact, Poisson log ; $N_{\text{couverts}} \sim \mathcal{B}(R, C)$ exactement, C étant la couverture exacte, calculée par le test par sommation sur la loi de D (et, pour Poisson, par convolution de la loi de l'exposition, $E^c = m - D + S/L$, calculée par transformée de Fourier rapide complétée de zéros jusqu'à une longueur donnée par `nextn()` : approximation numérique) ; région bilatérale exacte. *C3b* : portefeuille

d'hommes tronqué (âges 79 à 81 à l'entrée, origine et début de période le 2015-01-01, période jusqu'au 2016-12-31, rachat 0,05), $m = 600$, $R = 2\,000$, cellules 80 et 81 ; intervalles Poisson exact, Poisson log et produit-limite logit ; hypothèse nulle composite « couverture $\geq 0{,}95 - \eta$ », $\eta = 0{,}02$ déclaré (*choix de jugement*, sans source, appuyé sur les couvertures exactes calculées pour la spécification de #14) ; région $N \geq \text{qbinom}(\alpha_k, R, 0{,}93) = 1\,811$, de fausse alarme au plus $1{,}711 \cdot 10^{-5}$ pour toute couverture vraie $\geq 0{,}93$ (borne exacte, par monotonie). Chaque ligne écrit à part l'**erreur Monte-Carlo** de la couverture, $\sqrt{C(1-C)/R}$, fonction du seul nombre de réplications, et l'**erreur d'estimation**, demi-largeur moyenne des intervalles, fonction de l'exposition de chaque bloc. En C1 et C2, chaque test est une réplication unique : la largeur de la région y est l'erreur d'estimation, et il n'y a pas d'erreur Monte-Carlo de couverture.

Table 18 : Famille C1 : cibles, régions et fausses alarmes, valeurs calculées par le test (`tests/unitaires/test_constats_brut.R` ; $\alpha_k = 2 \cdot 10^{-5}$ par test ; graines 20261611 à 20261616).

Cohorte	π (binomial)	Région de D	FA exacte	Région de U	$\mu^*/\mu - 1$
C1.1 décès, 60 ans, H	0,008153179142	[697 ; 939]	$1{,}917 \cdot 10^{-5}$	[−133,169304 ; 140,755387]	$1{,}121 \cdot 10^{-5}$
C1.2 décès, 80 ans, F	0,03150085056	[2 917 ; 3 388]	$1{,}927 \cdot 10^{-5}$	[−265,646914 ; 272,957234]	$4{,}385 \cdot 10^{-5}$
C1.3 décès, 100 ans, F	0,182091641	[17 691 ; 18 731]	$1{,}998 \cdot 10^{-5}$	[−644,959663 ; 650,399965]	$2{,}754 \cdot 10^{-4}$
C1.4 rachat, 40 ans, $q^r = 0{,}05$, sans décès, unité contrat	0,05	[4 709 ; 5 296]	$1{,}989 \cdot 10^{-5}$	[−335,760302 ; 342,851378]	$7{,}027 \cdot 10^{-5}$
C1.5 décès, 60 ans, H, rachat concurrent $q^r = 0{,}30$	non testé	—	—	[−121,820313 ; 129,412807]	$5{,}000 \cdot 10^{-4}$
C1.6 décès, 100 ans, F, censure à $c = 183/365$	$\pi_c = 0{,}09586612604$	[9 192 ; 9 986]	$1{,}957 \cdot 10^{-5}$	[−466,670347 ; 473,207095]	$2{,}754 \cdot 10^{-4}$

En C1.5, le produit-limite est comparé à q^*, dont l'écart relatif à $1 - e^{-\mu_d}$ vaut $-4{,}864 \cdot 10^{-4}$; région $|z| \leq 4{,}2649$, fausse alarme nominale (asymptotique). En C1.6, la cible du binomial « cellule » est 0,1745649731, à $-4{,}13$ % de la loi vraie (section 9.3) ; le produit-limite y est à l'état `ensemble_a_risque_vide` : il refuse d'extrapoler là où le binomial s'appuie sur son hypothèse de répartition. *Identités* vérifiées à côté des constats, par des tests ordinaires : $E^i = n$ et $\hat{q}$ produit-limite $= \hat{q}$ binomial en C1.1 à C1.4 ; en C1.6, taux binomial $= D/((n-D)c+D)$ et E^c + `complement_initial_fenetre` $= nc$; en C3a, bornes du produit-limite égales aux bornes logit binomiales sur les 2 000 blocs. Couvertures exactes de C3a, écrites en dur et recalculées par le test : binomial exact 0,974129060872, binomial logit 0,958230588612, Poisson exact 0,968699258108, Poisson log 0,955857903542 ; régions [1 916 ; 1 975], [1 877 ; 1 951], [1 902 ; 1 967] et [1 871 ; 1 947], à $\alpha_k = 3 \cdot 10^{-5}$.

Fausse alarme familiale

Elle est bornée par l'inégalité de l'union pondérée, valable *sans hypothèse d'indépendance* ; les tests d'une même cohorte (binomial et Poisson) étant fortement dépendants, aucune valeur « sous indépendance » n'est donnée.

Table 19 : Allocation du niveau entre les familles (graines 20261611 à 20261616, 20261621, 20261631, 20261641, distinctes de celles du simulateur).

Famille	Tests	$\sum \alpha_k$	Nature de la fausse alarme
C1	12, à $2 \cdot 10^{-5}$	$2{,}4 \cdot 10^{-4}$	binomial exacte ; Poisson bornée exactement ; produit-limite (C1.5) nominale asymptotique
C2 par cellule	$2 \times K$, à $1{,}5 \cdot 10^{-4}/K$	$3{,}0 \cdot 10^{-4}$	nominale asymptotique (produit-limite : à un déplacement mesuré près ; Poisson : exacte si E^c était fixe, valeur calculée et écrite par le test)
C2 globaux	2, à 10^{-4}	$2{,}0 \cdot 10^{-4}$	nominale asymptotique (produit-limite : à un déplacement mesuré près)
C3a	4, à $3 \cdot 10^{-5}$	$1{,}2 \cdot 10^{-4}$	exacte
C3b	6, à $2 \cdot 10^{-5}$	$1{,}2 \cdot 10^{-4}$	bornée exactement
Total	190	$9{,}8 \cdot 10^{-4} \leq 10^{-3}$	dont exacte ou bornée exactement au plus $4{,}16 \cdot 10^{-4}$ (somme calculée par le test : $4{,}153 \cdot 10^{-4}$), nominale asymptotique $5{,}2 \cdot 10^{-4}$

La batterie unitaire porte ainsi deux familles de constats, celle du simulateur (au plus 10^{-3}, section 16.3) et celle-ci : sa fausse alarme est au plus $1{,}98 \cdot 10^{-3}$, nominale pour les parties asymptotiques.

Puissance et témoins

Deux témoins exécutés par le fichier montrent que les tests détectent une erreur plausible : en C1.6, l'exposition initiale « fenêtre » à la place de « cellule » donne un taux hors de la région binomiale (puissance exacte, calculée par sommation sur $\mathcal{B}(n, \pi_c)$, égale à 1 à 10^{-6} près) ; en C2, une cible décalée d'un an d'âge fait rejeter les deux tests globaux (constat sous la graine déclarée). Les tests par cellule de C2 n'ont pratiquement pas de puissance contre un tel décalage : ce sont les globaux qui la portent (puissances calculées par la spécification, A.4). La convention « prévisible » de l'ensemble à risque du produit-limite n'est pas détectable par ces constats (écart de l'ordre de 10^{-3} en relatif sur q, spécification, A.4) : sa garantie est le cas K (section 9.4).

Statuts et sources d'erreur (E2.20).

— *Résultats exacts* : les cibles sous le modèle simulé, la loi binomiale de D en C1 et de N_{couverts} en C3a, les couvertures exactes de C3a, la borne exponentielle de Poisson, la borne de C3b, l'inégalité de l'union, la puissance exacte du témoin C1.6.

— *Résultats asymptotiques* : les régions du produit-limite (C1.5, C2) et des tests de Poisson de C2 sous le modèle de survie.

— *Approximations numériques* : quantiles binomiaux et de Poisson, seuils t de la borne exponentielle (arrondis vers l'extérieur et vérifiés), convolution par transformée de Fourier.

— *Approximations de la cible* (C2), d'écart relatif écrit : le produit-limite comparé à $1 - e^{-\mu_d}$ plutôt qu'à q^* (environ $-7 \cdot 10^{-5}$), μ^* et la variance du test signé calculés à $L = 365$ pour toutes les cellules (au plus $6{,}7 \cdot 10^{-7}$ sur les 83 cellules retenues, mesuré). La première déplace z_x d'au plus $1{,}4 \cdot 10^{-3}$ par cellule et la statistique globale de $-7{,}4 \cdot 10^{-3}$ (mesurés sous la graine déclarée) ; la fausse alarme du produit-limite, test global compris, est nominale asymptotique à ce déplacement près, qui croît comme $\sqrt{D}$ et ne s'annule pas quand l'exposition croît.

— *Constat de simulation* : le passage des tests sous les graines déclarées.

— *Erreur Monte-Carlo* : en C3, la dispersion du nombre d'intervalles couvrants, fonction de R. *Erreur*

d'estimation : la largeur des régions de C1 et C2 et la demi-largeur des intervalles de C3, fonction de l'exposition.

— *Hypothèses* : indépendance des blocs et des cohortes sous le générateur (hypothèse usuelle, non démontrée) ; validité asymptotique des régions de C1.5 et de C2 ; $\eta = 0{,}02$, choix de jugement.

9.10 Fonctions du module

Table 20 : Définitions de premier niveau de R/brut.R.

Nom	Statut	Entrées	Sortie et rôle
ESTIMATEURS_BRUT	constante	—	estimateurs disponibles, dans l'ordre canonique des résultats
EXPOSITION_INITIALE_BRUT	constante	—	"cellule" : définition de l'exposition initiale du binomial, sans paramètre (section 9.3)
TOLERANCE_EFFECTIF	constante	—	10^{-9} : tolérance absolue sur $\lvert E^i - D \rvert$
ETATS_BRUT	constante	—	états possibles d'une cellule, par estimateur (section 9.6)
DENOMINATEUR_INSTANTS	constante	—	133 590 (365×366), entier : dénominateur commun des instants de kaplan_meier
ENSEMBLE_A_RISQUE_KM	constante	—	"ferme" : convention d'ensemble à risque (section 9.4)
HYPOTHESES_BRUT	constante	—	textes des hypothèses, par estimateur et pour la probabilité, inscrits dans les paramètres et dans la fiche
PORTEE_PROBABILITE	constante	—	portée de la probabilité de Poisson selon la cellule (année d'âge, année civile, âge × année)
STATUTS_BRUT	constante	—	statut, loi supposée et calcul de chaque grandeur (tableau 17)
controler_parametres_brut()	interne	estimateurs, niveau, par_annee	refus des valeurs (section 9.1) ; appelée par valider_plan() et, en appel direct (controler = TRUE), par la fonction d'étape (issue #22)
taux_poisson()	interne	D, E^c, niveau	taux, bornes exactes et asymptotiques, état (section 9.2)
taux_binomial()	interne	D, E^i, niveau, tolérance	taux, bornes, état, cellules où $D = E^i > 0$ (section 9.3)
taux_kaplan_meier()	interne	fenêtres transmises par l'étape exposition ou observations (appel direct), table d'exposition, segmentation, transition, période, délai, convention, niveau, paramètres de l'étape exposition	tables kaplan_meier et kaplan_meier_sauts, diagnostics (section 9.4)

Nom	Statut	Entrées	Sortie et rôle
calculer_taux_bruts()	interne	table d'exposition, segmentation, par_annee, convention, unité d'exposition, estimateurs, niveau ; pour kaplan_meier : observations (fenêtres transmises par l'étape exposition dans une construction, table des observations de l'étape donnees en appel direct ; issue #38), transition, période, délai, paramètres de l'étape exposition ; controler (TRUE par défaut)	fonction d'étape brut : paramètres effectifs, résultat, diagnostics, avertissements (section 9.7) ; avec controler = FALSE, réservé à construire_table(), ni la segmentation, ni les paramètres de l'étape, ni (pour kaplan_meier) la transition, la période et le délai ne sont recontrôlés (issue #22)

Partie VI

Module lissage

Lissage : R/lissage.R

Module prévu (E2.12) : lissages des taux bruts. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

10.1 Fonctions du module

Table 21 : Définitions de premier niveau de R/lissage.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/lissage.R absent			

PARTIE VII

Module positionnement

Positionnement sur une table de référence : R/positionnement.R

Module prévu (E2.12, E2.15) : positionnement d'une table d'expérience sur une table de référence. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

11.1 Fonctions du module

TABLE 22 : Définitions de premier niveau de R/positionnement.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/positionnement.R absent			

PARTIE VIII

Module prospective

Tables prospectives : R/prospective.R

Module prévu (E2.13, v2) : tables prospectives, toute table prospective étant une surface de mortalité.

12.1 Fonctions du module

TABLE 23 : Définitions de premier niveau de R/prospective.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/prospective.R absent			

PARTIE IX

Module fermeture

Fermeture aux âges extrêmes : R/fermeture.R

Module prévu (E2.12) : fermeture de la table aux âges extrêmes. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

13.1 Fonctions du module

TABLE 24 : Définitions de premier niveau de R/fermeture.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/fermeture.R absent			

PARTIE X

Module validation

Validation et suivi d'expérience : R/validation.R

Module prévu (E2.12, E2.18) : validation de la table construite et suivi d'expérience. Il sera créé par une branche ultérieure (docs/feuille-de-route.md, § 2).

14.1 Fonctions du module

TABLE 25 : Définitions de premier niveau de R/validation.R.

Nom	Statut	Entrées	Sortie et rôle
aucune fonction : fichier R/validation.R absent			

Partie XI

Module rapport

Fiche de construction et empreintes : R/rapport.R

Le module rapport porte les quantités des rapports, pas leur rendu (E4.1) : la fiche de construction, les empreintes des données reçues (section 15.2), du résultat et du code (section 15.3 ; issue #67) et le texte du plan et de la fiche (section 15.4 ; ADR 0011, qui remplace le point 12 de l'ADR 0007). Le rejeu outillé, avec le contrôle de version_fiche, est dans R/construire_table.R, le module rapport ne pouvant pas appeler construire_table() (section 6.4 ; ADR 0012, point 3). Le rapport figé HTML et l'écriture des tables viendront avec la branche claude/rapport-fige (décision M68, C.3 ; ADR 0011, troisième annotation du 20/10/2026).

15.1 Fiche de construction

La *fiche de construction* est la description lisible par machine d'une construction, qui suffit à la rejouer à l'identique (E4.5 ; ADR 0005, point 6 ; ADR 0007, point 6.2, précisé par l'ADR 0011, point 5). Elle est assemblée par assembler_fiche() et rangée dans le champ fiche de l'objet. Elle ne contient **aucun horodatage** : deux constructions identiques ont des fiches identiques. Ses champs, dans cet ordre, en huit blocs :

— **identification** : version_fiche (VERSION_FICHE, 4) ; versions (tablexp, lue dans la description du package installé, et R.version.string) ; plateforme, dans l'ordre sysname, release et machine (système, version, machine, lus par Sys.info()), blas (BLAS, par extSoftVersion()), lapack (LAPACK, par La_library()) et bibliotheque_c (bibliothèque C, par extSoftVersion() ; ci-dessous, « Bibliothèque C ») ;

— **données** : roles, dans l'ordre du catalogue des rôles (CATALOGUE_ROLES) ; provenance, dans l'ordre fichier, empreinte, algorithme, et provenance_mention (« aucun fichier (données construites en mémoire) » si provenance est NULL) ; empreintes, dans l'ordre algorithme, representation, donnees, fichier, algorithme_fichier, representation_resultat, resultat, code, statut_resultat_code (sections 15.2 et 15.3) ;

— **plan** : plan, le **plan canonique** (ci-dessous) ;

— **paramètres retenus**, dans l'ordre du calcul : transition_code, code interne de la transition retenue ; fin_effective ; unite_observation ; recette ; convention (demandée, native, retenue, forcée) ; unite_exposition ; par_annee ;

— **exclusions** : exclusions (par motif, avec leur nombre ; une ligne exclue sous plusieurs motifs déclarés compte sous chacun) ; nombre_lignes_exclues (integer, nombre de lignes distinctes exclues, 0L

sans exclusion ; ci-dessous) ;

— **hypothèses** : avertissement_unite, s'il y a lieu (message de l'avertissement unite_contrat) ; hypothese_censure ; hypotheses_brut, **seulement si l'étape brut est exécutée** : copie des paramètres effectifs de l'étape estimateurs, niveau, exposition_initiale, tolerance_effectif, hypotheses et statuts, plus ensemble_a_risque et denominateur_instants si l'estimateur kaplan_meier est retenu (section 9.7) ;

— **aléa** : graine et graine_mention (NA et « aucune étape aléatoire » dans cette branche) ; generateur (GENERATEUR) ;

— **avertissements** : avertissements, en dernier.

Ordre des champs (issue #64 ; avis d'actuary du 20/10/2026, consigné sous la décision M69, point 5). L'ordre suit la lecture d'un certificateur (E3.7) : ce qui identifie la construction, puis les données reçues, avec les rôles, la provenance et l'empreinte des données, que la procédure de rejeu fait vérifier avant toute exécution ; le plan déclaré ; les paramètres retenus, dans l'ordre où le calcul les fixe ; les exclusions ; les hypothèses, regroupées ; l'aléa ; les avertissements, en dernier. Auparavant, l'ordre de construction dispersait ces éléments : l'empreinte des données venait après l'aléa, la provenance était séparée des rôles, et hypothese_censure et hypotheses_brut étaient séparés par l'aléa, la provenance, les empreintes et les avertissements. L'ordre est fixé par une seule constante, ORDRE_CHAMPS_FICHE : assembler_fiche() construit la fiche puis la réordonne par ordonner_fiche(), qui garde les seuls champs présents ; un plan sans étape brut a donc le même ordre, sans hypotheses_brut. Un champ présent hors de cet ordre est une erreur de programmation, jamais un champ retiré en silence. Le corps de la fiche écrite (section 15.4) suit cet ordre, et les blocs de lecture en sont une partition (ci-dessous, « Blocs de lecture »). Seul l'ordre des noms de la fiche change : aucune valeur, aucune table ni aucun verdict.

Plan canonique (ADR 0011, points 5.1 et 5.2 ; issue #42). Deux plans de même sens doivent donner la même fiche et le même objet : avant l'issue #42, un plan aux champs permutés, à borne de période stockée en entier ou au délai 0L donnait des fiches différentes, et, pour la période, des paramètres d'étape différents (ADR 0011, « Contexte », mesure 3). construire_table() applique donc canoniser_plan() une seule fois, juste après valider_plan() : champs de premier niveau dans l'ordre de CHAMPS_PLAN ; transition dans l'ordre depart, arrivee ; periode dans l'ordre debut, fin, chaque borne de classe Date à stockage double ; delai_declaration en double ; graine : un integer sans NA est rendu tel quel, quelle que soit sa longueur, et un double unique, fini, à valeur entière et de valeur absolue au plus .Machine$integer.max est converti en integer, parce que set.seed() convertit son argument en entier et que deux graines de même valeur doivent donner la même fiche ; toute autre graine est une erreur de programmation (ADR 0011, annotation du 20/10/2026 du point 5.1 (g)). Cette conversion est **sans effet dans cette branche** : aucune étape ne tire d'aléa, si bien que valider_plan() refuse toute graine (graine_inattendue) avant la canonisation ; le contrat de validation de la graine sera fixé avec la première étape aléatoire (issue #61) ; paramètres de chaque étape dans l'ordre de PARAMETRES_ETAPES ; valeurs des trois champs qui forment un ensemble dans l'ordre de leur catalogue (ci-dessous, issue #63) : segmentation dans l'ordre de ROLES_SEGMENTATION, etapes$donnees$exclusions dans l'ordre du catalogue des motifs c(MOTIFS_LIGNES, MOTIFS_TETE) (tableau 12), etapes$brut$estimateurs dans l'ordre de ESTIMATEURS_BRUT ; toute autre valeur inchangée. Les conversions sont sans perte (tout entier de R est un double exact) et la canonisation est idempotente. Un stockage que la liste fermée de ces conversions ne prévoit pas, une étape absente du catalogue, ou un nom d'étape ou de paramètre en double, qui serait sinon réduit en silence à sa première occurrence, arrête le calcul par une erreur de programmation : il ne peut venir que d'un plan non validé, valider_plan() refusant ces cas en amont. Le plan canonique n'est pas un plan « résolu » : une convention déclarée plus_fine y reste plus_fine, la convention retenue étant dans fiche$convention, car réécrire une déclaration par sa résolution serait une autre déclaration (aucun défaut implicite, ADR 0007, point 5.1). Le *plan effectif* d'E4.5 est ainsi la paire formée du plan canonique et des paramètres retenus

(paramètres effectifs des étapes et, dans la fiche, transition_code, fin_effective, unite_observation, recette, convention, unite_exposition, par_annee, hypothese_censure, hypotheses_brut) (note de lecture d'E4.5, décision M64, Q6). L'ordre des rôles et celui des noms de la provenance n'ont d'effet ni sur les étapes ni sur l'empreinte des données ; la fiche les consigne dans un ordre fixe (décision M64, Q4). L'**asymétrie est voulue** : les données sont empreintées telles qu'elles sont reçues, parce que l'empreinte identifie l'entrée, tandis que le plan est canonisé selon son sens, parce que la fiche identifie la déclaration.

Ordre des valeurs des champs ensembles (issue #63 ; ADR 0011, quatrième annotation du 20/10/2026, qui révise le point 5.1 (h) ; décision M69, point 2). segmentation, exclusions et estimateurs sont des ensembles : valider_plan() y refuse les doublons, si bien que deux plans qui ne diffèrent que par l'ordre de ces valeurs ont le même sens et doivent, selon la règle ci-dessus, donner la même fiche. Tant que cet ordre était conservé tel qu'il était déclaré, la fiche se contredisait pour estimateurs : fiche$plan gardait l'ordre déclaré, alors que fiche$hypotheses_brut$estimateurs et les résultats de l'étape brut sont toujours dans l'ordre de ESTIMATEURS_BRUT (section 9) ; pour exclusions et segmentation, seul l'ordre de présentation changeait (lignes et colonnes des étapes, ordre des avertissements exclusion), aucune valeur. Les vecteurs d'ordre sont passés en argument à canoniser_plan() (ordres, liste nommée par le chemin du champ dans le plan : segmentation, etapes/donnees/exclusions, etapes/brut/estimateurs), comme CHAMPS_PLAN et PARAMETRES_ETAPES : canoniser_plan() ne lit ainsi aucune constante d'un autre module, le seul endroit qui sait quel champ suit quel catalogue est l'appel, et la fonction se teste avec des catalogues fabriqués ; ce choix de localité n'est pas imposé par le graphe des modules, qui autoriserait la lecture directe. La liste est construite au moment de l'appel par ordres_canoniques() (R/construire_table.R), et non par une constante de premier niveau : sans champ Collate dans DESCRIPTION, les fichiers de R/ sont évalués par ordre alphabétique à l'installation, et construire_table.R l'est avant donnees.R, dont les constantes (ROLES_SEGMENTATION, MOTIFS_LIGNES, MOTIFS_TETE) ne sont pas encore définies ; brut.R (ESTIMATEURS_BRUT) l'est déjà, mais la fonction lit les trois vecteurs au même endroit. Aucune valeur n'est retirée en silence : avant de réordonner, une valeur absente du vecteur d'ordre, un doublon, un NA, un attribut ou un type autre que character arrêtent le calcul par une erreur de programmation (garde défensive, valider_plan() refusant ces cas en amont) ; une étape absente du plan (plan sans brut) n'a rien à canoniser. Un emboîtement de segments, s'il devient nécessaire avec plusieurs variables de segmentation, sera porté par un paramètre déclaré à part, jamais par l'ordre de segmentation. **Effet** : les plans R1 à R6, déjà dans l'ordre canonique, sont inchangés ; pour un plan non canonique sur l'un de ces champs, fiche$plan et l'ordre de présentation des étapes et des avertissements changent, aucune valeur numérique. La forme canonique des valeurs du plan fait partie de ce que numérote version_fiche : une fiche de version 3 ou ultérieure garantit cet ordre, une fiche de version 2 ne le garantit pas ; l'issue #63 tombe sous l'incrément de 2 à 3 porté par l'issue #57 (ADR 0011, point 10). Les tests de tests/unitaires/test_construire_table.R vérifient, pour chacun des trois champs et pour les trois ensemble, que l'objet d'un plan permuté est identical() hors execution à celui du plan canonique, fiche comprise, que la canonisation est idempotente et que son résultat reste valide, que le rejeu de la fiche d'un plan permuté redonne l'objet, et que la valeur inconnue, le doublon et l'attribut donnent l'erreur de programmation.

Copies retirées (issue #42, avec l'issue #22, point 6 ; décision M64, Q2). Jusqu'à la version 1 de la fiche, transition, periode, delai_declaration et segmentation figuraient à la fois dans fiche$plan et au premier niveau de la fiche, sans que celle-ci dise laquelle faisait foi, periode voisinant en outre fin_effective. Ces quatre copies sont retirées ; restent transition_code et fin_effective, qui sont des paramètres retenus et non des copies, ainsi que les reprises de l'objet nécessaires à une fiche autosuffisante (convention, recette, exclusions, avertissements, hypotheses_brut ; E3.7). Le changement de l'ensemble des champs fait passer version_fiche de 1 à 2 (ADR 0011, point 10) ; il a été visé par actuary et consigné par la seconde régénération des références (section 4.5).

Nombre de lignes exclues (issue #57 ; décision M69, point 1). `exclusions` compte les lignes par motif : une ligne qui porte deux motifs déclarés y compte deux fois, si bien que la somme des décomptes surestime le volume écarté. `nombre_lignes_exclues` donne le nombre de lignes *distinctes* exclues, repris de l'étape `donnees` (`resultat$nombre_lignes_exclues`, section 7.3) : c'est le nombre d'éléments de l'union des numéros de lignes exclues sous les motifs déclarés. Les lignes conservées étant exactement les autres, le rapprochement « lignes reçues = lignes conservées + `nombre_lignes_exclues` » est une identité exacte, par construction, et non une approximation. Un plan sans exclusion, ou dont les motifs déclarés ne portent sur aucune ligne, donne `0L`. Le champ suit `exclusions` dans la fiche et appartient au bloc de lecture « hypothèses » (ci-dessous, « Blocs de lecture »). Par décision du mainteneur (M69, point 1 ; issue #69), l'outil restitue des grandeurs objectives, le jugement de pertinence des exclusions et leur justification restant à l'entreprise et au certificateur, et aucune justification déclarée n'entre dans le plan. La même décision renvoie à l'issue #69 la question réglementaire des exclusions et les grandeurs qui mesureraient leur importance (décès et exposition retirés par les exclusions, SMR des lignes exclues). L'ajout du champ change l'ensemble des champs et fait passer `version_fiche` de 2 à 3 (ADR 0011, point 10 (c)) ; l'incrément est porté par le premier commit de la branche `claude/fiche-rapport` qui change la définition de la fiche, celui de l'issue #57, et il est le seul de la branche (ADR 0011, point 10 (a), et troisième et quatrième annotations du 20/10/2026). Le changement a été visé par `actuary` ; aucune table, aucun taux ni aucun verdict ne change.

Bibliothèque C (issue #77 ; ADR 0011, seconde annotation du 24/10/2026). `fiche$plateforme$bibliotheque_c`, dernier sous-champ de `plateforme`, est rendu par `bibliotheque_c()` : c'est l'élément `iconv` de `extSoftVersion()`, tel quel. R 4.3.3 construit cette valeur dans la fonction C `do_eSoftVersion` du fichier `src/main/platform.c` de ses sources (étiquette `R-4-3-3` du dépôt Subversion de R, https://svn.r-project.org/R/tags/R-4-3-3/src/main/platform.c, l. 3592–3610, relu le 24/10/2026), où le préprocesseur retient la première de quatre formes, dans cet ordre :

1. « GNU libiconv X.Y » (« Apple or GNU libiconv X.Y » sous macOS) si R est lié à la bibliothèque GNU libiconv, **même sur un système glibc** ;
2. « win_iconv » sous Windows ;
3. « glibc X.Y » sous la glibc sans libiconv, la version étant celle que rend la fonction C `gnu_get_libc_version` **à l'exécution**, donc celle de la glibc chargée et non celle de la compilation ;
4. « unknown » sinon, gardé tel quel.

Le champ n'identifie la glibc, donc la bibliothèque mathématique (`libm`) dont viennent les écarts d'un ulp entre machines (section 15.3), **que si sa valeur commence par « glibc »** ; sinon il désigne l'implémentation d'iconv (libiconv, win_iconv) ou rien (« unknown »), jamais la `libm`, et la fiche n'en dit pas davantage. Rien n'est deviné : si l'élément est absent, `NA`, vide ou fait de blancs, ou n'est pas un texte, le champ vaut la mention « non relevée » (`MENTION_BIBLIOTHEQUE_C_NON_RELEVEE`), distincte de la valeur brute « unknown » de R. Sur un hôte hors de la plateforme de référence (Ubuntu 24.04.5, R 4.3.3 ; *constat de mesure* du 24/10/2026), la fonction rend « glibc 2.39 » ; sur la plateforme de référence, l'en-tête des références régénérées le 24/10/2026 porte « glibc 2.35 » (section 4.5). La forme Windows (« win_iconv », attendue d'après le cas 2) n'a pas encore été mesurée : elle reste *non vérifiée* par mesure (critère 2 de l'issue #77, à fournir par le mainteneur). Comme tout `fiche$plateforme`, le champ est **hors du périmètre** de l'empreinte du résultat (`CHEMINS_HORS_RESULTAT`, section 15.3) et des grandeurs des références, dont il n'occupe que l'en-tête. Son ajout change l'ensemble des champs et fait passer `version_fiche` de 3 à 4 (ADR 0011, point 10 (c)) : sans l'incrément, une fiche de version 3, qui n'a pas le champ (« champ inexistant à cette version »), ne se distinguerait pas d'une fiche où la bibliothèque n'a pas été relevée (« relevé tenté, sans valeur »). C'est le seul incrément de la branche `claude/rapport-fige` ;

fiche$version_fiche étant dans le périmètre de l'empreinte du résultat, celle-ci change pour les six plans de référence. Ces deux valeurs, avec l'empreinte du code, que change tout commit de R/, sont les seules des références qui aient changé (visa d'actuary du 24/10/2026, décision M83 ; section 4.5). Aucune table, aucun taux ni aucun verdict ne change.
Les champs qui n'ont pas d'objet (convention et unité d'exposition d'un plan sans étape exposition) valent NULL ou NA. **Rejeu** : construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance) redonne un objet identical() hors execution ; c'est un test de la batterie de reproductibilité, aussi à partir du texte de la fiche. Le rejeu outillé, rejouer_fiche(donnees, fiche), fait cet appel après avoir contrôlé la fiche et les données, et refuse en particulier une fiche dont version_fiche n'est pas une version admise, la version courante seule (VERSIONS_FICHE_REJEU ; section 6.4). La fiche s'écrit et se relit en texte (section 15.4), sous les contraintes posées par l'ADR 0007, point 12.2 (texte lisible par un relecteur, aller-retour identique, nombres compris, aucune évaluation de code à l'import, R de base, version), maintenues par l'ADR 0011, qui remplace ce point.

15.2 Empreinte des données

Algorithme et portée

L'empreinte identifie l'entrée reçue par le moteur ; elle n'a pas de rôle de sécurité. Elle est un MD5, calculé par tools::md5sum() sur un fichier temporaire écrit en binaire (empreinte_donnees()), l'algorithme étant nommé dans la fiche (empreintes$algorithme, "md5") pour qu'un changement ultérieur soit visible et daté (décision M45, P8 ; ADR 0007, point 9). R 4.3.3 n'offre ni SHA-256 ni empreinte directe d'un vecteur d'octets dans tools (ADR 0007, mesures), et le moteur n'admet aucun package hors priorité base. L'empreinte n'est pas calculée sur serialize() : l'en-tête de la sérialisation porte la version de R qui écrit, si bien qu'à données identiques l'empreinte changerait avec la version de R. Elle est calculée sur une *représentation textuelle canonique*, définie indépendamment de la manière de l'écrire, qu'un tiers peut recalculer avec n'importe quel outil MD5.

La fiche porte : algorithme ; representation (ENTETE_CANONIQUE, "tablexp canonique 2") ; donnees ; fichier et algorithme_fichier, reçus par provenance (NA sans fichier) ; puis representation_resultat, resultat, code et statut_resultat_code, qui portent les empreintes du résultat et du code et leur statut (section 15.3). L'argument provenance est NULL ou une **liste nue**, dont le seul attribut admis est celui des noms, de **exactement trois éléments**, nommés fichier, empreinte et algorithme dans un ordre quelconque, chacun une chaîne (character) de longueur 1, sans attribut, ni NA ni vide ; un data.frame, un objet de classe, une liste munie d'un autre attribut, un élément de plus (même nommé NA) ou une chaîne munie de noms, d'une dimension ou d'une classe sont refusés : refus provenance_invalide (controler_provenance(), par liste_aux_noms() et chaine_unique(), qui s'appuient sur forme_exacte() ; issues #21 et #48).

Représentation canonique

texte_canonique() écrit les octets suivants (ADR 0007, point 9.3, et annotation du 17/10/2026, point f) :

— **périmètre** : les seules colonnes auxquelles un rôle est déclaré, **toutes les lignes reçues**, avant exclusions, dans l'ordre reçu ; les exclusions sont des paramètres du plan, non une propriété de l'entrée ; une base dont les lignes sont permutées a une autre empreinte, et ses résultats ne sont pas garantis identiques au bit près (E4.6 ne porte que sur des données identiques) : les lignes conservées et les observations suivent l'ordre reçu ; les grandeurs entières ou à valeur entière (exposition en jours, événements, sorties concurrentes), les tables de Kaplan-Meier et les diagnostics sont restés identiques ;

l'exposition centrale, les compléments initiaux, l'exposition initiale, les taux, les probabilités et leurs bornes (Poisson et binomial), sommés par cellule dans l'ordre des lignes (somme_par_cellule()), peuvent différer dans les derniers bits. Écart relatif maximal mesuré : $1{,}6 \cdot 10^{-14}$ sur l'entrée de référence (10^5 lignes, plans R1 à R6, six permutations, R 4.3.3, 19/10/2026), atteint par la borne supérieure exacte (borne_sup_exacte de l'estimateur binomial, plan R2) ; les taux et l'exposition centrale restent au plus à $5{,}73 \cdot 10^{-15}$. L'écart croît avec le nombre de termes sommés par cellule : sur l'exposition centrale (plan R1, une permutation tirée sous la graine 20261019), $3{,}97 \cdot 10^{-15}$ à 10^5 lignes, $1{,}37 \cdot 10^{-14}$ à 10^6 et $1{,}73 \cdot 10^{-14}$ à $3 \cdot 10^6$. C'est un **constat de mesure, non une borne démontrée** ; le test de permutation de la batterie de reproductibilité (plans R6, R4, R5, R3 et R2, R1 étant couvert par R6 ; deux permutations, un tirage sous la graine 20261019 et l'ordre inverse) le contrôle à la tolérance relative 10^{-12} (constante TOLERANCE_PERMUTATION, classe existante de CLASSES_TOLERANCE ; issue #39) ; il vérifie l'invariance à l'ordre des lignes, non la précision des sommes, que garde la comparaison aux références de non-régression dans l'ordre reçu (classe 10^{-14}) : un écart dépendant de l'ordre inférieur à 10^{-12} n'y est pas détecté ; les grandeurs qui ne sont pas des doubles, les doubles de classe exacte, les tables de Kaplan-Meier et les diagnostics de l'étape brut y sont comparés par identical(), les trois empreintes (données, lignes conservées, observations) devant au contraire différer ; tout double qui n'est dans aucune de ces deux classes (« double non classé ») fait échouer le test ;

— **ordre des colonnes** : par nom de rôle, en ordre d'octets ; le nom de colonne n'entre pas dans le texte (il est dans fiche$roles) : renommer une colonne ne change pas l'empreinte ;

— **structure** : une première ligne tablexp canonique 2<TAB><nombre de lignes><TAB><nombre de colonnes>, puis, pour chaque colonne, une ligne d'en-tête #<rôle><TAB><type> suivie d'une valeur par ligne ; lignes terminées par LF, aucun CR, fichier écrit octet par octet ;

— **valeurs** : double écrit en **octets IEEE 754** (représentation 2, issue #35 ; ADR 0011, point 2) : une valeur finie x s'écrit en seize chiffres hexadécimaux minuscules, les huit octets de $x + 0$ dans l'ordre petit-boutiste (writeBin(x + 0, raw(), size = 8, endian = "little") ; l'ajout de 0 normalise le zéro négatif), par exemple 666666666666ee3f pour 0,95, 000000000000f03f pour 1 et 0100000000000000 pour le plus petit sous-normal ; une valeur non finie s'écrit par un jeton, NA, NaN, Inf ou -Inf, **lu sur la valeur reçue**, jamais sur $x + 0$, et non par ses octets : ceux d'un NA ou d'un NaN ne sont pas stables d'une opération à l'autre (NA_real_ et NA_real_ + 1 n'ont pas les mêmes octets sous x86-64), et l'aide de R ne garantit pas qu'une opération préserve la distinction entre NA et NaN (ADR 0011, point 2.2, et seconde annotation du 19/10/2026) ; un double muni d'un attribut de dimension s'écrit comme le vecteur nu ; integer en %d ; logical en TRUE, FALSE, NA ; Date en ISO 8601, l'année toujours sur quatre chiffres (sprintf("%04d-%02d-%02d", ...) sur les composantes de la date, composantes_date() ; format.Date écrivait 999-01-01 sans zéro initial), et une date manquante par un jeton lu sur la valeur reçue, NA ou NaN, sans les confondre, comme pour un double (ADR 0011, point 4, et seconde annotation du 19/10/2026, point (d) ; issue #19) ; l'écrivain refuse, en garde-fou des contrôles de l'étape donnees, une date hors du domaine (date_hors_domaine, ici pour la seule colonne écrite, son rôle et ses lignes dans details) et une date non entière (date_non_entiere : 100 et 100,5 jours s'écriraient de même) ; chaînes et facteurs (convertis en chaînes, les niveaux n'entrent pas) : octets UTF-8 entre guillemets droits, avec cinq échappements (\\, \n, \r, \t, \"), NA sans guillemets pour une valeur manquante ;

— **codage** : les chaînes sont converties en UTF-8 puis traitées en octets ; le moteur refuse une chaîne au codage bytes (chaine_codage_bytes), une chaîne qui n'est pas de l'UTF-8 valide (chaine_non_utf8) et, dans une session dont la locale n'est ni UTF-8 ni latin-1, une chaîne non ASCII non marquée, dont le codage est indéterminé (chaine_codage_indetermine). Sous ces règles, l'empreinte ne dépend pas des paramètres régionaux.

Deux formes, un seul écrivain (ADR 0011, point 3 ; décision M64, Q1). `ecrire_valeurs()` écrit les valeurs d'une colonne sous deux formes, qui ne diffèrent que pour les doubles : la forme « empreinte » ci-dessus, seule employée par `texte_canonique()` et par le texte empreinté du résultat (section 15.3), et la forme « texte », pour les textes qu'un relecteur lit et qu'on relit (références de non-régression, section 4.5 ; texte du plan et de la fiche, section 15.4), où un double fini s'écrit `sprintf("%.17g", x + 0)` et un non fini comme R l'écrit (`NA`, `NaN`, `Inf`, `-Inf`), le jeton étant lu, là aussi, sur la valeur reçue. Dix-sept chiffres significatifs suffisent à désigner un double sans ambiguïté, quinze non ; encore faut-il que l'écriture soit correctement arrondie et la relecture exacte. Sous la bibliothèque C de GNU (glibc), l'arrondi correct de l'écriture et la relecture exacte sont des *constats de mesure* (ADR 0011, points 2.4 et 3.2 ; relecture exacte constatée sur 1 399 465 doubles), la relecture étant appuyée sur un argument d'ordre de grandeur, non une preuve. Sous Windows, ni l'un ni l'autre ne tient toujours (constat sur le poste du mainteneur, ADR 0011, point 3.2). Les entiers, logiques, chaînes, facteurs et dates s'écrivent de même dans les deux formes. Une forme inconnue est refusée (`forme_ecriture_inconnue`).

Pourquoi des octets, et non `%.17g`, dans l'empreinte. L'écriture décimale dépend de la bibliothèque C du système : sous Windows, elle n'arrondit pas toujours correctement le dix-septième chiffre (constat sur le poste du mainteneur, ADR 0011, « Contexte », mesure 1), si bien qu'à données identiques au bit près l'empreinte d'une base portant une colonne double différerait d'un poste à l'autre. Les octets ne dépendent que de la norme IEC 60559 (IEEE 754), que R exige de toute plateforme (aide de `double`), et de l'ordre des octets, fixé. Deux autres écritures ont été écartées (ADR 0011, point 2.6) : `%a`, dont l'aide de `sprintf` dit le format non défini de façon unique, et le plus court texte décimal relu à l'identique, parce que la lecture décimale de R n'est pas correctement arrondie. **Statut** : écriture exacte par construction (norme IEC 60559 exigée par R, ordre des octets fixé, aucune mise en forme par la bibliothèque C), recoupée sous glibc par un juge indépendant (Python, `struct.pack`, section 15.5) ; sous Windows et macOS, **non mesurée** (tests sous Windows ajournés, décision M61, issue #49 ; ADR 0011, point 1). L'entrée de référence des tests ne porte aucune colonne double : ses colonnes s'écrivent à l'identique sous les représentations 1 et 2, et seul l'en-tête change son empreinte.

Le libellé de type de l'en-tête est pris dans une **liste fermée de six chaînes**, par des tests faits dans l'ordre du tableau 26 (`ecrire_valeurs()`) ; le premier qui réussit fixe le libellé. Un type hors de la liste est refusé (`type_non_canonique`), sans conversion. C'est une seconde barrière : le contrôle des types des rôles (`type_incompatible`, section 7.2) refuse déjà, avant le calcul de l'empreinte, toute classe hors du catalogue ; aucun rôle n'admet aujourd'hui une colonne logique, si bien que le libellé `logical` n'est pas produit par `construire_table()`.

Table 26 : Libellés de type de la représentation canonique, dans l'ordre des tests.

Ordre	Libellé	Test
1	`factor`	facteur, ordonné ou non
2	`Date`	classe exactement `Date` (une classe `Date` augmentée échoue et tombe au test 3) ; valeur hors de [0001-01-01, 9999-12-31] ou non entière refusée (`date_hors_domaine`, `date_non_entiere`)
3	refus	toute autre classe (`POSIXct`, `difftime`, `AsIs`…)
4	`character`	chaîne
5	`integer`	entier
6	`double`	double
7	`logical`	logique
8	refus	tout autre type de base (liste, complexe, octets bruts…)

Pourquoi `factor` reste distinct de `character`. Les valeurs s'écrivent de la même façon ; seul le libellé distingue une colonne facteur d'une colonne de chaînes aux mêmes valeurs. L'empreinte identifiant l'entrée, deux `data.frame` dont une colonne n'a pas la même classe sont deux entrées différentes, que la fiche doit permettre de distinguer. L'empreinte ne dépend pas, en revanche, des niveaux d'un facteur ; c'est pourquoi aucun résultat ne doit en dépendre (clés de segmentation, section 8.2).

Règle durable. Changer la représentation canonique (libellé, ordre des tests, format d'une valeur) change l'empreinte des données de toutes les fiches, parce que la première ligne du texte empreinté porte la version : c'est un changement de résultat, qui appelle l'incrément de la version de la représentation (`ENTETE_CANONIQUE`, `tablexp canonique N`, portée par `fiche$empreintes$representation`), au plus une fois par branche, et le visa (ADR 0011, point 10). Une empreinte nomme ainsi sa définition. Cette version est distincte de `version_fiche`, qui numérote la structure de la fiche (ensemble des champs et types canoniques). La représentation est passée de 1 à 2 avec l'issue #35 (doubles en octets) : l'empreinte des données des six plans de référence a changé, l'entrée restant la même, ce qu'a prouvé la garde « entrée différente » (section 4.5, « Règles durables »).

15.3 Empreintes du résultat et du code

La fiche porte, outre l'empreinte des données, une empreinte du résultat et une empreinte du code (E3.6 ; ADR 0005, point 6), calculées depuis l'issue #67 (ADR 0011, point 11, et cinquième à huitième annotations du 20/10/2026 ; décisions M70, M71 et M72). Comme celle des données, chacune est un MD5 (`ALGORITHME_EMPREINTE`) d'octets écrits en binaire dans un fichier temporaire, supprimé à la sortie (`md5_octets()`), et non de `serialize()`, dont l'en-tête porte la version de R.

Empreinte du résultat

— **Objet** : identifier conjointement l'entrée, dans ses colonnes de rôle (par `fiche$empreintes$donnees`, qui est dans le périmètre), la déclaration (`fiche$plan`, `fiche$roles`) et le résultat (`table`, `etapes`, `avertissements`, reste de la fiche).

— **Texte empreinté** : le texte de l'objet `construction` écrit par `texte_objet()` sous la grammaire de ligne du texte de la fiche (section 15.4 : `chemin<TAB>type<TAB>longueur<TAB>valeurs`, lignes `@noms`, `@longueur`, `@colonnes` et `@lignes`, segments `[i]`, profondeur au plus `PROFONDEUR_MAX_TEXTE`), les valeurs en forme « empreinte » (doubles en octets IEEE 754, jetons des non-finis lus sur la valeur reçue ; section 15.2), précédé de la première ligne `tablexp resultat 1` (`REPRESENTATION_RESULTAT`) et **sans ligne d'en-tête** `#`. Ce texte n'est jamais relu : il n'existe que pour être haché.

— **Version** : la première ligne numérote la définition du texte empreinté (grammaire de ligne, forme des valeurs, périmètre) ; elle est consignée dans `fiche$empreintes$representation_resultat`, avant `resultat`, distincte de `representation`, qui numérote la représentation des données, et de `version_fiche`, qui numérote la structure de la fiche (ADR 0011, point 10, et cinquième annotation du 20/10/2026, (a)). Changer cette définition change l'empreinte du résultat de toutes les fiches et suit la règle durable de la représentation canonique (section 15.2 : incrément de la version, au plus une fois par branche, et visa).

— **Périmètre** : l'objet entier **moins** les chemins de `CHEMINS_HORS_RESULTAT`, liste fermée déclarée dans le code : `execution`, qui change d'une exécution à l'autre ; `fiche$versions` et `fiche$plateforme`, qui décrivent l'environnement et non le calcul ; `fiche$empreintes$resultat`, l'empreinte elle-même ; `fiche$empreintes$code`, qui ne porte pas sur le résultat, et `fiche$empreintes$statut_resultat_code`, qui décrit les deux empreintes et change avec elles ; enfin `etapes$donnees$resultat$lignes_conservees` (décision M71, point 1). Cette dernière est la copie de l'entrée privée des lignes exclues, avec toutes ses colonnes, sa classe, ses attributs et ses

noms de lignes : elle est entièrement déterminée par l'entrée, identifiée, dans ses colonnes de rôle, par l'empreinte des données (`fiche$empreintes$donnees`), et par `numeros_conserves`, qui reste dans le périmètre, et ce qu'elle porte en plus (colonnes hors rôle, noms de lignes, classe, attributs) n'est lu par aucun module, le moteur ne lisant l'entrée conservée que colonne de rôle par colonne de rôle. Ce surplus n'est couvert par aucune empreinte : celle des données ne porte que sur les colonnes de rôle (décision M71, point 1 ; ADR 0007, point 9.3). La garder dans le périmètre ferait échouer `construire_table()` sur des entrées valides que la grammaire ne sait pas écrire : noms de lignes non automatiques dès qu'une exclusion retire une ligne, colonne hors rôle `POSIXct`, classe `tbl_df`, attribut quelconque, dont l'attribut `simulation` d'un jeu simulé. Options écartées (décision M71) : la lecture stricte du périmètre, qui refuserait ces entrées ; une copie réduite aux colonnes de rôle, qui changerait le contrat « objet résultat » (ADR 0007, point 6.3) pour doubler une entrée déjà identifiée ; une forme « empreinte » tolérante aux classes et aux attributs inconnus, qui les ignorerait en silence ; le retrait de la copie, changement de contrat hors de la branche. L'exclusion de l'attribut `simulation` par son nom, d'abord retenue (décision M70, point 2), est devenue sans objet, l'attribut ne se trouvant qu'à ce chemin.

— **Classes et attributs** : la classe `construction` de la racine est vérifiée et non écrite (la première ligne en tient lieu) ; dans le périmètre, tout attribut hors de ceux de la grammaire et toute valeur sans écriture dans le texte (par exemple un `POSIXct`) arrêtent le calcul par une **erreur de programmation**, qui nomme le chemin : le refus de l'écrivain est converti, rien n'est ignoré en silence. Aucune entrée valide connue n'y conduit : c'est un *constat* sur les témoins de la batterie unitaire (attributs de colonne, facteurs munis de `contrasts` ou de sous-classe, colonnes d'identifiant nommées, dans les deux unités), non une propriété démontrée. Il repose sur la normalisation de la table des observations par l'étape `donnees` (section 7.4, « Copie des colonnes dans la table des observations ») ; une colonne recopiée sans normalisation le contredirait, comme l'a montré la régression des noms de lignes corrigée par l'issue #67, survenue alors que cette normalisation existait déjà.

— **Facteurs** (décision M71, point 2) : un facteur est admis en forme « empreinte », écrit par ses libellés sous le type `factor`, comme dans l'empreinte des données ; il reste refusé en forme « texte » (grammaire 1, section 15.4). Seul est admis un facteur de classe exactement `factor` ou `c("ordered", "factor")` et dont les attributs sont parmi `names`, `levels` et `class` ; tout autre facteur, sous-classe comprise (par exemple `c("factor", "x")`), est refusé (`serialisation_impossible`, converti en erreur de programmation). Aucune construction valide n'est ainsi refusée, par construction : les facteurs de la table des observations sont reconstruits par `categorie`, qui ne produit que ces deux classes (section 7.4). Les niveaux ne sont pas écrits, et le libellé de type est `factor` pour un facteur ordonné ou non (tableau 26) : deux facteurs aux mêmes libellés et aux niveaux différents, ou un `factor` et un `ordered` aux mêmes libellés, ont donc la même empreinte. Ce choix vaut parce qu'aucun résultat ne dépend des niveaux (clés de cellule converties en chaînes, section 8.2) : sa **condition de validité** est le test d'invariance aux niveaux de `tests/unitaires/test_exposition.R` et le témoin de l'issue #67 ; s'ils tombaient, le choix serait à rouvrir.

— **Calcul** : `construire_table()` assemble l'objet (`fiche$empreintes$resultat` y vaut d'abord `NA`, posé par `empreintes_construction()`), calcule `empreinte_resultat()` sur cet objet, range la valeur dans `fiche$empreintes$resultat`, puis ajoute `execution` (section 6.2). Le coût supplémentaire, dû surtout à l'écriture de la table des observations, a été mesuré à 10^6 lignes et admis pour la branche (décision M71, point 5 ; E1.3 ; mesures consignées par l'ADR 0011, septième annotation du 20/10/2026, (f), et par M71) ; son optimisation, qui devait garder le même texte empreinté, est l'objet de l'issue #71. Celle-ci a réécrit l'écriture des entiers et des dates sans une chaîne par ligne, à texte empreinté inchangé octet pour octet, identité *constatée* sur les cas vérifiés, non démontrée (section 15.5, « Écriture des entiers et des dates »). Le gain est un constat de mesure à 10^6 lignes, hors

plateforme de référence, consigné avec le commit `cf24e73` ; la cible de l'issue, un temps d'empreinte du résultat au plus égal à celui de l'empreinte des données (`empreinte_donnees()`), n'est pas atteinte, et le mainteneur a accepté ce gain, cible manquée consignée (21/10/2026).

— **Statut** : écriture exacte (octets fixés, MD5 d'octets), mais valeur **dépendante de la plateforme** : à entrée, plan et code identiques, un double du résultat peut différer au dernier bit d'une machine à l'autre, et l'empreinte change. *Constat de mesure* (20/10/2026) sur un hôte hors de la plateforme de référence (Ubuntu 24.04.5, glibc 2.39, processeur Intel, BLAS et LAPACK 3.12.0) : R1, R2, R3 et R6 y ont la même empreinte du résultat que sur la plateforme de référence, R4 et R5 une autre, parce qu'un élément de `probabilite_borne_sup_asymptotique` y diffère d'un ulp. Cette grandeur est calculée par `-expm1(-borne)` (`calculer_taux_bruts()`), dont l'argument est identique au bit près des deux côtés : l'écart naît dans `expm1()`, fonction de la bibliothèque mathématique du système (`libm` de la glibc ; sur la plateforme de référence, sous Ubuntu 22.04.5, glibc 2.35 : *constat de mesure*, relevé par le script de plateforme dans le journal du job « Tests (R 4.3.3) » de la CI du 20/10/2026, sur l'image de référence, de même empreinte que celle de l'en-tête des références ; issue #72 ; les en-têtes des références la portent depuis leur régénération du 24/10/2026, sous « glibc (libm) » et sous `fiche$plateforme/bibliotheque_c`), et non dans LAPACK ; la mesure ne sépare pas la version de la glibc de la variante qu'elle choisit selon le processeur (ADR 0008, seconde annotation du 19/10/2026). De plus, la forme « empreinte » distingue `NA` de `NaN`, alors que R ne garantit pas lequel sort d'une opération sur `NA` (ADR 0011, point 2.2). L'empreinte du résultat est donc un **témoin d'identité sur la même machine seulement** (E3.6, E4.6 ; ADR 0011, point 11.2), **jamais un critère de rejeu chez un tiers**, dont le critère reste la comparaison à tolérance par classes (section 15.4, « Rejeu par un tiers »). La fiche le dit dans `statut_resultat_code` (ci-dessous).

Empreinte du code

— **Objet** : identifier le **texte source** du moteur effectivement installé, modifications non commitées comprises ; non son comportement numérique, qui dépend aussi de R et du BLAS, consignés à part (`fiche$versions`, `fiche$plateforme`), et de la `libm`, dont la fiche ne consigne la version que par `fiche$plateforme$bibliotheque_c`, et seulement quand sa valeur commence par « glibc » (section 15.1, « Bibliothèque C » ; issue #77) ; le script de plateforme la consigne aussi (issue #72).

— **Définition** (option (iv), décision M70, point 1) : le MD5 du *relevé des MD5* des fichiers `R/*.R`, et non celui de leur texte concaténé : une ligne `<md5>`, deux espaces, `R/<fichier>`, LF, par fichier, les fichiers triés par octets (ADR 0011, septième annotation du 20/10/2026, (d)). Ce relevé est la sortie de `md5sum`, si bien qu'un tiers recalcule l'empreinte depuis la racine des sources par `LC_ALL=C md5sum R/*.R | md5sum`, la locale `C` fixant l'ordre des fichiers.

— **Calcul** : `EMPREINTE_CODE`, constante de premier niveau de `R/rapport.R`, vaut `empreinte_sources(".")`. Elle est évaluée une seule fois, pendant `R CMD INSTALL`, qui exécute le code de premier niveau depuis la racine des sources et range la valeur dans la base de chargement différé du package ; à l'exécution, `empreintes_construction()` lit cette chaîne et le moteur n'ouvre aucun fichier (E4.4 ; ADR 0011, point 8.1). `empreinte_sources()` n'emploie que des fonctions de base (`read.dcf()`, `list.files()`, `tools::md5sum()`) et aucune constante d'un autre module, les fichiers de `R/` étant évalués par ordre alphabétique à l'installation (section 15.1).

— **Garde** : la racine doit être celle des sources de `tablexp` (`DESCRIPTION` lisible, `Package: tablexp`, `R/rapport.R` présent) ; sinon, ou si un fichier de `R/` est illisible (`tools::md5sum()` rend alors `NA`), l'empreinte vaut `NA`, jamais le MD5 d'un relevé incomplet ou portant « NA » : rien n'est deviné (décision M72). L'installation n'échoue pas, et le statut le dit en toutes lettres (`STATUT_CODE_INTROUVABLE`, ci-dessous) : une installation sur une plateforme non mesurée reste utilisable et le signale. `R CMD`

INSTALL évalue aussi les fichiers .r, .S, .s et .q de R/, que le relevé ne couvre pas : un test vérifie que R/ ne contient que des fichiers *.R.

— **Statut** : objet portable (texte source, fins de ligne LF imposées par .gitattributes) et écriture exacte (MD5 d'octets). Le mécanisme, à savoir que le répertoire courant de R CMD INSTALL est la racine des sources, est un *constat de plateforme* sous Linux et R 4.3.3, non documenté par R, non mesuré sous Windows ni macOS (issue #49) ; il est surveillé par le test d'égalité au recalcul (section 15.5).

— **Options écartées** : (i), un manifeste sous inst/ écrit par un script et vérifié par la CI, donne la même valeur mais identifie le texte *déclaré* au dernier passage du script, non le texte installé : une installation locale d'un arbre modifié sans relancer le script aurait une empreinte périmée sans que rien le dise, et la CI ne le verrait qu'au push ; cette option demande en outre un contrôle de CI de plus. (i) reste le **repli documenté**, à rouvrir si le mécanisme de (iv) se révélait défaillant (garde ou test en échec, plateforme non conforme ; ADR 0011, sixième annotation du 20/10/2026). (ii), la version du package seule, est écartée parce qu'elle n'a pas changé depuis la création du package (0.0.0.9000) alors que R/ a été modifié par de nombreux commits ; (iii), le SHA du commit, parce qu'il est absent à l'exécution dans les jobs R sans git et qu'il ignore les modifications non commitées (décisions M69, point 3, et M70).

Statut des deux empreintes

fiche$empreintes$statut_resultat_code vaut statut_empreintes(EMPREINTE_CODE) : le libellé du résultat (STATUT_EMPREINTE_RESULTAT), le séparateur " ; ", puis celui du code calculé (STATUT_EMPREINTE_CODE) ou, si l'empreinte du code est NA, celui des sources introuvables (STATUT_CODE_INTROUVABLE) ; texte du moteur marqué UTF-8 (décision M71, point 3). Les trois libellés sont :

— « résultat : témoin d'identité sur la même machine seulement (E3.6, E4.6), jamais critère de rejeu chez un tiers » ;

— « code : MD5 du relevé des MD5 des fichiers R/*.R, fait à l'installation (recalcul depuis la racine des sources : LC_ALL=C md5sum R/*.R | md5sum) ; identifie le texte source, non le comportement numérique, qui dépend aussi de R, du BLAS et de la libm » ;

— « code : non calculée, sources introuvables à l'installation ».

La fiche ne dit donc jamais d'une empreinte plus que ce qu'elle établit. Dans les références de non-régression, les deux empreintes sont comparées par leur forme, et leurs valeurs écrites dans l'en-tête (section 4.5).

15.4 Texte du plan et de la fiche

Objet (ADR 0011, points 6, 8, 9 et 12, et troisième annotation du 19/10/2026 ; décision M66 ; issue #50). Le plan s'exporte et s'importe (E5.4) ; la fiche s'écrit, se relit et se rejoue (E3.7). Quatre fonctions exportées du module rapport convertissent un objet en texte et inversement : ecrire_plan(), lire_plan(), ecrire_fiche() et lire_fiche(). Elles rendent et reçoivent des **octets** (raw), jamais un fichier : l'appelant écrit par writeBin(octets, fichier) et lit par readBin(fichier, "raw", file.size(fichier)) (E4.4 ; ADR 0011, point 8). Des octets, et non une chaîne, parce que l'écriture en mode texte produirait des fins de ligne CRLF sous Windows (hypothèse non mesurée) et que la lecture ligne à ligne de R réencode selon la locale : les octets sont le contrat, et un fichier altéré, par exemple par un éditeur qui ajoute des CR, est refusé avec un message qui nomme l'altération plutôt que corrigé en silence. Ce format remplace le report du point 12 de l'ADR 0007, dont il garde les cinq

contraintes : texte lisible par un relecteur, aller-retour identique, nombres compris, aucune évaluation de code à l'import, R de base, version portée par le texte.

Grammaire du texte (grammaire 1)

— **forme** : UTF-8, lignes terminées par LF, LF final, ni CR ni marque d'ordre des octets ;

— **première ligne** : `tablexp plan 1` (`PREMIERE_LIGNE_PLAN`) ou `tablexp fiche 1` (`PREMIERE_LIGNE_FICHE`), qui nomme la nature et la version de la grammaire. La version est portée par le texte seul, jamais par un champ de l'objet, et le lecteur refuse toute autre première ligne, sans migration (ADR 0011, point 10) ;

— **en-tête** : des lignes `# <clé> : <valeur>`, **hors aller-retour**, que le lecteur ignore. Elles rappellent au relecteur les conventions (`CONVENTIONS_TEXTE` : grammaire, doubles, dates, chaînes, délais, codage). La fiche y ajoute `version_fiche`, la représentation canonique de l'empreinte des données, les blocs de lecture (`# lecture`) et la procédure de rejeu (`# rejeu`), décrits plus bas ;

— **corps** : une ligne par feuille, `chemin<TAB>type<TAB>longueur<TAB>valeurs`, dans l'ordre de l'objet (parcours en profondeur), jamais trié ; le chemin joint les segments par `/` ; un élément sans nom, ou dont le nom ne peut pas être un segment (vide, `NA`, en double, contenant `/`, une tabulation ou une fin de ligne, ou commençant par `@`, `#` ou `[`), a pour segment `[i]`, son rang, le nom restant porté par la ligne `@noms`, si bien que tout nom fait l'aller-retour ;

— **structure** : chaque conteneur déclare son effectif. Une liste nommée a une ligne `chemin/@noms`, puis ses éléments ; une liste sans noms a une ligne `chemin/@longueur` (`integer`, une valeur n), puis exactement ses éléments `chemin/[1]` à `chemin/[`n`]` ; un `data.frame` a ses lignes `chemin/@colonnes` et `chemin/@lignes`, puis ses colonnes (noms de lignes automatiques seulement) ; un vecteur atomique nommé, de longueur au moins 1, a sa ligne `chemin/@noms`, puis sa ligne de valeurs ; `NULL` s'écrit `chemin<TAB>NULL<TAB>0`. Conséquence : **tout texte écrit par `ecrire_plan()` ou `ecrire_fiche()` dont on retire au moins une ligne du corps, en le coupant à une frontière de ligne, est refusé à la lecture**, faute de quoi un fichier tronqué serait relu comme un objet plus court. Un vecteur atomique nommé de longueur 0 est pour cela refusé à l'écriture : sa ligne `@noms` seule est aussi l'écriture complète d'une liste nommée vide, si bien que son texte privé de la ligne de valeurs serait relu sans refus (ADR 0011, seconde annotation du 20/10/2026 ; aucun plan valide ni aucune fiche du package n'en contient). **Limite admise** : un texte édité à la main qui porte un tel vecteur est relu sans refus, si bien que le lecteur admet un objet que l'écrivain refuse ;

— **types** : `double`, `integer`, `logical`, `character`, `Date`, `NULL` (`TYPES_TEXTE`) ; valeurs écrites par `ecrire_valeurs()` en forme « texte » (section 15.2 : doubles en `%.17g`, dates ISO à quatre chiffres, chaînes entre guillemets avec cinq échappements, `NA` sans guillemets, distinct de `"NA"`). Un facteur est hors de la grammaire 1 : sans ses niveaux, il n'a pas d'aller-retour identique, et aucun plan valide n'en porte ; seul le texte empreinté du résultat, qui n'est jamais relu, l'admet par ses libellés (`texte_objet()` en forme `"empreinte"`, section 15.3). Une `Date` à stockage entier s'écrit comme la même date à stockage double et se relit en double ;

— **profondeur** : nombre de segments du chemin, sans le dernier s'il commence par `@` ; au plus `PROFONDEUR_MAX_TEXTE` (32), contrôlée à l'écriture et, à la lecture, avant toute reconstruction, si bien qu'aucun texte ne fait déborder la pile de R (débordement constaté entre 800 et 1 000 niveaux sur une pile de 8 Mo, profondeur des fiches de R1 à R6 égale à 4 : ADR 0011, troisième annotation, point (g)).

La forme de ligne et l'écrivain des valeurs sont ceux des références de non-régression (section 4.5) ; les **formats restent distincts**, et la grammaire 1 s'en écarte sur sept points (ADR 0011, troisième annotation,

point (f)) : liste sans noms (@longueur ici, @noms vide là) ; noms qui ne peuvent pas être des segments (écrits [i] ici, refusés là) ; facteur (exclu ici, admis là) ; lecture des doubles (lecteur de forme ici, stricte là) ; lignes propres aux références (@classe, @empreinte) ; en-tête # ; profondeur bornée.

Lecture sans évaluation

Le lecteur découpe les octets sur LF, puis chaque ligne sur la tabulation, et convertit par as.numeric(), as.integer() et as.Date() sur une forme contrôlée, plus le désechappement des chaînes ; il n'emploie **jamais** parse(), eval(), source() ni dget() (ADR 0007, point 2.3). Un texte qui contient du code R est relu comme une chaîne ou refusé, jamais évalué. Les types autres que double exigent leur écriture canonique ; une date relue doit, réécrite, redonner son texte. Pour les **doubles**, le lecteur est un *lecteur de forme* (décision M64, Q3) : tout décimal que as.numeric() lit en un double fini est admis, ainsi que les jetons NA, NaN, Inf et -Inf, et la valeur retenue est le double lu : un plan édité à la main peut écrire 0.95 au lieu de 0.94999999999999996. La valeur de référence pour un tiers est l'écriture %.17g que l'export réécrit. Un décimal non nul relu 0 (sous-dépassement, par exemple 1e-400) est **refusé** (décision M66) : l'erreur relative serait de 100 %, et un délai de déclaration écrit 1e-400 passerait la validation comme 0. **Statut** de l'aller-retour : exact pour tous les types sauf double ; pour un double, celui de la forme « texte » (section 15.2 : écriture correctement arrondie et relecture exacte constatée sous glibc, non démontrée). Un texte écrit sous Windows, dont le dix-septième chiffre peut différer, se relit en le même double dans le cas testé : c'est un *constat de mesure*, que contredisent d'autres cas : sous Windows, sur 551 008 textes %.17g, 5 ont été relus en un autre double, tous d'exposant supérieur à 266 (constat sur le poste du mainteneur, ADR 0011, point 3.2).

Table 27 : Refus de la lecture et de l'écriture du texte (module rapport ; numéro de la ligne fautive dans details$ligne, chemin fautif dans details$chemin).

Motif	Condition
texte_non_octets	texte reçu autrement qu'en octets (raw)
texte_version_inconnue	texte vide, ou première ligne autre que celle attendue (nature ou version de grammaire)
texte_marque_ordre_octets	marque d'ordre des octets en tête
texte_retour_chariot	octet CR (fins de ligne CRLF ou CR)
texte_octet_nul	octet nul
texte_sans_lf_final	dernière ligne sans LF
texte_utf8_invalide	ligne qui n'est pas de l'UTF-8 valide
texte_ligne_mal_formee	texte sans corps ; ligne hors de la forme (champ vide, longueur illisible, ligne d'en-tête dans le corps) ; ligne @longueur, @noms, @colonnes ou @lignes de type ou de valeur invalide ; NULL nommé ; colonne non atomique
texte_type_inconnu	type hors de TYPES_TEXTE (factor compris)
texte_longueur_incoherente	nombre de valeurs différent de la longueur déclarée ; NULL de longueur non nulle ; @longueur supérieur au nombre de lignes restantes (contrôlé avant toute allocation) ; noms et valeurs en nombres différents ; colonne d'une autre longueur que @lignes
texte_chemin_en_double	chemin présent deux fois
texte_profondeur_excessive	profondeur au-delà de PROFONDEUR_MAX_TEXTE
texte_chemin_inattendu	chemin autre que celui qu'attend la structure : élément manquant ou en trop, ligne après la fin de l'objet, corps qui ne commence ni par @noms ni par @longueur

Motif	Condition
texte_valeur_invalide	valeur hors de l'écriture de son type : chaîne sans guillemets droits ou à échappement inconnu ; double qui n'est ni un décimal lu en un double fini ni un jeton ; entier non canonique ou hors de l'étendue des entiers ; logique autre que TRUE, FALSE, NA ; date hors de la forme AAAA-MM-JJ, inexistante, hors du domaine ou non canonique
texte_double_sous_depassement	décimal non nul relu 0
serialisation_impossible	à l'écriture : racine qui n'est pas une liste ; facteur (en forme « empreinte » du texte empreinté du résultat, seulement un facteur de classe ou d'attribut non admis, section 15.3) ; classe ou attribut hors de la grammaire ; data.frame classé ou aux noms de lignes non automatiques ; colonne non atomique ; Date de stockage autre que double ou integer ; vecteur atomique nommé de longueur 0 ; profondeur au-delà de PROFONDEUR_MAX_TEXTE ; objet construction entier passé à ecrire_fiche() au lieu de sa fiche

L'écriture peut aussi lever les refus de l'écrivain des valeurs (codage d'une chaîne, date hors du domaine ; section 15.2).

Plan importé, fiche relue, rejeu

ecrire_plan() écrit le plan tel qu'il est reçu, sans validation ni canonisation, et lire_plan() rend un plan **non validé** : le module rapport ne peut pas appeler R/construire_table.R, où vit valider_plan() (graphe des modules, section 1.3). La validation se fait chez l'appelant, par valider_plan(), ou à l'entrée de construire_table(), qui l'appelle toujours : un plan importé reçoit exactement les refus d'un plan construit en mémoire (ADR 0011, point 9.2). Pour un plan canonique, lire_plan(ecrire_plan(p)) est identical() à p ; pour un autre plan, au stockage des dates près. lire_fiche() ne contrôle pas version_fiche : ce contrôle appartient au rejeu, et le rejeu outillé rejouer_fiche() refuse une version non admise sous le motif version_fiche_non_admise (section 6.4 ; ADR 0011, troisième annotation du 19/10/2026, point (i), troisième annotation du 20/10/2026 et annotation du 24/10/2026 ; ADR 0012, point 3.5).

Rejeu par un tiers (E3.7 et sa note de lecture du 19/10/2026 ; ADR 0011, point 12), rappelé par la ligne # rejeu de la fiche écrite : le relecteur vérifie version_fiche, la représentation canonique et l'empreinte des données de la fiche ; il exécute construire_table(donnees, fiche$roles, fiche$plan, fiche$provenance) sur le plan relu du texte ; il compare les grandeurs à tolérance par classes (section 4.5) sur une autre machine, au bit près sur la même (E4.6). L'empreinte du résultat ne sert de témoin que sur la même machine (section 15.3) : sur une autre machine, une empreinte du résultat différente n'est pas un échec du rejeu. Le rejeu outillé, rejouer_fiche(), fait les vérifications, l'exécution et la comparaison des témoins sur la même machine ; la comparaison à tolérance par classes sur une autre machine n'est pas outillée (section 6.4).

Blocs de lecture. La ligne # lecture de la fiche écrite range ses champs en trois blocs (BLOCS_LECTURE_FICHE), qui suivent les trois questions d'E3.7 : ce qui a été *calculé*, avec quelles *hypothèses*, de quoi *rejouer*. Un test vérifie que les blocs partitionnent les champs de la fiche (aucun champ manquant, en double ni étranger), si bien qu'un champ ajouté sans bloc fait échouer la batterie. L'affectation, validée par actuary le 20/10/2026 (ADR 0011, point 6.5), est la suivante, les champs de chaque bloc étant dans l'ordre de la fiche (ORDRE_CHAMPS_FICHE, issue #64) : *calculé* : plan, transition_code, fin_effective, unite_observation, recette, convention, unite_exposition, par_annee ; *hypothèses* : exclusions, nombre_lignes_exclues, avertissement_unite, hypothese_censure,

hypotheses_brut, avertissements ; *rejeu* : version_fiche, versions, plateforme, roles, provenance, provenance_mention, empreintes, graine, graine_mention, generateur. Le champ empreintes est rangé en « rejeu » à cause de l'empreinte des données, préalable du rejeu ; il porte aussi l'empreinte du résultat, simple témoin d'identité sur la même machine (statut_resultat_code, section 15.3), et celle du code. Un champ absent de la fiche (hypotheses_brut sans étape brut) est omis de la ligne.

15.5 Vérifications

tests/unitaires/test_construire_table.R compare l'empreinte d'un petit data.frame, colonnes facteur, chaîne et double comprises, à une valeur calculée hors de R (texte écrit par printf, octets des doubles par struct.pack de Python, empreinte par md5sum de GNU coreutils). Il vérifie l'écrivain des valeurs dans ses deux formes : les vecteurs de test de l'ADR 0011 (octets calculés hors de R), sous-normaux, plus petit normal et maximum, jetons des non-finis lus sur la valeur reçue quels que soient leurs octets, zéro négatif, attribut de dimension, mélange de valeurs finies et de jetons, longueurs limites, types autres que double identiques dans les deux formes, refus ; et un échantillon de 10^5 doubles tirés sous graine, dont le MD5 du texte doit être celui que produit le juge Python, hors de R. Le contrôle tests/controles/ecriture_doubles.R, **hors batterie et hors CI** (durée, dépendance à Python), étend ce recoupement à 10^6 doubles et aux cas limites, ligne à ligne, et sort avec un code non nul sur un écart ou si Python est absent ; il ne s'exécute pas sous Windows, où le texte %.17g qu'il transmet au juge n'est pas toujours correctement arrondi. Les dates sont vérifiées sur quatre chiffres d'année (0001, 0019, 0999, 9999), sur tous les jours de 1930 à 2024 (identiques à l'écriture de format.Date, si bien que les références, dont les dates sont dans cet intervalle, n'ont pas changé), avec les jetons NA et NaN distincts (deux empreintes calculées hors de R) et les refus date_hors_domaine et date_non_entiere ; construire_table() traite une date NaN comme un manquant.

Écriture des chaînes (octets_chaines(), issue #22, point 1). Les chaînes sont écrites par blocs de lignes : les corps échappés, marqués bytes pour être copiés sans traduction en toute locale, sont collés par un paste(), puis convertis en octets par un seul charToRaw() par bloc, au lieu d'une liste de trois vecteurs d'octets par ligne. Une valeur manquante prend le corps « un guillemet » : sa ligne s'écrit de trois guillemets suivis d'un LF, puis est ramenée à NA et LF par un gsub(). Ce marqueur ne peut pas apparaître dans une autre ligne : les cinq échappements étant appliqués à la barre oblique inverse d'abord, puis à LF, CR, la tabulation et au guillemet, un corps échappé ne contient ni LF ni guillemet qui ne soit précédé d'une barre oblique inverse ; il ne commence donc jamais par un guillemet et n'en porte jamais deux de suite. Une autre ligne, guillemet ouvrant, corps, guillemet fermant et LF, porte ainsi au plus deux guillemets consécutifs (le dernier caractère du corps et le guillemet fermant, ou les deux guillemets d'un corps vide) devant sa fin de ligne, jamais trois, et le motif, qui se termine par LF, ne peut pas chevaucher deux lignes. Les blocs sont bornés par bornes_blocs() à 2^{30} octets de texte avant ce gsub(), sous la limite de $2^{31} - 1$ octets d'une chaîne R (aide Memory-limits) ; une ligne plus longue que la borne forme seule son bloc et s'écrit directement en octets. La règle d'écriture des chaînes (section 15.2) est inchangée. tests/unitaires/test_construire_table.R compare les octets produits à ceux de l'écriture antérieure, recopiée dans le test comme oracle, sur 17 cas adverses (NA et "NA", chaîne vide, chaînes non ASCII en UTF-8 et en latin-1, cinq échappements, suites de guillemets, répétitions, longueurs 0 et 1, ligne de plus de 5 000 octets) pour 8 bornes de bloc, de 1 à 2^{30} octets, dans la session et dans un sous-processus lancé sous LC_ALL=C ; il vérifie les bornes rendues par bornes_blocs() sur 45 bornes et 3 jeux de corps (partition des lignes dans l'ordre, bloc de plusieurs lignes d'au plus la borne et maximal, ligne trop longue seule dans son bloc et seule marquée). Les statuts sont distincts : l'argument du marqueur *établit*, par raisonnement sur la construction, que la substitution ne porte que sur les lignes des valeurs manquantes ; l'identité des octets à ceux de l'écriture antérieure n'est que *constatée* sur ces cas. Le script tests/mesures/octets_chaines.R, hors batterie et sans seuil, mesure la durée et

la mémoire de cette écriture et de l'empreinte à deux rôles caractère sur un jeu déterministe.

Écriture des entiers et des dates (octets_entiers(), octets_lignes_indices(), issue #71). La règle d'écriture (section 15.2) est inchangée : un integer s'écrit comme sprintf("%d", x), une date en ISO 8601 ; seul l'assemblage des octets change, qui ne passe plus par une chaîne par ligne. octets_entiers() forme une matrice d'octets d'une colonne par valeur et d'une ligne par position (signe, chiffres, LF), le nombre de chiffres étant celui de la plus grande valeur absolue, deux au moins pour le jeton NA ; un chiffre de rang $k > 0$ n'est écrit que si le quotient de la valeur absolue par 10^k est non nul, si bien qu'aucun zéro de tête n'apparaît ; les places inoccupées valent l'octet nul, qui n'apparaît dans aucune écriture et que la lecture de la matrice colonne par colonne retire. octets_lignes_indices() écrit une fois chaque date distincte, puis recopie ses octets, LF compris, aux lignes où elle figure, par sequence() ; les jetons NA et NaN d'une date manquante sont lus comme avant sur la valeur reçue. Dans les deux formes, ecrire_valeurs() emploie ces deux fonctions. tests/unitaires/test_construire_table.R compare les octets produits à ceux de l'écriture antérieure, recopiée dans le test comme oracle : pour les entiers, 14 cas (vecteur vide, NA, zéro, signes, de 1 à 10 chiffres, $\pm(2^{31}-1)$, noms, dimension, constante, 10^5 valeurs pseudo-aléatoires sans générateur), par octets_entiers() et par ecrire_valeurs() dans les deux formes, plus une sortie écrite à la main ; pour octets_lignes_indices(), 8 suites d'indices (vide, texte vide, répétitions, ordre inverse, 10^5 indices) ; pour les dates, NA, NaN, stockage integer, valeurs répétées et 10^5 jours du domaine, dans les deux formes. tests/test_reproductibilite.R vérifie en outre que l'empreinte du résultat de chacun des six plans de référence est égale à celle que donne un oracle, empreinte_resultat() réévaluée avec les écrivains antérieurs des entiers et des dates ; l'oracle remplace une valeur figée, l'empreinte du résultat ne valant que sur une même machine (section 15.3). Les plans de référence ne portent aucun entier négatif (les dates s'écrivent en ISO, sans signe) : le signe des entiers est gardé par les tests unitaires, non par cet oracle. Les statuts sont ceux de l'écriture des chaînes : l'identité des octets à ceux de l'écriture antérieure est *constatée* sur ces cas, non démontrée. Le script tests/mesures/temps_construire_table.R, hors batterie et sans seuil, mesure le temps des deux empreintes (médiane et étendue de mesures répétées).

tests/unitaires/test_serialisation.R vérifie le texte du plan et de la fiche (section 15.4) : forme des octets, première ligne et en-tête hors aller-retour ; aller-retour identical() sur des cas adverses (NA et "NA", NA et NaN, chaînes non ASCII, doubles extrêmes et sous-normaux, dates aux bornes du domaine, listes et vecteurs vides, NULL, noms qui ne peuvent pas être des segments) ; code R en valeur, en nom ou en chemin, jamais évalué (témoin non créé), et absence d'appel à une fonction d'évaluation dans les lecteurs ; chaque refus de la lecture et de l'écriture, avec son numéro de ligne ou son chemin ; lecteur de forme des doubles et refus du sous-dépassement ; plan importé soumis aux mêmes refus qu'un plan en mémoire ; liste sans noms, troncature à chaque frontière de ligne, profondeur 32 admise et 33 refusée ; partition des champs de la fiche par les blocs de lecture ; rejeu identical() hors execution. La batterie de reproductibilité refait l'aller-retour et le rejeu pour les six plans de référence et refuse chacun des préfixes stricts du corps de la fiche écrite de R1. tests/test_reproductibilite.R vérifie que l'empreinte est la même dans des processus R lancés sous LANG=C et sous LANG=C.UTF-8, égale à cette valeur, et que le rejeu de la fiche redonne l'objet. tests/unitaires/test_construire_table.R vérifie aussi chaque cas de refus de la provenance décrit à la section 15.2 (issue #21).

Empreintes du résultat et du code (issue #67 ; section 15.3). tests/unitaires/test_construire_table.R vérifie, pour le résultat : l'identité exacte sur deux constructions, au rejeu de la fiche et au recalcul sur l'objet ; l'insensibilité aux chemins hors du périmètre (execution, fiche$versions, fiche$plateforme, les trois derniers champs de fiche$empreintes, et lignes_conservees munie d'une colonne POSIXct et d'attributs) ; la sensibilité, sur des copies, à un ulp d'un double du périmètre, à la première ligne, à l'empreinte des données, au plan et aux rôles ; l'égalité à une valeur calculée hors de R, sur un petit objet (texte écrit et haché par Python, doubles

par struct.pack), qui exerce les jetons, le zéro négatif, les dates, un facteur, les chaînes échappées, un data.frame, une liste sans noms et l'exclusion des chemins ; l'erreur de programmation sur un attribut inattendu (dont simulation hors de lignes_conservees), une classe de racine autre que construction et une valeur sans écriture ; le facteur, admis en forme « empreinte » et refusé en forme « texte », une colonne de rôle en facteur aux niveaux permutés donnant la même empreinte ; la construction acceptée, avec la même empreinte que sans attribut, quand chaque colonne de rôle porte un attribut label, dans l'unité tête comme dans l'unité contrat, id_tete compris, et quand sexe ou segment est un facteur muni de contrasts ou de sous-classe, dans les deux unités (décisions M71, point 4, et M72). Pour le code : EMPREINTE_CODE n'est pas NA et égale le recalcul fait par la batterie sur R/*.R de la racine qu'elle a installée, en R et, si md5sum et sh sont présents, par LC_ALL=C md5sum R/*.R | md5sum ; R/ ne contient que des fichiers *.R ; empreinte_sources() rend NA sans DESCRIPTION, pour un autre paquet, sans R/rapport.R et pour un fichier illisible (lien cassé), et le recalcul sur une racine conforme ; le statut a le libellé exact, marqué UTF-8, et nomme les sources introuvables si et seulement si l'empreinte du code est NA. tests/test_reproductibilite.R recalcule l'empreinte du résultat dans ses tests d'invariance (plan sans brut, plan sans kaplan_meier) et exerce la comparaison par jeton des références (section 4.5).

15.6 Fonctions du module

Table 28 : Définitions de premier niveau de R/rapport.R.

Nom	Statut	Entrées	Sortie et rôle
VERSION_FICHE	constante	—	version de la fiche (4)
VERSIONS_FICHE_REJEU	constante	—	versions de fiche admises au rejeu outillé : VERSION_FICHE, la version courante seule (section 6.4)
ENTETE_CANONIQUE	constante	—	"tablexp canonique 2", première ligne du texte empreinté et fiche$empreintes$representation (version de la représentation canonique)
ALGORITHME_EMPREINTE	constante	—	"md5"
REPRESENTATION_RESULTAT	constante	—	"tablexp resultat 1", première ligne du texte empreinté du résultat et fiche$empreintes$representation_resultat
STATUT_EMPREINTE_RESULTAT	constante	—	libellé du statut de l'empreinte du résultat (témoin d'identité sur la même machine seulement) ; littéral non ASCII lu par la sonde UTF-8 des batteries
STATUT_EMPREINTE_CODE	constante	—	libellé du statut de l'empreinte du code calculée (relevé des MD5 de R/*.R, commande de recalcul)
STATUT_CODE_INTROUVABLE	constante	—	libellé du statut de l'empreinte du code NA (sources introuvables à l'installation)

Nom	Statut	Entrées	Sortie et rôle
bornes_blocs()	interne	corps échappés (codage bytes), limite en octets	bornes debut et fin des blocs de collage d'octets_chaines() et indicateur seule : découpage glouton dans l'ordre des lignes, une ligne comptant les octets de son corps plus trois (deux guillemets et LF) ; un bloc de plusieurs lignes fait au plus limite octets et la ligne suivante n'y tiendrait pas ; une ligne plus longue que limite forme seule son bloc (seule)
octets_chaines()	interne	chaînes, rôle, limite (facultative, 2^{30} octets, réservée aux tests du découpage)	octets canoniques des chaînes ; refus de codage ; écriture par blocs de lignes (bornes_blocs()), un paste() et un charToRaw() par bloc, une ligne plus longue que limite écrite seule
octets_lignes()	interne	textes ASCII	octets, une valeur par ligne
octets_lignes_indices()	interne	textes ASCII, indices sans NA	octets des lignes textes[i], égaux à ceux d'octets_lignes() sur ces lignes ; chaque texte écrit une fois, ses octets recopiés par indice (sequence()) ; écriture des dates (issue #71)
octets_entiers()	interne	entiers (integer)	octets de sprintf("%d", x), une valeur par ligne, écrits en octets sans chaîne intermédiaire ; écriture des entiers (issue #71)
texte_date()	interne	jours depuis 1970-01-01, entiers, finis, dans le domaine	écriture ISO 8601 AAAA-MM-JJ, année sur quatre chiffres ; partagée par ecrire_valeurs() et le contrôle de canonicité du lecteur (valeurs_texte())
ecrire_valeurs()	interne	colonne, forme, rôle	libellé de type et octets ; forme "empreinte" (doubles en octets IEEE 754) ou "texte" (doubles en %.17g) ; refus forme_ecriture_inconnue, type_non_canonique, et, en garde-fou, date_hors_domaine, date_non_entiere
texte_canonique()	interne	données, rôles, fichier	représentation canonique écrite en binaire
empreinte_donnees()	interne	données, rôles	MD5 de la représentation canonique
controler_provenance()	interne	provenance	refus provenance_invalide
empreintes_construction()	interne	données, rôles, provenance	champ empreintes de la fiche, dans l'ordre de la section 15.1 ; resultat à NA, posé ensuite par construire_table() ; code égal à EMPREINTE_CODE
statut_empreintes()	interne	empreinte du code	statut_resultat_code : libellé du résultat, " ; ", libellé du code calculé ou, si l'empreinte du code est NA, des sources introuvables ; marqué UTF-8

Nom	Statut	Entrées	Sortie et rôle
md5_octets()	interne	octets	MD5 des octets, écrits en binaire dans un fichier temporaire supprimé à la sortie
CHEMINS_HORS_RESULTAT	constante	—	liste fermée des sept chemins hors du périmètre de l'empreinte du résultat (section 15.3)
empreinte_resultat()	interne	objet construction	MD5 du texte de l'objet privé de CHEMINS_HORS_RESULTAT, en forme "empreinte", de première ligne REPRESENTATION_RESULTAT, sans en-tête ; classe de la racine vérifiée ; erreur de programmation sur un attribut ou une valeur sans écriture
empreinte_sources()	interne	racine des sources	MD5 du relevé des MD5 de R/*.R (égal à LC_ALL=C md5sum R/*.R | md5sum) ; NA si la racine n'est pas celle de tablexp ou si un fichier est illisible
EMPREINTE_CODE	constante	—	empreinte du code, empreinte_sources(".") évaluée une fois par R CMD INSTALL depuis la racine des sources ; NA si elles sont introuvables
canoniser_plan()	interne	plan validé, champs, paramètres d'étape, ordres	plan canonique (ADR 0011, point 5.1) : champs dans l'ordre de CHAMPS_PLAN, transition en (depart, arrivee), periode en (debut, fin) à bornes Date stockées en double, délai en double, graine en integer, paramètres d'étape dans l'ordre de PARAMETRES_ETAPES, valeurs de segmentation, exclusions et estimateurs dans l'ordre des vecteurs de ordres (issue #63) ; idempotente ; erreur de programmation sur un stockage non prévu, un nom d'étape ou de paramètre en double, ou, dans un champ ensemble, une valeur hors du vecteur d'ordre, un doublon, un NA, un attribut ou un type autre que character ; appelée une fois par construire_table() après valider_plan()
MENTION_BIBLIOTHEQUE_C_NON_RELEVEE	constante	—	mention « non relevée » de fiche$plateforme$bibliotheque_c, quand la valeur lue est absente, NA, vide ou faite de blancs, ou n'est pas un texte (section 15.1)

Nom	Statut	Entrées	Sortie et rôle
bibliotheque_c()	interne	versions des bibliothèques (par défaut celles de R ; paramètre pour les tests)	élément iconv, tel quel (« unknown » compris), sinon la mention « non relevée » (constante ci-dessus) ; n'identifie la glibc que si la valeur commence par « glibc » (section 15.1)
assembler_fiche()	interne	rôles, plan, recette, étapes, avertissements, empreintes, provenance	fiche de construction : plan canonique, rôles dans l'ordre du catalogue, provenance dans l'ordre (fichier, empreinte, algorithme) ; champ hypotheses_brut si l'étape brut est exécutée ; champs réordonnés par ordonner_fiche() selon ORDRE_CHAMPS_FICHE
ordonner_fiche()	interne	fiche, ordre	fiche réordonnée selon l'ordre, réduit aux champs présents (hypotheses_brut absent sans étape brut) ; erreur de programmation sur un champ présent hors de l'ordre, jamais retiré en silence
ORDRE_CHAMPS_FICHE	constante	—	ordre des champs de la fiche en huit blocs (identification, données, plan, paramètres retenus, exclusions, hypothèses, aléa, avertissements) ; source unique de l'ordre du corps de la fiche écrite
informations_execution()	interne	instant de début	champ execution, hors comparaison
PREMIERE_LIGNE_PLAN	constante	—	"tablexp plan 1", première ligne du texte du plan (nature et version de la grammaire)
PREMIERE_LIGNE_FICHE	constante	—	"tablexp fiche 1", première ligne du texte de la fiche
TYPES_TEXTE	constante	—	types admis d'une ligne du texte : double, integer, logical, character, Date, NULL
PROFONDEUR_MAX_TEXTE	constante	—	profondeur maximale d'une ligne du texte (32), contrôlée à l'écriture et à la lecture
CONVENTIONS_TEXTE	constante	—	conventions d'écriture rappelées en lignes d'en-tête # <clé> : <valeur> du texte, ignorées à la lecture
BLOCS_LECTURE_FICHE	constante	—	partition des champs de la fiche en trois blocs (E3.7) : calculé, hypothèses, rejeu, champs de chaque bloc dans l'ordre de ORDRE_CHAMPS_FICHE ; rappelée par la ligne d'en-tête # lecture
joindre_chemin()	interne	chemin, segment	chemin d'un élément, segments séparés par /

Nom	Statut	Entrées	Sortie et rôle
segments_texte()	interne	noms, longueur	segment de chemin de chaque élément : son nom, ou [i] (rang) si le nom est absent, vide, NA, en double, contient /, une tabulation ou une fin de ligne, ou commence par @, # ou [
texte_objet()	interne	liste, première ligne, en-tête, forme	octets UTF-8 du texte : une ligne par feuille, dans l'ordre de l'objet ; valeurs par ecrire_valeurs() en forme "texte" (défaut : plan, fiche) ou "empreinte" (empreinte_resultat()) ; refus serialisation_impossible (facteur en forme "texte" ; en forme "empreinte", facteur de classe autre qu'exactement factor ou c("ordered", "factor") ou d'attribut autre que names, levels, class ; classe ou attribut non écrits, Date de stockage autre que double ou integer, vecteur atomique nommé de longueur 0, profondeur au-delà de PROFONDEUR_MAX_TEXTE)
valeurs_texte()	interne	type, textes d'une ligne, refus	valeurs relues sans évaluation ; refus texte_valeur_invalide, texte_double_sous_depassement (décimal non nul relu 0)
lire_texte()	interne	octets, première ligne, nature	objet relu sans évaluation ; refus nommés texte_..., numéro de la ligne fautive dans details$ligne
ecrire_plan()	exportée	plan	octets du texte du plan, écrit tel qu'il est reçu, sans validation ni canonisation
lire_plan()	exportée	octets	plan relu, **non validé** : à passer à valider_plan() ou à construire_table()
ecrire_fiche()	exportée	fiche	octets du texte de la fiche (x$fiche, refus serialisation_impossible pour l'objet construction entier) ; l'en-tête rappelle version_fiche, la représentation canonique, les blocs de lecture (# lecture) et la procédure de rejeu (# rejeu)
lire_fiche()	exportée	octets	fiche relue, dont le rejeu est construire_table(donnees, f$roles, f$plan, f$provenance)

PARTIE XII

Module simulation

Simulateur : R/simulation.R

Le dépôt ne contient aucune donnée réelle : tous les jeux sont simulés sous une *loi vraie* connue et une graine (E1.5). Le simulateur est le modèle multi-états exécuté en avant (E2.19 ; ADR 0003, point 9) : il produit des lignes individuelles portant les rôles du catalogue, sur lesquelles le moteur est vérifié. Il reproduit le même modèle que le moteur, pas le même code : il n'appelle que `donnees` et `outils`, et aucun module du moteur ne l'appelle (section 1.3). La version de cette branche est *minimale* : deux transitions concurrentes, vivant → décédé et en cours → racheté, une tête par contrat, troncature à gauche et censure par la période. Les autres formes de données (agrégats, stocks successifs), les têtes multi-contrats et l'injection de défauts paramétrés viendront avec une branche ultérieure (`docs/feuille-de-route.md`, § 2). Son point d'entrée est `simuler_portefeuille(loi, portefeuille, periode, graine)`, exporté (ADR 0007, point 3.2). Le contenu de fond vient de la spécification d'`actuary` (issue #11, partie A.2) et de son erratum du 17/10/2026.

16.1 Loi vraie et tables synthétiques

La loi vraie est une liste de deux lois, `deces` et `rachat` ; chacune associe une table de référence (classe `table_reference`, section 7.5) et un facteur par sexe, `c(H = , F = )`, réel fini ≥ 0. La loi vraie et chacune de ses deux lois sont des **listes nues**, dont le seul attribut admis est celui des noms, d'exactement autant d'éléments que de noms attendus (`deces` et `rachat` ; `table` et `facteurs`), dans un ordre quelconque : un `data.frame`, un objet de classe, une liste munie d'un autre attribut ou un élément de plus, même nommé `NA`, sont refusés (`loi_invalide` ; `liste_aux_noms()`, section 2.3 ; issue #44). Pour un sexe s, la loi effective est

$$q_x^s = \min\bigl(f_s\, q_x,\ 1\bigr),$$

où q_x est lu dans la table du sexe si elle y figure, sinon dans la table `ensemble` (`loi_effective()`). Une table prospective (colonne `annee`) est refusée par le simulateur minimal. Seuls les sexes effectivement tirés (proportion d'hommes non nulle pour `H`, différente de 1 pour `F`) sont construits et contrôlés ; ils sont consignés dans `sexes_controles`.

Deux tables synthétiques, de statut `synthetique` et porteuses de leurs paramètres (attribut `parametres`), sont construites par le module :

— **Gompertz-Makeham** (`table_gompertz_makeham()`) : intensité $\mu(x) = A + B\,c^x$, avec $A \geq 0$, $B \geq 0$, $c > 1$ déclarés, sans valeur par défaut, sur des âges entiers déclarés ; la probabilité annuelle

est l'intégrale exacte de μ sur $[x, x+1)$:

$$q_x = 1 - \exp\Big(-A - B\,c^x\,\frac{c-1}{\ln c}\Big)\,;$$

la table est sans distinction de sexe (`ensemble`). Les tests emploient $A = 2 \cdot 10^{-4}$, $B = 2{,}5 \cdot 10^{-5}$, $c = 1{,}1$, âges 0 à 120 (*choix de conception* de la spécification, sans prétention de réalisme), qui donnent $q_{60} = 0{,}008153179142$ et $q_{100} = 0{,}303486068312$, et des facteurs $f_H = 1$, $f_F = 0{,}6$;

— **table constante** (`table_constante()`) : $q_x = q$ à tous les âges déclarés ; elle porte la loi de rachat ($q = 0{,}05$ dans les tests de portefeuille), ou une transition absente ($q = 0$). Une loi de rachat par ancienneté (axe secondaire) est hors du périmètre de la branche.

Hypothèse de répartition dans l'année

Le simulateur suppose une **force constante par âge entier**, $\mu_x = -\ln(1-q_x)$, dans l'échelle des années d'âge (une année d'âge vaut 1, quelle que soit sa durée de 365 ou 366 jours ; section 8.1), et des **risques concurrents indépendants**. L'hypothèse est consignée dans le résultat (`HYPOTHESE_REPARTITION`). Elle est retenue pour deux raisons :

— sous cette hypothèse, $q_x = 1 - e^{-\mu_x}$ *exactement* ; le décompte des événements rapporté à l'exposition centrale est alors l'estimateur du maximum de vraisemblance de μ_x du modèle à force constante par âge entier, avec troncature à gauche et censure indépendante, pour une observation en temps continu (résultat exact), et la cible des estimateurs à venir est connue exactement. Sur des dates au jour (section 7.1), l'exposition est arrêtée au début du jour de toute sortie, d'où un écart relatif d'ordre $m_x/(2L_x)$, où m_x est la somme des forces de la transition et des sorties concurrentes, négligeable mais non nul (forme exacte : section 9.2) ;

— la force constante est stable par addition des forces : une force constante dans chaque décrément équivaut à une force constante dans la table à décréments multiples, alors que la répartition uniforme n'a pas cette propriété. Deux forces indépendantes s'additionnent, et la probabilité de décès en présence du rachat a une forme close :

$$q_x^{(d)} = \frac{\mu_x^d}{\mu_x^d + \mu_x^r}\Big(1 - e^{-(\mu_x^d + \mu_x^r)}\Big)\,;$$

sous l'hypothèse de répartition uniforme dans chaque table à décrément simple, de taux q'^d et q'^r, la survie à double décrément $(1 - t\,q'^d)(1 - t\,q'^r)$ est quadratique en t dès que les deux taux sont strictement positifs, et la table à décréments multiples n'est plus uniforme : l'hypothèse ne serait plus la même pour les deux transitions (résultat exact, calcul direct).

Une probabilité $q_x = 1$ à un âge atteignable rend la force infinie : le simulateur la refuse (`qx_egal_a_un`).

16.2 Portefeuille simulé

Entrées

Le portefeuille est une liste dont tous les champs sont requis, sans valeur par défaut (`CHAMPS_PORTEFEUILLE`), selon deux modes :

— `uniforme` : `n` contrats, `proportion_hommes`, `age_min`, `age_max`, `origine` (date au plus égale au début de la période) ; le jour d'entrée est uniforme discret sur les jours de [origine, fin], l'âge à l'entrée uniforme continu sur [age_min, age_max), et la naissance est l'entrée moins $\mathrm{round}(\text{âge} \times 365{,}25)$ jours ; l'âge exact est ensuite recalculé depuis les dates, selon la convention d'âge du moteur (section 7.1) ;

— `age_exact` : `n`, `proportion_hommes`, `age` (entier), `date_entree` ; toutes les têtes entrent à l'âge exact `age`, née le même jour et le même mois, `age` années avant l'entrée. Ce mode sert au constat de simulation (section 16.3). Une entrée le 29 février est refusée (`entree_29_fevrier`) : la naissance « `age` années avant » serait ambiguë.

Le champ `mode` est une chaîne unique sans attribut, ni `NA` ni vide, au sens de `chaine_unique()` (une chaîne nommée ou munie d'une classe est refusée), prise parmi `uniforme` et `age_exact` ; le portefeuille porte exactement les champs de son mode, sans champ manquant, inconnu ni en double, et ses champs sont lus par leur nom exact, sans appariement partiel (un champ `mode_x` ne tient pas lieu de `mode`) ; sinon refus `portefeuille_invalide` (issue #60). Pour garantir une naissance strictement antérieure à l'entrée, il faut age_min $\times$ 365,25 ≥ 1 (la fonction `round()` de R arrondit au pair : $\mathrm{round}(0{,}5) = 0$) en mode uniforme, et un âge entier au moins égal à 1 en mode âge exact ; ces conditions sont contrôlées (`portefeuille_invalide`). La période d'observation est une liste nue, de champs exactement `debut` et `fin` (même contrôle `liste_aux_noms()` que la loi vraie ; refus `periode_invalide`), de deux dates (classe `Date`) uniques, non manquantes, finies et entières ; la censure a lieu à l'instant $B = \text{fin} + 1$ (le simulateur n'a pas de délai de déclaration).

Algorithme

1. Pour chaque transition et chaque tête, un **temps latent** est tiré par inversion du risque cumulé H, linéaire par morceaux sur l'âge (force constante par âge entier) : l'âge de l'événement est celui où H atteint $H(a_0) + E$, avec a_0 l'âge exact à l'entrée et $E \sim \mathrm{Exp}(1)$, cherché par `findInterval()` (`temps_latent()`).
2. La sortie est le **minimum** des deux temps latents et de la censure à B, comparés en temps continu ; une égalité, de probabilité nulle, est attribuée au décès.
3. L'instant de l'événement, dans l'année d'âge k à la fraction φ, est converti en jour : $\mathrm{anniv}(k) + \lfloor \varphi\, L_k \rfloor$, avec L_k la durée de l'année d'âge, borné à $[\text{entrée}, \mathrm{anniv}(k+1) - 1]$. Le jour reste dans l'année d'âge : **la probabilité d'un événement dans l'année k est conservée exactement** ; la discrétisation n'affecte que la répartition à l'intérieur de l'année, et la borne ne corrige que l'arrondi de la virgule flottante.

Ordre des tirages et graine

Les tirages ont lieu sous `sous_graine()`, avec la graine reçue en argument, sans défaut, sous `GENERATEUR` (section 3.3), dans un ordre fixé, qui fait partie du contrat de reproductibilité et est consigné (`ORDRE_TIRAGES`) : (1) sexe, `runif(n)` comparé à la proportion d'hommes ; (2) jour d'entrée, `sample.int()` avec remise ; (3) âge à l'entrée, `runif(n)` ; (4) décès, `rexp(n)` ; (5) rachat, `rexp(n)`. En mode âge exact, (2) et (3) ne sont pas tirés. Changer cet ordre change tout jeu simulé : c'est un changement de résultat.

Couverture de la table : aucune extrapolation

Le simulateur refuse (`table_trop_courte`) une loi qui ne couvre pas tous les âges atteignables, *avant tout tirage*, si bien que le refus ne dépend pas de la graine (`controler_portefeuille()`, `controler_couverture()`). Les âges atteignables sont bornés a priori par les seuls paramètres :

— **borne basse** : l'âge révolu à l'entrée est au moins

$$y_{\min} = \max\{y : 365\, y + \lceil y/4 \rceil \leq \mathrm{round}(\text{age_min} \times 365{,}25)\}$$

(`age_revolu_minimal()`), parce que y années comptent au plus $\lceil y/4 \rceil$ jours de 29 février ; la borne est prudente, les années séculaires non bissextiles ne faisant que retirer des jours. Elle donne $y_{\min} = 20$

pour age_min $= 20$ comme pour 21 : la borne $\lfloor$age_min$\rfloor$ serait fausse, une naissance arrondie au jour pouvant donner l'âge révolu 20 à une tête entrée à 21 ans (erratum A.2, point 4). En mode âge exact, la borne basse est l'âge déclaré ;

— **borne haute** : age_max $+$ $(B -$ origine$)/365{,}25 + 1$ en mode uniforme, et age $+$ $(B -$ date_entree$)/365{,}25 + 1$ en mode âge exact. *Preuve.* Soit $\mathrm{span}(y)$ le nombre de jours de la naissance à l'anniversaire y. Pour $y \leq 130$ (âges des tables des tests), $\mathrm{span}(y) \geq 365{,}25\,y - 2{,}75$ et $\mathrm{span}(y) \leq 365{,}25\,y + 0{,}75$; une tête d'âge révolu y à B vérifie $\mathrm{span}(y) \leq B - n$, et, en mode uniforme, $B - n \leq (B -$ origine$) +$ age_max $\times\, 365{,}25 + 0{,}5$; donc $y \leq$ age_max $+ (B -$ origine$)/365{,}25 + 3{,}25/365{,}25$, strictement inférieur à la borne. En mode âge exact, $B - n = (B -$ date_entree$) + \mathrm{span}($age$)$ et le même raisonnement donne une marge de $3{,}5/365{,}25$. *Statut des deux encadrements de* span : ils sont établis par calcul exhaustif, toutes naissances du 1er janvier 1700 au 31 décembre 2150 et $1 \leq y \leq 130$ (minimum $-2{,}75$, maximum $+0{,}75$, remesurés sur R 4.3.3 avec les fonctions de dates du module) ; le calendrier grégorien étant périodique de 400 ans, cette plage contient un cycle complet de dates de naissance, et les encadrements valent pour toute naissance (résultat exact). Au-delà de 130 ans, l'encadrement bas mesuré jusqu'à 260 ans est $-3{,}75$ (atteint pour $y = 207$, naissance au 2096-03-01), l'encadrement haut restant $+0{,}75$; la même preuve donne alors une marge de $4{,}25/365{,}25$ en mode uniforme ($4{,}5/365{,}25$ en mode âge exact), inférieure à un an pour $y \leq 260$.

Après les tirages, une assertion interne vérifie que les âges parcourus restent dans ces bornes : un écart serait une erreur de programmation, jamais un refus dépendant de la graine. Avec le portefeuille des tests (age_max $= 75$, origine 2005-01-01, fin 2024-12-31), la borne haute vaut 96 (erratum A.2, point 1), couverte par la table de 0 à 120 ans.

Sortie

`simuler_portefeuille()` rend un `data.frame` de colonnes `id_contrat`, `id_tete` (égal à `id_contrat`), `sexe`, `date_naissance`, `date_entree`, `date_sortie` (manquante si la tête est en cours à la fin) et `cause_sortie` (`deces`, `rachat` ou `en_cours`), qui portent les noms des rôles du catalogue. Toutes les lignes sont conservées, y compris celles sorties avant le début de la période, qui exercent la troncature à gauche. Les lignes sont cohérentes au sens exact, naissance $<$ entrée $\leq$ sortie $\leq$ fin, et passent les contrôles du module `donnees` sans exclusion, pour les deux transitions (constat des tests).

L'attribut `simulation` consigne : la loi vraie (tables, facteurs, loi effective par transition et par sexe avec q_x et μ_x, transitions, hypothèse de répartition, `sexes_controles`) ; le portefeuille et la période reçus ; la graine, `GENERATEUR` et l'ordre des tirages ; la version de R ; des décomptes (lignes, décès, rachats, en cours, sorties et entrées antérieures au début de la période).

Refus du simulateur : `loi_invalide`, `portefeuille_invalide`, `periode_invalide`, `entree_29_fevrier`, `table_trop_courte` et `qx_egal_a_un` ; et, par `sous_graine()`, `graine_invalide`.

16.3 Constats de simulation

Retrouver la loi vraie est une batterie de tests, chacun un *constat de simulation* qui écrit le nombre de têtes, le niveau, la région d'acceptation et la probabilité de fausse alarme (E2.19, E2.20). Le constat du simulateur minimal porte sur cinq cohortes de $n = 10^6$ têtes, entrées le 2016-03-15 à l'âge exact x (mode `age_exact`), période du 2016-03-15 au 2017-12-31 : l'année d'âge x est observée entière, sans censure par la période. On compte les événements D datés avant l'anniversaire $x + 1$. Sous le modèle simulé, D suit exactement la loi binomiale $\mathcal{B}(n, p)$, p étant la probabilité de l'événement dans l'année.

— **Région d'acceptation** : bilatérale exacte, $[\mathrm{qbinom}(\alpha_k/2, n, p), \mathrm{qbinom}(1 - \alpha_k/2, n, p)]$.

— **Correction multiple** : partage égal du niveau, justifié par l'inégalité de l'union, $\alpha = 10^{-3}$ global, $K = 5$, $\alpha_k = 2 \cdot 10^{-4}$.

— **Fausse alarme exacte** de chaque test : $FA_k = P(D < \text{lo}) + P(D > \text{hi})$, calculée et écrite par le test.

— **Fausse alarme familiale** : elle est bornée par $\sum_k FA_k = 9{,}857 \cdot 10^{-4} \leq 10^{-3}$ (inégalité de l'union, valable *sans hypothèse d'indépendance* ; c'est le critère vérifié). Sous indépendance des cinq cohortes, tirées sous des graines distinctes (20261016 à 20261020), elle vaut $1 - \prod_k (1 - FA_k) = 9{,}853 \cdot 10^{-4}$; cette indépendance est l'hypothèse usuelle sur le générateur, non démontrée, et la valeur n'est donnée qu'à titre d'information.

Table 29 : Constat de simulation du simulateur minimal : probabilité exacte p sous le modèle, région d'acceptation et fausse alarme exacte (`tests/unitaires/test_simulation.R`).

Cohorte	p	Région	FA_k
1. décès, $x = 60$, H, sans rachat	0,008153179142	[7 821 ; 8 490]	$1{,}951 \cdot 10^{-4}$
2. décès, $x = 80$, F (facteur 0,6), sans rachat	0,03150085056	[30 853 ; 32 152]	$1{,}981 \cdot 10^{-4}$
3. décès, $x = 100$, F, sans rachat	0,182091641	[180 658 ; 183 528]	$1{,}995 \cdot 10^{-4}$
4. rachat, $x = 40$, $q^r = 0{,}05$, sans décès	0,05	[49 191 ; 50 812]	$1{,}983 \cdot 10^{-4}$
5. décès, $x = 60$, H, rachat concurrent $q^r = 0{,}30$	0,006859320306	[6 554 ; 7 168]	$1{,}947 \cdot 10^{-4}$

La probabilité du test 5 est la forme close $q^{(d)}$ ci-dessus ; le rachat concurrent y est porté à 0,30 pour que le test soit sensible à son oubli (choix de la spécification). La graine étant fixe, chaque test est déterministe : sa fausse alarme est la probabilité, sur les graines, qu'une implémentation juste échoue. Les valeurs observées de D sont écrites par la batterie à chaque exécution ; elles ne sont pas reproduites ici (E3.4).

Statuts et sources d'erreur (E2.20).

— *Résultats exacts* : la formule de q_x de Gompertz-Makeham (intégrale de μ), la forme close $q^{(d)}$, la loi binomiale de D sous le modèle, les régions d'acceptation et les FA_k (quantiles et probabilités de la loi binomiale), l'inégalité de l'union, la fausse alarme familiale $1 - \prod_k (1 - FA_k)$, sous l'hypothèse d'indépendance des cohortes (hypothèse sur le générateur, non démontrée), la conservation de la probabilité annuelle par la conversion en jours, les bornes de couverture.

— *Résultat asymptotique* : aucun dans ce constat.

— *Approximations numériques* : l'évaluation en virgule flottante de q_x, de μ_x et des temps latents, des quantiles binomiaux et des FA_k ; le critère $\sum_k FA_k \leq 10^{-3}$ est vérifié sur les valeurs évaluées.

— *Constat de simulation* : le passage des cinq tests.

— *Erreur Monte-Carlo* : la dispersion binomiale de D autour de np, due au nombre fini de têtes simulées ; c'est elle que mesure la largeur de la région. *Erreur d'estimation* : elle n'intervient pas ici, D étant comparé à la probabilité exacte du modèle, sans taux estimé sur une exposition ; pour les taux bruts, elle est écrite par les constats de la section 9.9.

16.4 Vérifications

`tests/unitaires/test_simulation.R` vérifie : les valeurs des tables synthétiques (valeurs de la spécification, et recoupement par `stats::integrate()`, que le simulateur n'emploie pas) ; les dates d'anniversaire, 29 février compris (recoupement par `as.Date()` ; comparaison à `age_revolu()` du module `donnees`, écrite indépendamment ; l'accord des jours civils et des anniversaires du simulateur avec les arithmétiques des modules `donnees` et `exposition` est vérifié en outre par `tests/unitaires/test_exposition.R`, section 8.4) ; la cohérence exacte des lignes et l'âge révolu à l'entrée ; le passage du module `donnees` sans exclusion, pour les deux transitions ; l'identité des jeux à graine égale et la restauration de l'état du générateur de l'appelant ; chaque refus, dont `table_trop_courte` indépendant de la graine ; le constat ci-dessus.

tests/test_reproductibilite.R vérifie qu'un jeu simulé est identique, attribut compris, dans un autre processus R.

16.5 Fonctions du module

Table 30 : Définitions de premier niveau de R/simulation.R.

Nom	Statut	Entrées	Sortie et rôle
CAUSES_SIMULEES	constante	—	causes produites et transitions associées
HYPOTHESE_REPARTITION	constante	—	hypothèse consignée : force constante par âge entier, risques indépendants
ORDRE_TIRAGES	constante	—	ordre des tirages, consigné
CHAMPS_PORTEFEUILLE	constante	—	champs requis du portefeuille, par mode
table_gompertz_makeham()	interne	A, B, c, âges	table synthétique de Gompertz-Makeham
table_constante()	interne	q, âges, nom	table synthétique à q_x constant
loi_effective()	interne	loi d'une transition, cause, sexes	q_x^s et μ_x par sexe
jours_civils()	interne	année, mois, jour	jours depuis 1970-01-01 (arithmétique entière)
jour_anniversaire()	interne	composantes de naissance, âge	jour de l'anniversaire (29 février : 1er mars des années non bissextiles)
composantes_distinctes()	interne	jours	composantes de date, calculées sur les valeurs distinctes
age_anniversaire()	interne	composantes, année, jour	âge au dernier anniversaire
date_unique()	interne	valeur	date unique entière et finie
controler_portefeuille()	interne	portefeuille, période	refus des entrées ; bornes d'âge a priori
age_revolu_minimal()	interne	écart minimal en jours	borne basse $y_{\min}$
controler_couverture()	interne	loi effective, bornes, sexes	refus table_trop_courte, qx_egal_a_un
temps_latent()	interne	loi, composantes, âge et fraction à l'entrée, entrée, tirages E	instant et jour de l'événement latent
simuler_portefeuille()	exportée	loi, portefeuille, période, graine	jeu simulé et attribut simulation

Partie XIII

Index et traçabilité

Index et traçabilité

17.1 Index des fonctions exportées

Le tableau 31 recense les fonctions exportées de `tablexp` (directives `export` de `NAMESPACE` ; ADR 0007, point 3.4) : les points d'entrée et les fonctions que l'application appelle. Les autres fonctions sont internes ; elles figurent dans le tableau des fonctions de leur module.

Table 31 : Fonctions exportées de `tablexp`.

Fonction	Module	Rôle	Section
`catalogue_roles()`	`donnees`	catalogue fermé des rôles, lu par l'application pour la déclaration des rôles	7.2
`table_reference()`	`donnees`	constructeur contrôlé d'une table de référence, appelé par l'application pour l'import de tables	7.5
`simuler_portefeuille()`	`simulation`	point d'entrée du simulateur : jeu simulé sous une loi vraie et une graine	16
`valider_plan()`	`construire_table`	validateur du plan, contrôle de saisie que l'application appellera (E4.3)	6.1
`construire_table()`	`construire_table`	point d'entrée unique : exécute un plan de construction et retourne l'objet `construction`	6
`rejouer_fiche()`	`construire_table`	rejeu outillé d'une fiche (E3.7) : contrôles, un appel de `construire_table()`, bilan de rejeu	6.4
`ecrire_plan()`	`rapport`	texte du plan en octets UTF-8, pour l'export du plan (E5.4) ; l'appelant écrit le fichier	15.4
`lire_plan()`	`rapport`	plan relu d'octets, sans évaluation, pour l'import du plan (E5.4) ; non validé	15.4
`ecrire_fiche()`	`rapport`	texte de la fiche de construction en octets UTF-8 ; l'appelant écrit le fichier	15.4
`lire_fiche()`	`rapport`	fiche relue d'octets, sans évaluation, pour le rejeu (E3.7)	15.4

17.2 Traçabilité des exigences

Le tableau 32 relie les exigences du cahier des charges (`docs/exigences.md`) aux sections qui les décrivent. Une exigence qui n'y figure pas n'est pas encore mise en œuvre (méthodes de lissage, de positionnement, de fermeture et de validation, rapport figé, interface, tables de référence embarquées) ; une exigence mise en œuvre en partie renvoie aux sections de la partie réalisée ; le tableau est complété avec chaque partie rédigée.

Table 32 : Exigences et sections qui les décrivent.

Exigence	Objet	Section
E1.1	construction d'une table d'expérience, estimation d'une transition	Objet du document ; 6
E1.3	rigueur, traçabilité, reproductibilité	3, 4, 15.1
E1.5	aucune donnée réelle : jeux simulés sous une loi vraie et une graine	16
E2.2	catalogue fermé des rôles, lecture d'une colonne par son rôle	7.2
E2.3	recette la plus fine, nommée, refus si aucune ; convention d'âge, forçage plus grossier ; conventions de temps communes	7.1, 8.1
E2.4	unité d'observation, fusion par tête, incohérences entre contrats	7.4
E2.5	période déclarée, délai de déclaration	7.1, 8.2
E2.6	troncature à gauche, censure à droite, censure non informative nommée	8.2
E2.7	refus par défaut, motifs en liste fermée, exclusions déclarées, données inchangées	7.3
E2.8	pondération par les montants : rôle refusé tant que ses intervalles ne sont pas spécifiés	7.2
E2.9	segmentation déclarée, vide explicite admis (le test d'homogénéité viendra avec la validation)	6.1, 8.2
E2.11	noyau multi-états : une table estime une transition ; seconde transition couverte par les tests	6.1, 16
E2.12	taux bruts : estimateurs de Poisson, binomial et produit-limite avec troncature à gauche, hypothèses de répartition dans la cellule (en partie : les autres méthodes de la v1 viendront avec leurs modules)	9
E2.14	intervalles exacts et asymptotiques des taux bruts, avec leur statut (le bootstrap par têtes viendra avec les tables lissées et positionnées)	9.2, 9.3, 9.4, 9.5
E2.19	simulateur : loi vraie, entrées, sorties concurrentes, censure, constat de simulation ; retrouver la loi vraie par les taux bruts	16, 9.9
E2.20	quatre statuts, erreur Monte-Carlo et erreur d'estimation	3.1, 8.1, 16.3, 9.5, 9.9
E3.1	document LaTeX unique, une partie par module, charte d'`outil_USP`, tableaux de fonctions	ensemble du document
E3.2	fiche de chaque méthode : objet, hypothèses, estimateur, fonction, statut, références ; fiche de chaque vérification	9.2, 9.3, 9.4, 9.9
E3.3	test de concordance entre la documentation et le code	4.1
E3.4	aucun chiffre issu d'une exécution	Objet du document
E3.6	empreintes des données reçues, du fichier, du résultat et du code ; dépendance de l'empreinte du résultat à la plateforme (en partie : le rapport figé viendra avec son module)	15.2, 15.3
E3.7	relecture sans le code, rejeu à partir de la fiche, rejeu outillé	15.1, 15.4, 6.4
E4.1	R 4.3.3, package `tablexp`, point d'entrée unique, modules	1.2, 6
E4.2	packages de priorité `base` ; aucune dépendance à Java	5.1, 5.2

Exigence	Objet	Section
E4.3	frontière calcul / affichage ; contrôles de saisie dans le package	1.1, 6.1
E4.4	le moteur reçoit un `data.frame` et des rôles, refuse un type incompatible	1.1, 7.2
E4.5	plan exécuté d'un bloc, objet complet, contenu de la fiche	6, 15.1
E4.6	reproductibilité, graine locale, générateur fixé, aucun horodatage comparé	3, 6.3
E4.7	références de non-régression sur la plateforme de référence	4.3, 4.5
E4.9	un module par famille, index des fonctions contrôlé par la concordance	1, 17.1
E5.4	export et import du plan, écriture de la fiche (fonctions du moteur ; l'interface viendra avec l'application)	15.4
E5.5	messages qui disent ce qui manque, sans rien corriger	2.1, 7.3

17.3 Références

— M. Matsumoto, T. Nishimura, *Mersenne Twister : a 623-dimensionally equidistributed uniform pseudo-random number generator*, *ACM Transactions on Modeling and Computer Simulation*, 8, 1998, p. 3–30 (cité par la documentation de R, `?Random`) ; section 3.2.

— R Core Team, *R : A Language and Environment for Statistical Computing*, R Foundation for Statistical Computing, Vienne, 2024 (`https://www.R-project.org/` ; référence rendue par `citation()` sous R 4.3.3).